

Introducción a Godot

1. Preámbulo.
2. Bases de GDScript
3. Objetos (mallas)
4. Transformaciones geométricas
5. Transformaciones de proyección
6. Iluminación
7. Texturas
8. Selección



1. Preámbulo

- ! Godot es un motor que permite realizar juegos tanto 2D como 3D.
- ! Creado originalmente por Juan Linietsky y Ariel Manzur
- ! Código abierto en 2014
- ! Interfaz (API) con el Hardware gráfico
- ! Existen APIs para distintos lenguajes de programación
- ! Existen implementaciones para diversas plataformas (Windows, Apple, Linux).
- ! Utiliza las capacidades de aceleración de las tarjetas gráficas (GPUs)
- ! Desde la versión 4.0 usa Vulkan (anteriormente OpenGL)
- ! Godot Web site:
<https://godotengine.org/>

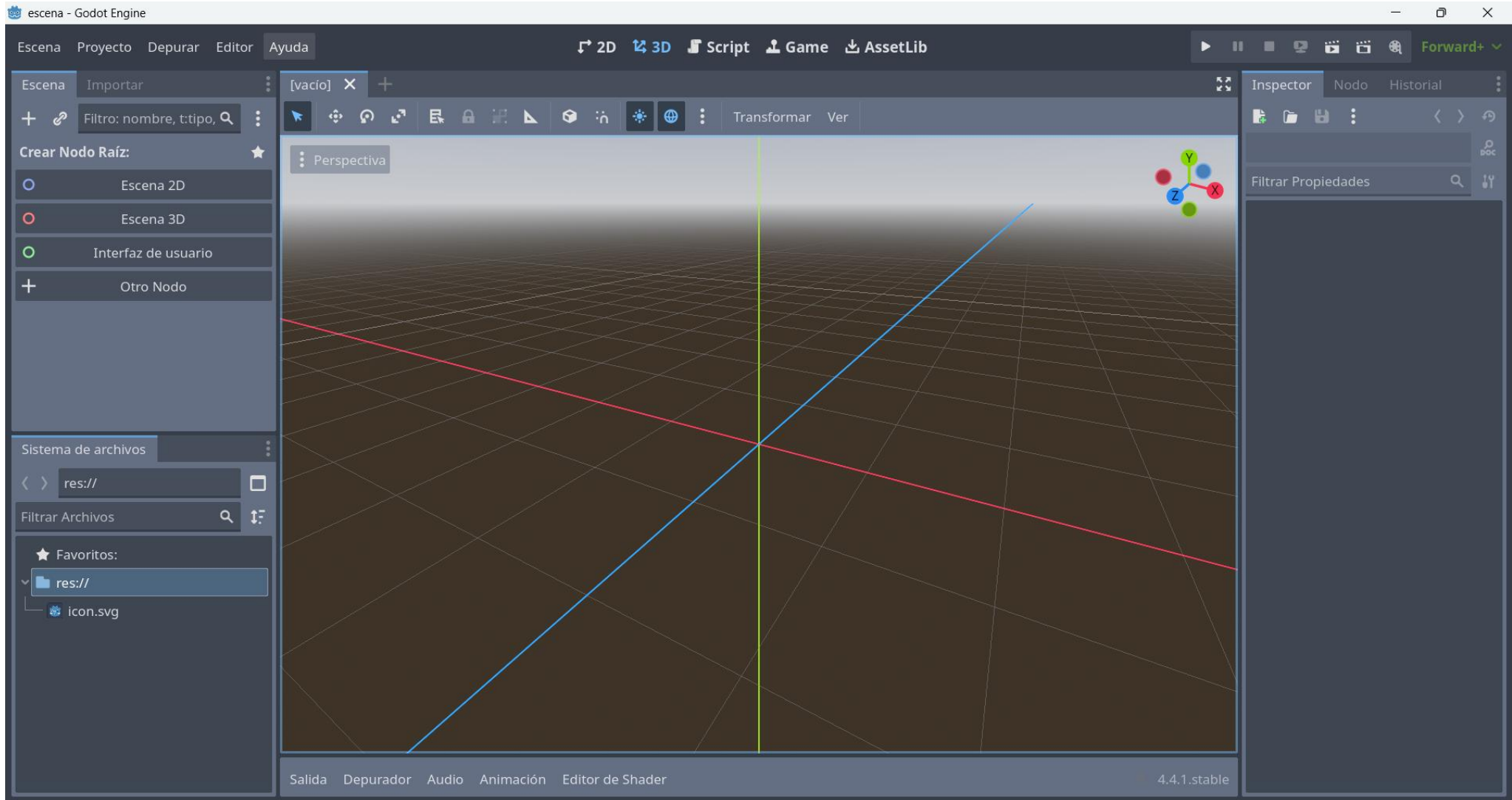


1. Preámbulo

- ! Vulkan, librería gráfica con especificación 1.0 febrero de 2016
- ! Última versión 1.4 diciembre de 2024

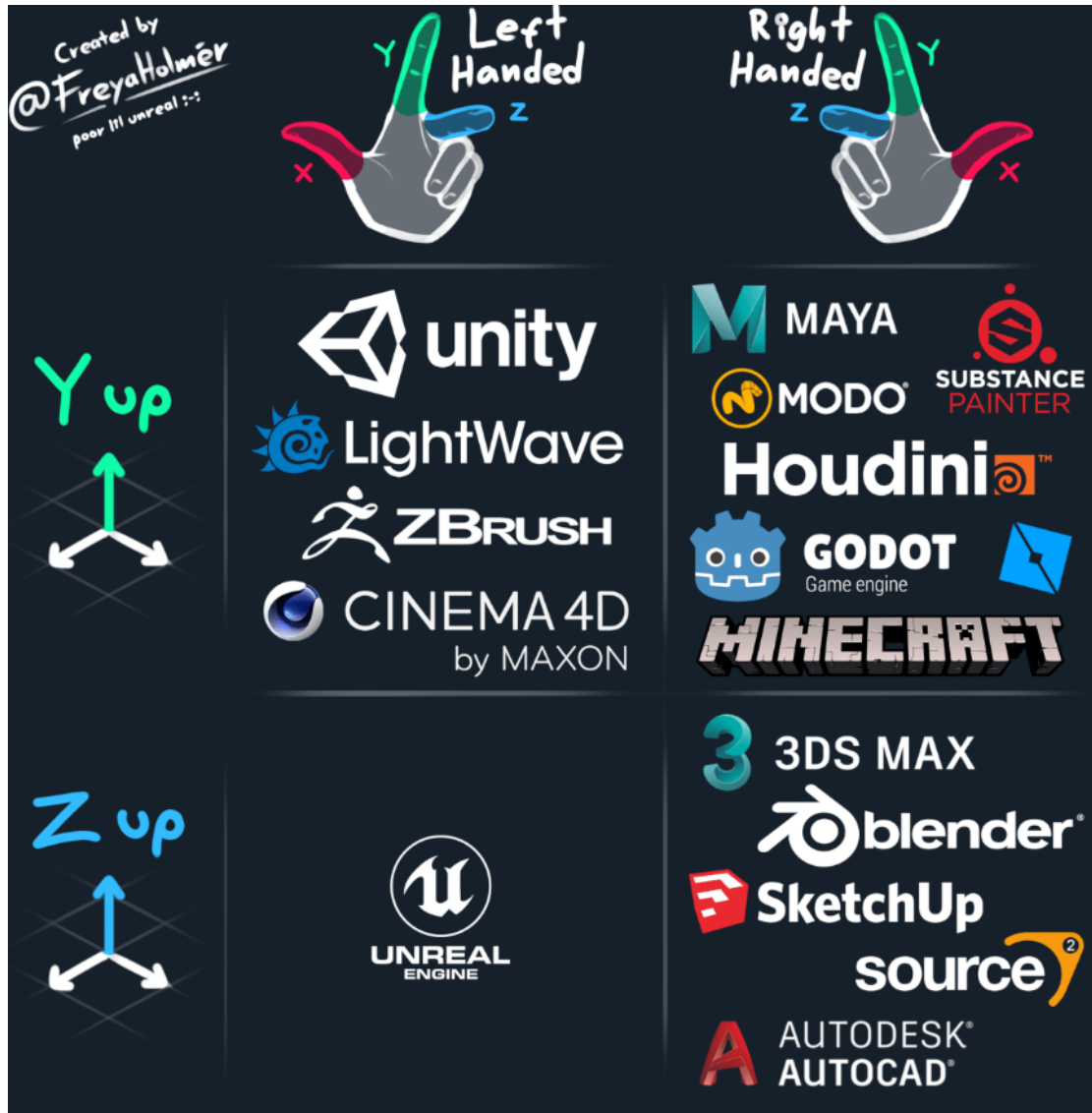


1. Preámbulo. Interfaz



Introducción a Godot

1. Preámbulo. Sistema coordenadas 3D

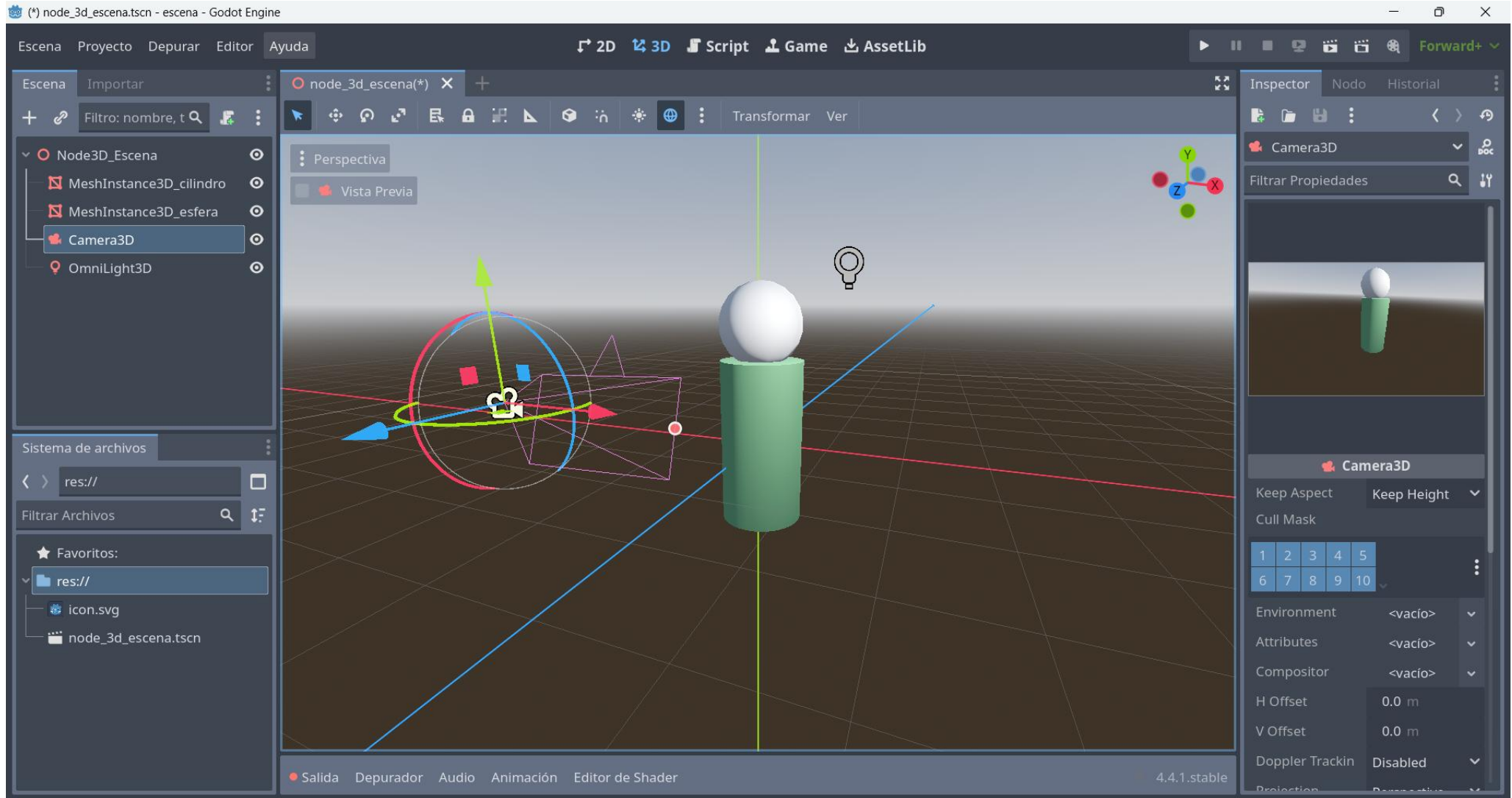


Fuente: <https://twitter.com/FreyaHolmer>



Introducción a Godot

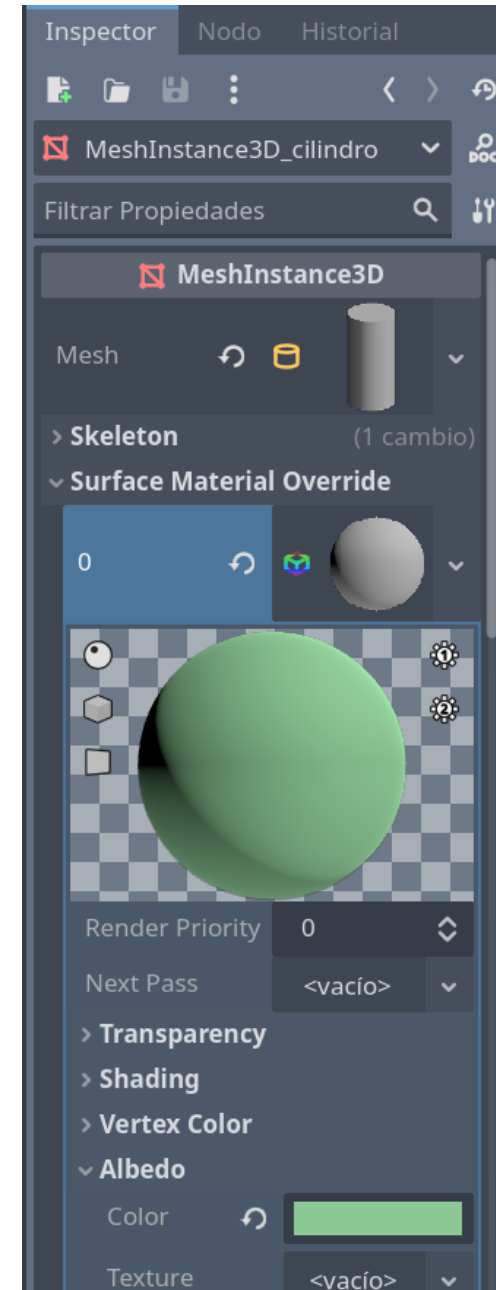
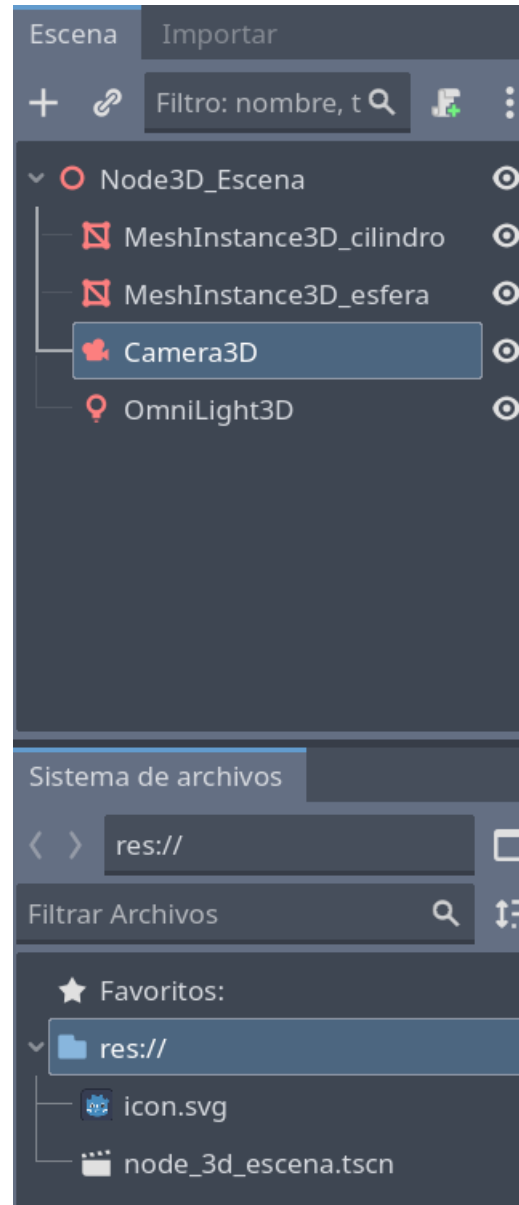
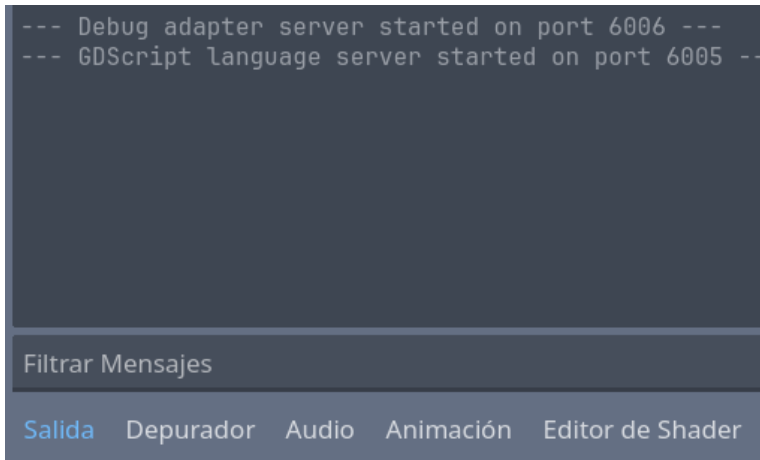
1. Preámbulo. Escena: objetos, cámaras, luces



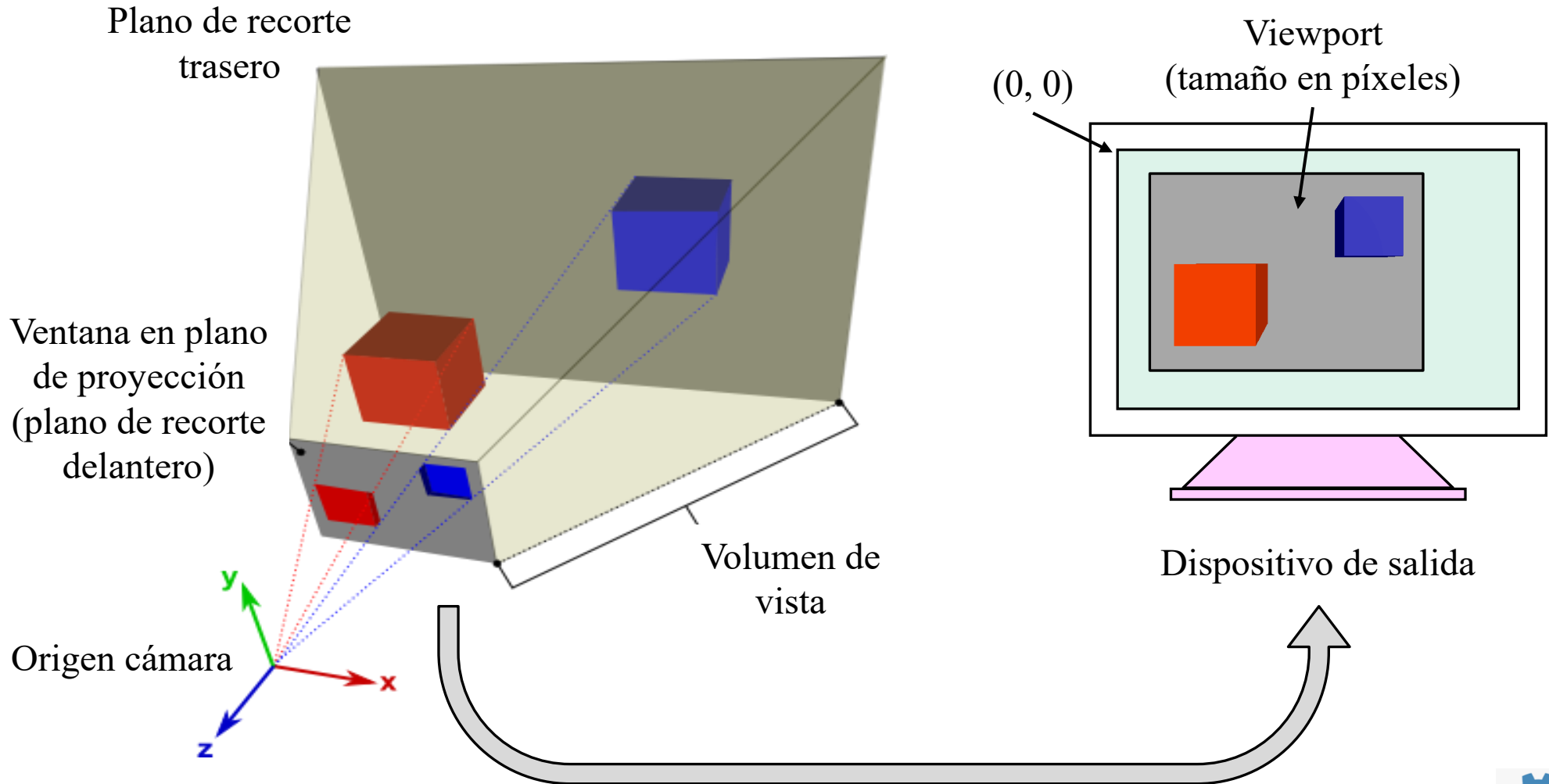
Introducción a Godot

1. Preámbulo

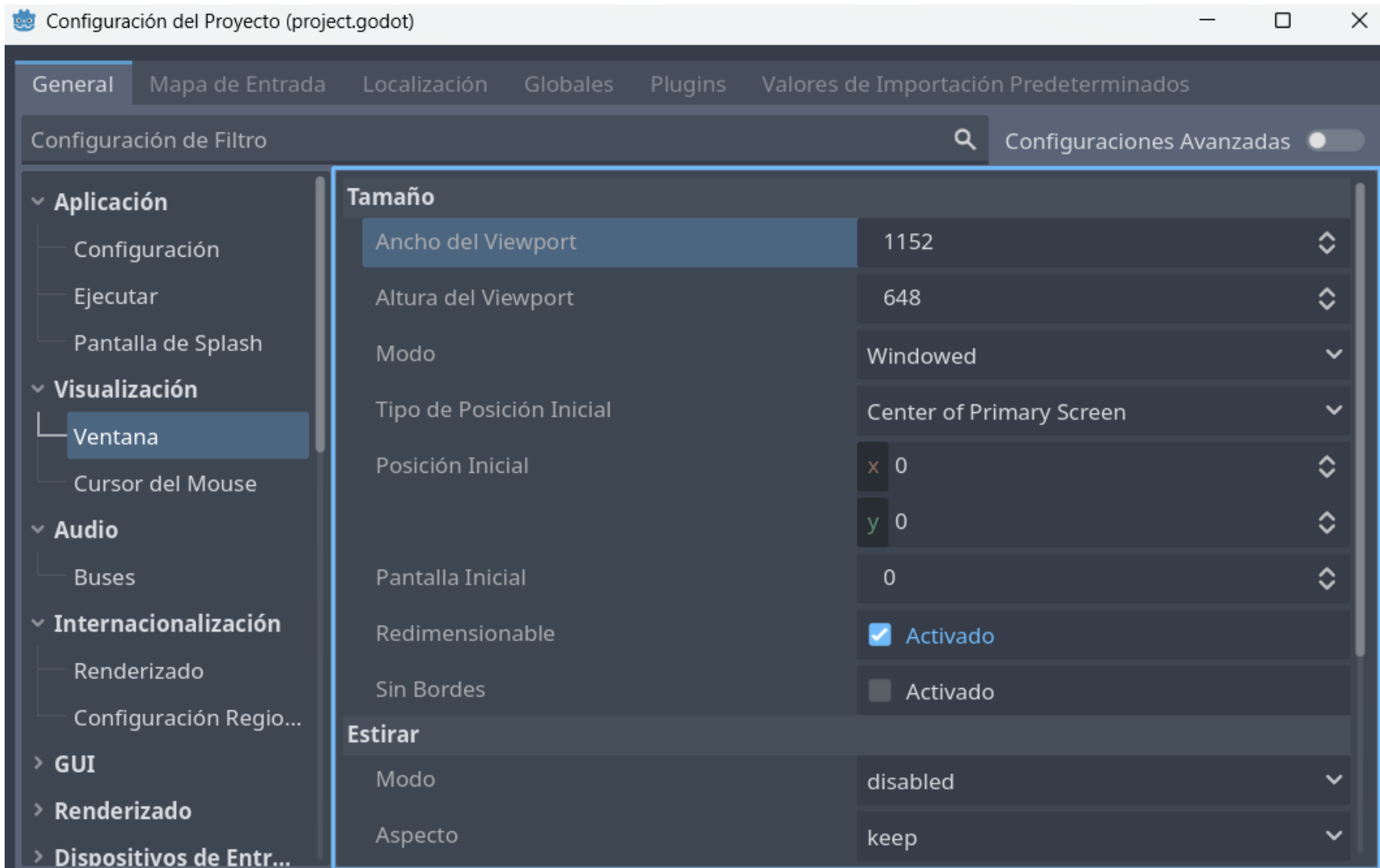
! Paneles



1. Preámbulo. Volumen de vista, ventana y viewport

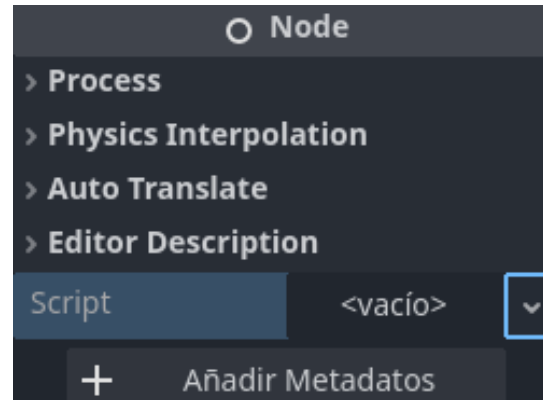


1. Preámbulo. Cambiando el viewport



2. Bases de GDScript

! Dado un nodo se crea un script con en **Inspector**



! Plantilla por defecto

```
1  extends MeshInstance3D
2  # Called when the node enters the scene tree for the first time.
3
4  func _ready() -> void:
5      pass # Replace with function body.
6
7  # Called every frame. 'delta' is the elapsed time since the previous frame.
8  func _process(delta: float) -> void:
9      pass
```



2. Bases de GDScript

! Comentarios

```
# Coordenadas de la base (cuadrado centrado en el origen, lado 1)  
var p1 = Vector3(-0.5, 0, -0.5)
```

```
""" Coordenadas de la base (cuadrado centrado en el origen, lado 1)  
var p1 = Vector3(-0.5, 0, -0.5)  
"""
```

! Bloques (tabulación) y fin de línea

```
4  ▸ func _ready():  
5    ▸ var delta:=0.1  
6    ▸ _process(delta)  
7  
8  ▸ func _process(delta):  
9    ▸ var angulo := 0.0
```



2. Bases de GDScript

! Variables

- Tipado dinámico

```
var aa = "texto"  
...  
func _ready():  
    aa = 5  
    print(aa)
```

- Tipado estático

```
var aa : String = "texto" # ó  
var aa := "texto"  
  
var bb : float = 1.5 # ó  
var bb := 1.5  
  
var cc : int = 1 # ó  
var cc := 1
```



2. Bases de GDScript

! Tipos de datos básicos

- int, float, bool, String, null

! Tipos de datos compuestos

- Color

```
# colores entre 0.0 y 1.0
```

```
var azul_medio : Color = Color (0.5, 0.5, 1.0) # ó
```

```
var azul_medio = Color (0.5, 0.5, 1.0)
```

```
# existen colores ya definidos
```

```
var color_figura : Color = Color. BLUE_VIOLET
```

```
# colores entre 0 y 255
```

```
var color_figura : Color = Color. from_rgba8 (200, 200, 100) # ó
```

```
var color_figura = Color. from_rgba8 (200, 200, 100)
```

```
# colores H (hue o tono), S (saturación) y V (valor o brillo)
```

```
var color_figura : Color = Color. from_hsv (0.2, 0.5, 1.0) # ó
```

```
var color_figura = Color. from_hsv (0.2, 0.5, 1.0)
```



2. Bases de GDScript

! Tipos de datos compuestos

- **Vector3, Vector3i**

```
var punto1 = Vector3 (1.25, 10.5, 20.0)
```

```
func _ready():  
    print(punto1)  
    print(punto1.x)      # acceder a una sola componente  
    print(punto1.y)  
    print(punto1.z)
```

```
var punto2 = Vector3i (1, 10, 20)
```

- **Plane** (normal, distancia desde origen según la normal) o **Plane** (tres puntos)

```
var plano = Plane (Vector3(1.0, 0.0, 0.0), 4)
```

```
var plano = Plane (Vector3(3.0,4.0,0.0), Vector3(6.0,0.0,1.0), Vector3(0.0,0.0,-1.0))
```



2. Bases de GDScript

! Tipos de datos compuestos

- AABB (origen, tamaño (ancho, alto, fondo)) (caja frontera, tamaño toma valores positivos)

```
var caja = AABB (Vector3(0, 0, -1), Vector3(4, 4, 2))
```

- Array

```
var array1 := Array [4.0, -0.6, 10.4, -27.3] # ó  
var array1 = [4.0, -0.6, 10.4, -27.3]  
...  
print (array1[0]) # acceder a un elemento
```

```
var array2 = [Vector3i (1, 10, 20), "texto", true, 56.4]
```

```
var array3 = [ ] # vacío
```



2. Bases de GDScript

- ! Tipos de datos compuestos
 - o **Array**. Operaciones

```
var array1 = [4.0, -0.6, 10.4, -27.3]
...
array1.insert(3, 5.8)  # posición, elemento
...
array1.remove_at(2)   # elimina un elemento
...
array1.append(5.6)    # añade un elemento por final
...
array1.push_back(-7.8) # también añade un elemento por final
```

```
var array1 = [4.0, -0.6, 10.4, -27.3]
var array2 = [6.9, -20.2, 15.7]
...
array1.append_array(array2) # unir dos arrays
```



2. Bases de GDScript

! Variables **export**.

! Se modifican con el **Inspector**

```
@export var altura: float = 1.5  
@export var flag: bool
```

```
@export_range (-10,10,0.5) var rango = 1.0
```



2. Bases de GDScript

! Condiciones `if`.

```
if (condición1):  
    ...  
elif (condición2): # equivale a else if  
    ...  
else:  
    ...
```

```
if (condición1 and condición2) or (condición3):  
    ...  
else:  
    ...
```



2. Bases de GDScript

! Ciclos `for`, `while`.

```
var n=10  
for i in n:  
    ...
```

```
for i in range (1, 10):  
    ...
```

```
for i in range (1, 10, 2):  
    ...
```

```
var array = [4.0, -0.6, 10.4, -27.3]  
for i in array    # equivale a array.size()  
    ...
```

```
var i = 0  
while i<10 # bucle mientras se cumpla una condición  
    ...  
    i += 1
```



2. Bases de GDScript

! Funciones **func**.

```
func funcion (param1, param2, ..): # devuelve void  
    ...
```

```
func funcion (param1, param2, ..):  
    ...  
    return dato      # devuelve un valor con tipado dinámico
```

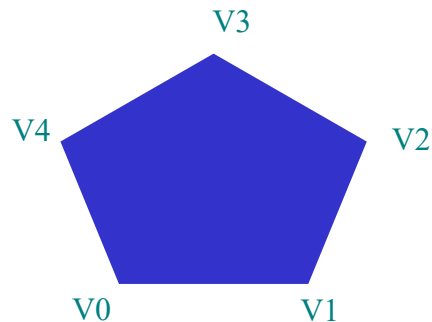
```
func funcion (param1, param2, ..) -> float:  
    ...  
    return dato      # devuelve un valor con tipado estático
```

```
func funcion (param1: int, param2: float, ..) -> array:  
    ...  
    return [dato1, dato2, ...] # parámetros con tipado estático
```

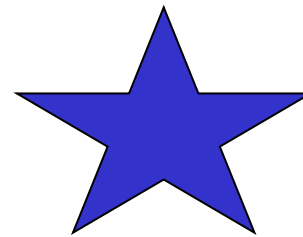


3. Objetos (mallas)

- ! Un objeto ha de estar formado por polígonos o caras planas
- ! Se pueden crear polígonos convexos y cóncavos.
- ! Pero lo normal es usar objetos definidos por mallas de triángulos
- ! El sentido positivo es el antihorario pero en mallas de triángulos se crean éstos en sentido horario



Polígono
convexo



Polígono
cóncavo

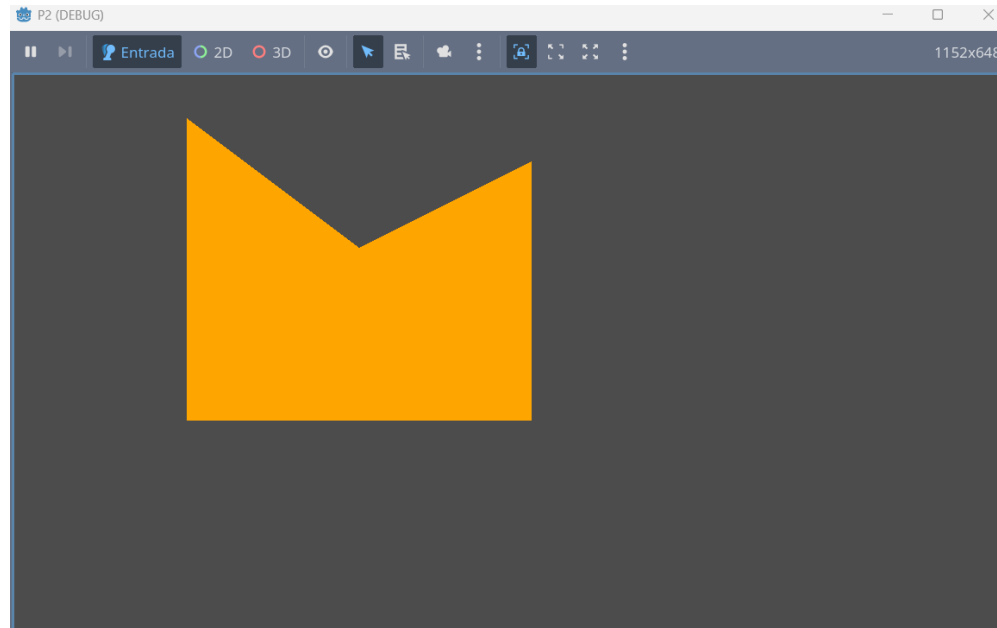


3. Objetos (mallas)

```
extends Node2D
```

```
func _ready() -> void:  
    _draw()
```

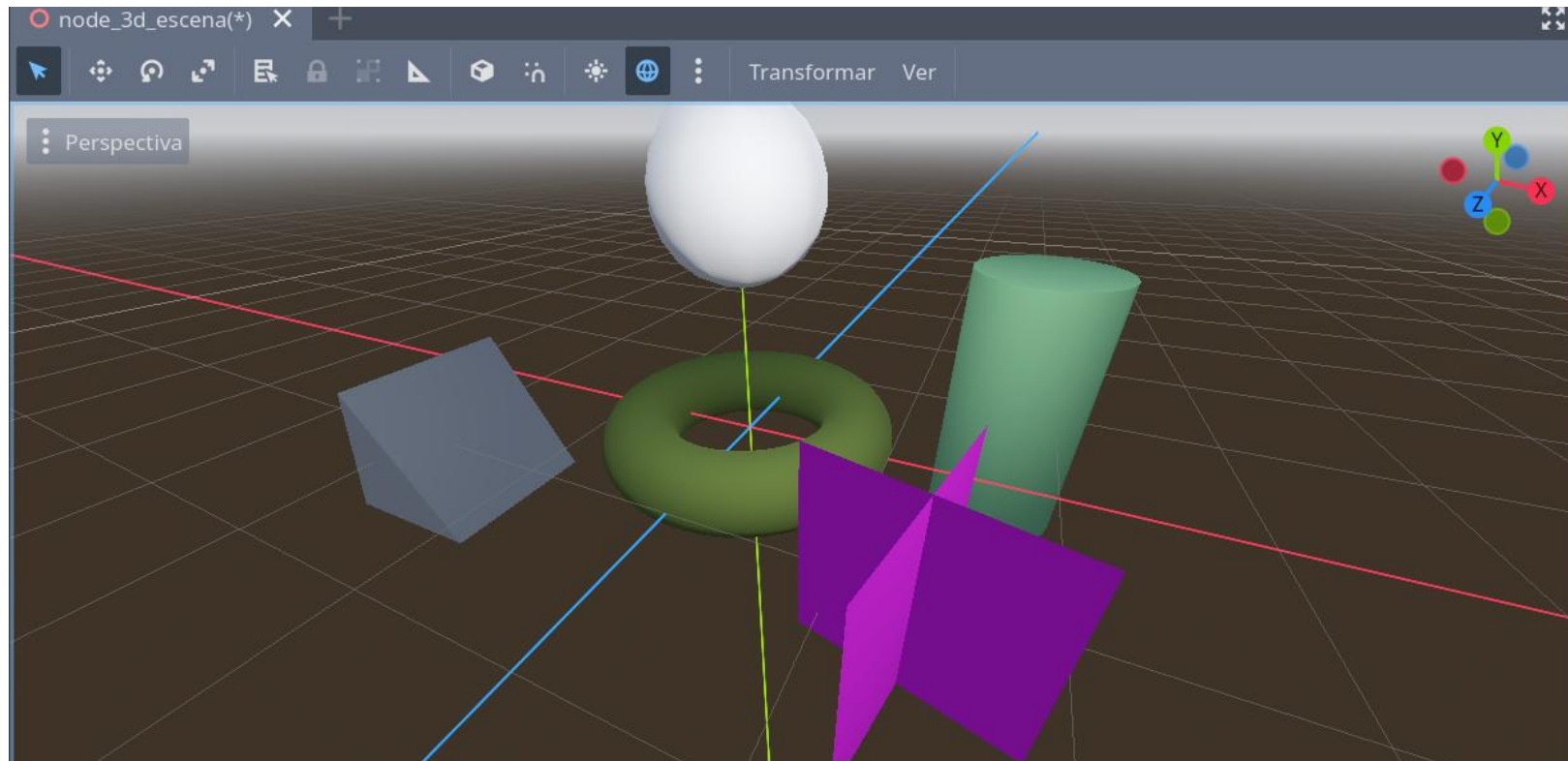
```
func _draw():  
    var puntos = [Vector2(600, 400), Vector2(600, 100),  
                  Vector2(400, 200), Vector2(200, 50), Vector2(200, 400)]  
    draw_colored_polygon(puntos, Color.ORANGE)
```



3. Objetos (mallas)

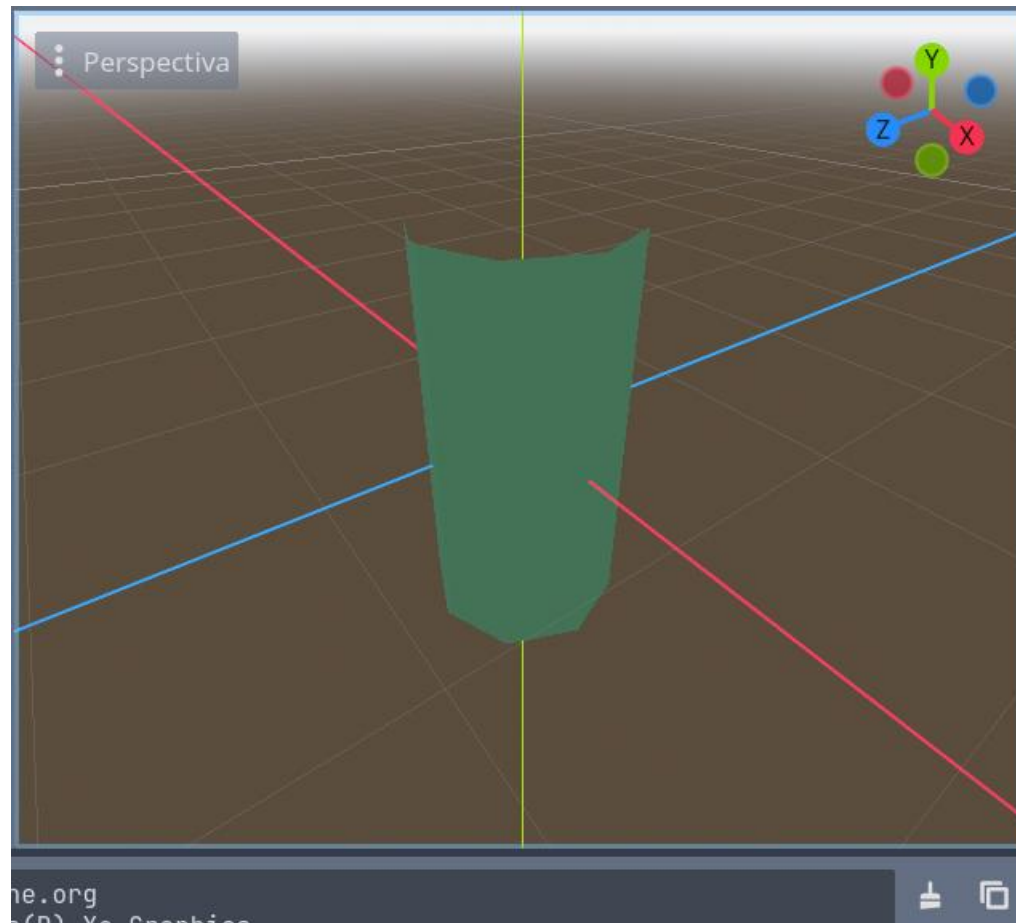
! Se crean objetos 3d usando **MeshInstance3D**, puede contener:

- **PrimitiveMesh** (caja, cápsula, plano, cilindro, esfera, prisma, toroide, texto).
- **ArrayMesh** (mallas generadas por código)



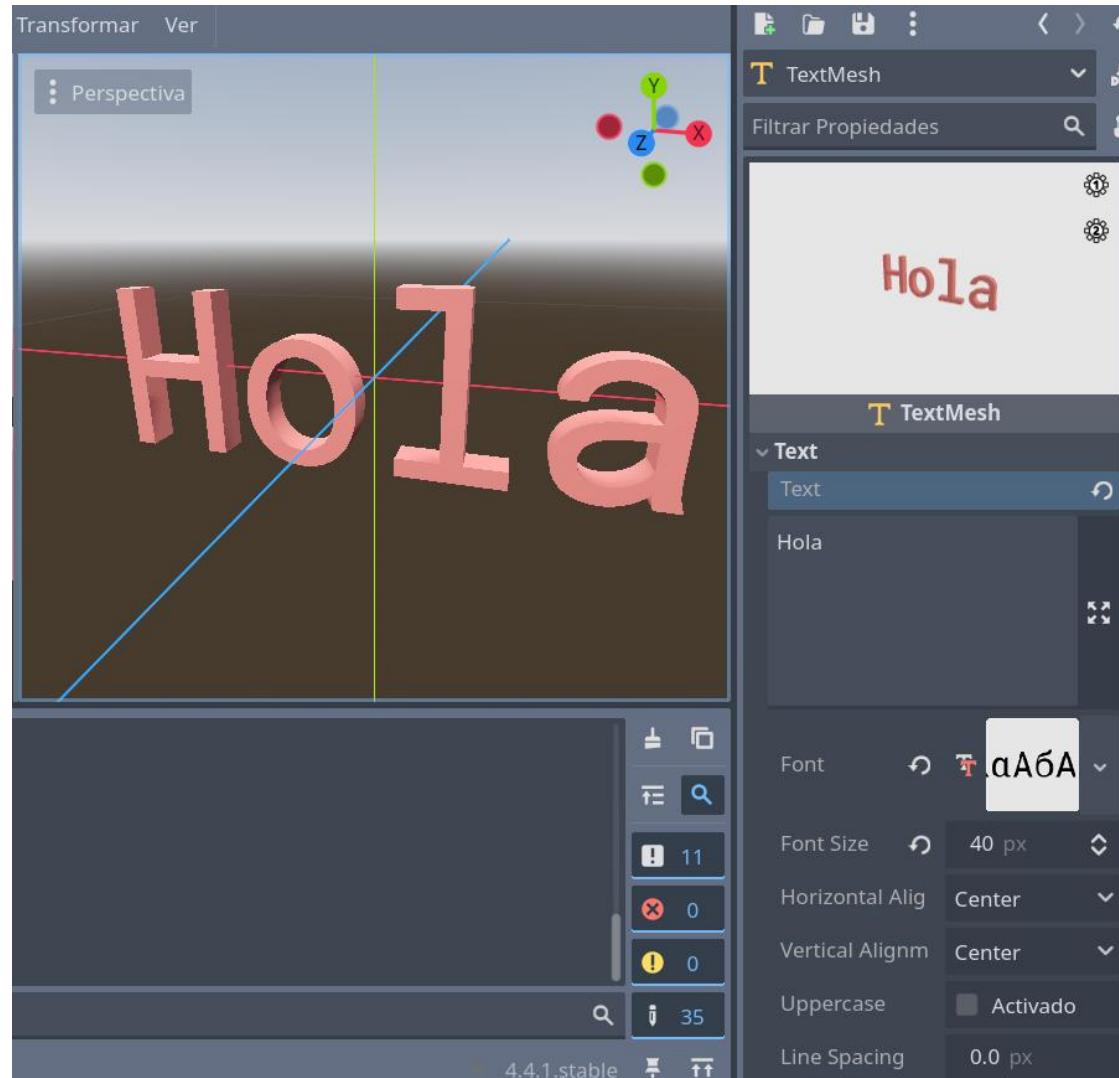
3. Objetos (mallas)

! Cambiando parámetros de **PrimitiveMesh**



3. Objetos (mallas)

! Texto **PrimitiveMesh** (usa fuentes TTF/OTF)



3. Objetos (mallas)

! Clase **SurfaceTool**. Permite crear mallas a partir de vértices y definir características de éstas

! Métodos:

- **begin**(tipo_primitiva). **Mesh.PRIMITIVE_POINTS**, **Mesh.PRIMITIVE_LINES**, **Mesh.PRIMITIVE_LINES_STRIP**, **Mesh.PRIMITIVE_TRIANGLE**, **Mesh.PRIMITIVE_TRIANGLE_STRIP**
- **add_vertex** (Vector3). Geometría de la malla
- **set_color** (Color)
- **set_normal** (Vector3D). Fijar la normal a un vértice
- **generate_normals**(). Calcula las normales
- **set_uv** (Vector2). Coordenadas de textura para un vértice
- **commit**(). Construye un **ArrayMesh**



3. Objetos (mallas)

! Malla creada con **ArrayMesh**. Lista de triángulos aislados (ejemplo octaedro)

```
extends Node3D # se crea en un node_3D
```

```
@export var v_superior: float = 1.0
```

```
@export var v_inferior: float = -1.0
```

```
@export var lado: float = 1.0
```

```
func _ready():
```

```
    var octaedro = MeshInstance3D.new()
```

```
    var mesh = ArrayMesh.new()
```

```
    var st = SurfaceTool.new()
```

```
    # Coordenadas de la figura
```

```
    var p1 = Vector3(-lado/2.0, 0, -lado/2.0)
```

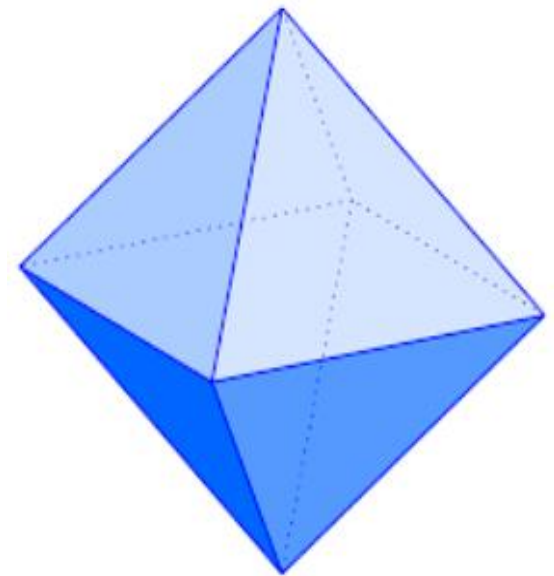
```
    var p2 = Vector3( lado/2.0, 0, -lado/2.0)
```

```
    var p3 = Vector3( lado/2.0, 0, lado/2.0)
```

```
    var p4 = Vector3(-lado/2.0, 0, lado/2.0)
```

```
    var p5 = Vector3(0, v_superior, 0)
```

```
    var p6 = Vector3(0, v_inferior, 0)
```



3. Objetos (mallas)

! Malla creada con **ArrayMesh**. Lista de triángulos aislados (ejemplo octaedro, continuación)

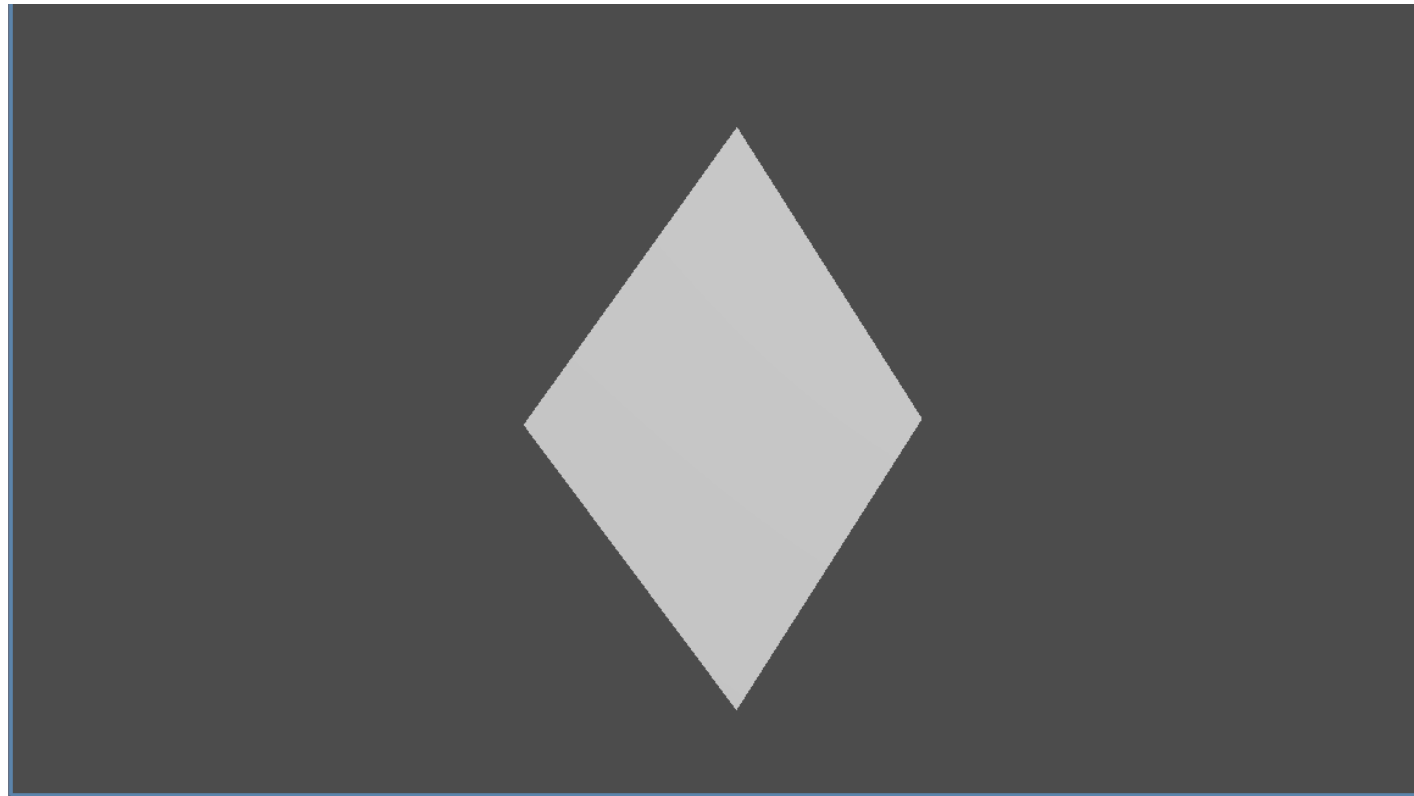
```
func _ready():  
    ...  
    # se indica triángulos sólidos  
    st.begin(Mesh.PRIMITIVE_TRIANGLES)  
    # parte superior  
    st.add_vertex(p1)  
    st.add_vertex(p2)  
    st.add_vertex(p5)  
  
    st.add_vertex(p2)  
    st.add_vertex(p3)  
    st.add_vertex(p5)  
  
    st.add_vertex(p3)  
    st.add_vertex(p4)  
    st.add_vertex(p5)  
  
    st.add_vertex(p4)  
    st.add_vertex(p1)  
    st.add_vertex(p5)  
    # parte inferior  
    st.add_vertex(p2)  
    st.add_vertex(p1)  
    st.add_vertex(p6)  
  
    st.add_vertex(p3)  
    st.add_vertex(p2)  
    st.add_vertex(p6)  
  
    st.add_vertex(p4)  
    st.add_vertex(p3)  
    st.add_vertex(p6)  
  
    st.add_vertex(p1)  
    st.add_vertex(p4)  
    st.add_vertex(p6)  
  
    st.commit(mesh)  
    octaedro.mesh = mesh  
    add_child(octaedro)
```



3. Objetos (mallas)

! Creada con [ArrayMesh](#). Lista de triángulos aislados (ejemplo octaedro, continuación)

Resultado por
defecto



3. Objetos (mallas)

! Clase **StandardMaterial3D**. Permite asignar materiales a una malla, como color base, color en cada vértice, reflexión difusa y especular (brillo), transparencia, emisión, etc.

! Propiedades:

- **albedo_color** color intrínseco o base de toda la figura.
- **vertex_color_use_as_albedo** bool
- **metallic** valor entre [0, 1], 0 reflexión difusa, 1 reflexión especular
- **roughness** valor entre [0, 1], 0 muy brillante, 1 mate
- **transparent** añade el canal alfa
- **material_override** asigna el material a un objeto **MeshInstance3D**



3. Objetos (mallas)

! Malla creada con **ArrayMesh**. Lista de triángulos aislados

! Añadiendo color a cada cara, **StandardMaterial3D** y **set_color**

```
...  
func _ready():  
    var octaedro = MeshInstance3D.new()  
    var mesh = ArrayMesh.new()  
    var st = SurfaceTool.new()  
    var material = StandardMaterial3D.new()  
  
    # Se indica un color para cada vértice  
    material.vertex_color_use_as_albedo = true  
  
    # Coordenadas de la figura  
    ...  
    # se indica triángulos sólidos  
    st.begin(Mesh.PRIMITIVE_TRIANGLES)
```

parte superior

```
st.set_color(Color(0.0, 0.0, 0.5))  
st.add_vertex(p1)  
st.add_vertex(p2)  
st.add_vertex(p5)
```

```
st.set_color(Color(1.0, 0.0, 0.0))  
st.add_vertex(p2)  
st.add_vertex(p3)  
st.add_vertex(p5)
```

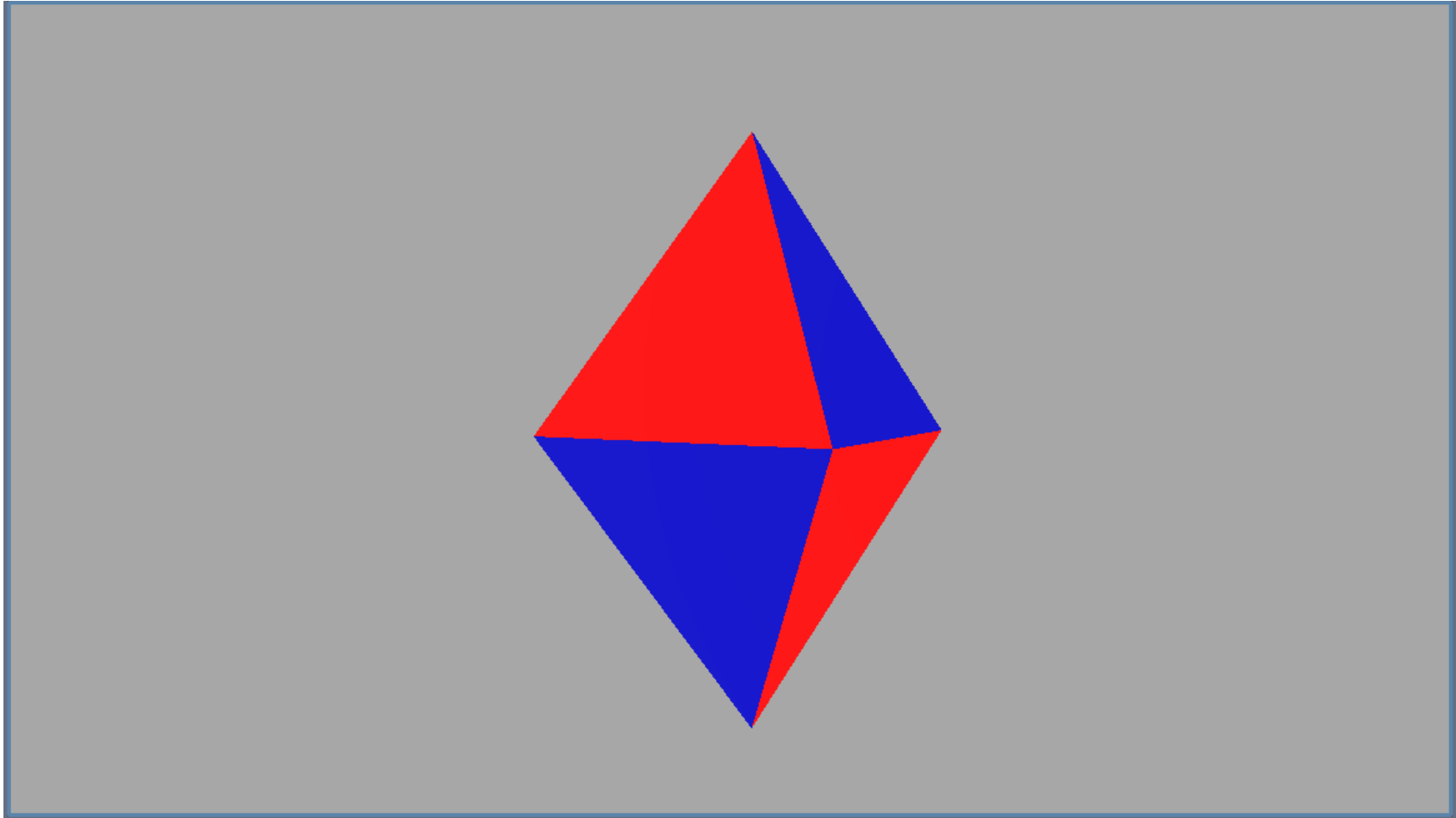
...

```
mesh=st.commit()  
octaedro.mesh = mesh  
octaedro.material_override=material  
add_child(octaedro)
```



3. Objetos (mallas)

- ! Malla creada con `ArrayMesh`. Lista de triángulos aislados
- ! Añadiendo color a cada cara, `StandardMaterial3D` y `set_color`



3. Objetos (mallas)

! Malla creada con **ArrayMesh**. Lista de triángulos aislados

! Añadiendo color a cada cara, **StandardMaterial3D** y **set_color**

! Un color distinto en cada vértice

...

```
st.set_color(Color(1.0, 0.0, 0.0))
```

```
st.add_vertex(p2)
```

```
st.set_color(Color(1.0, 1.0, 0.0))
```

```
st.add_vertex(p3)
```

```
st.set_color(Color(1.0, 0.0, 1.0))
```

```
st.add_vertex(p5)
```

```
st.set_color(Color(0.0, 1.0, 0))
```

```
st.add_vertex(p3)
```

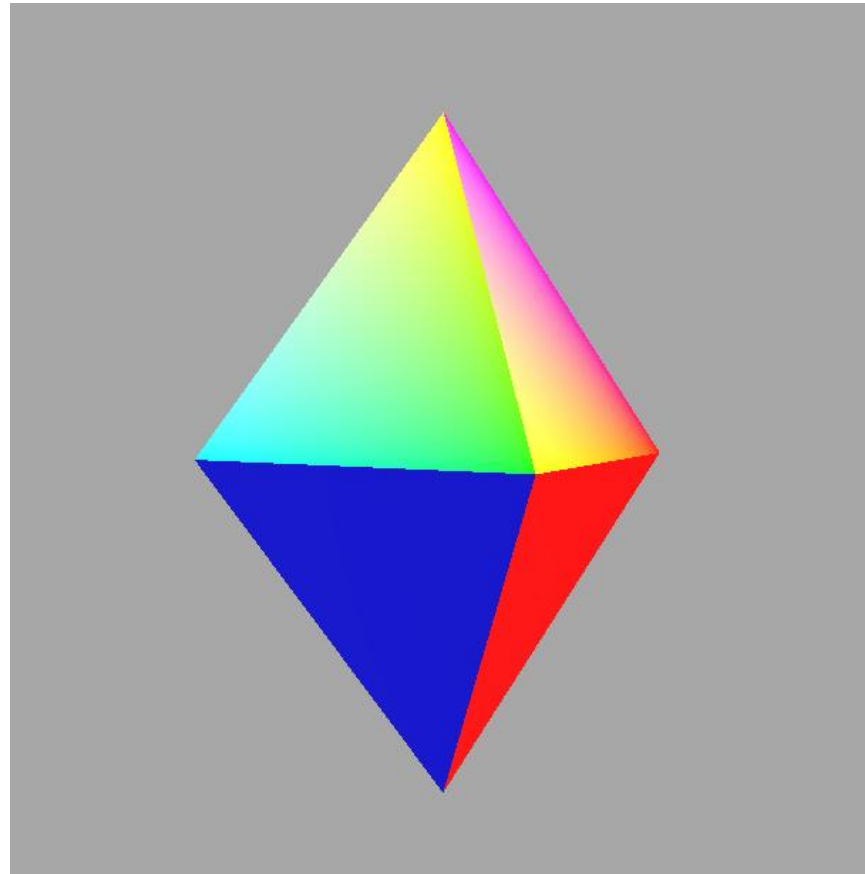
```
st.set_color(Color(0.0, 1.0, 1.0))
```

```
st.add_vertex(p4)
```

```
st.set_color(Color(1.0, 1.0, 0.0))
```

```
st.add_vertex(p5)
```

...



3. Objetos (mallas)

- ! Malla creada con **ArrayMesh**. Lista de triángulos aislados
- ! Transparencia en cada cara, **StandardMaterial3D** y **set_color**

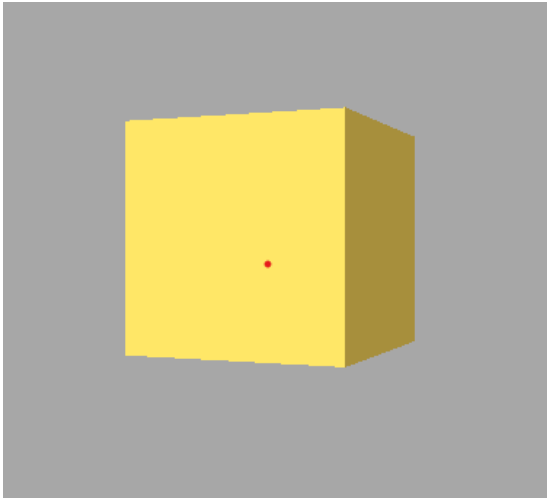
```
...
func _ready():
    ...
    # Se indica un color para cada vértice
    material.vertex_color_use_as_albedo = true
    material.transparency = BaseMaterial3D.TRANSPARENCY_ALPHA
    # Coordenadas de la figura
    ...
    st.begin(Mesh.PRIMITIVE_TRIANGLES) # se indica triángulos sólidos
    # parte superior
    st.set_color(Color(0.0, 0.0, 0.5, 0.5)) # se añade un valor para alfa
    st.add_vertex(p1)
    st.add_vertex(p2)
    st.add_vertex(p5)

    st.set_color(Color(1.0, 0.0, 0.0, 0.5)) # se añade un valor para alfa
    st.add_vertex(p2)
    st.add_vertex(p3)
    st.add_vertex(p5)
    ...
```

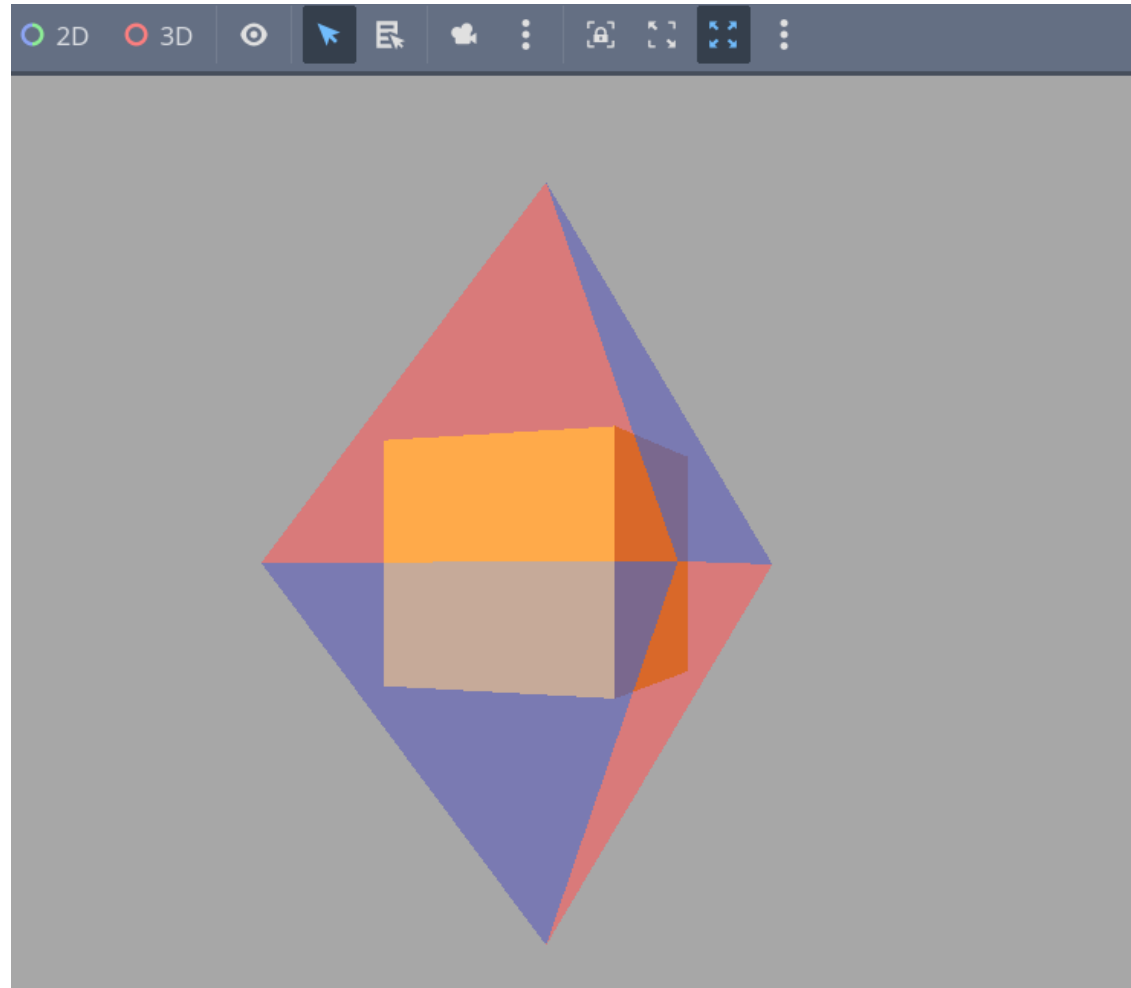


3. Objetos (mallas)

- ! Malla creada con **ArrayMesh**. Lista de triángulos aislados
- ! Transparencia en cada cara, **StandardMaterial3D** y **set_color**

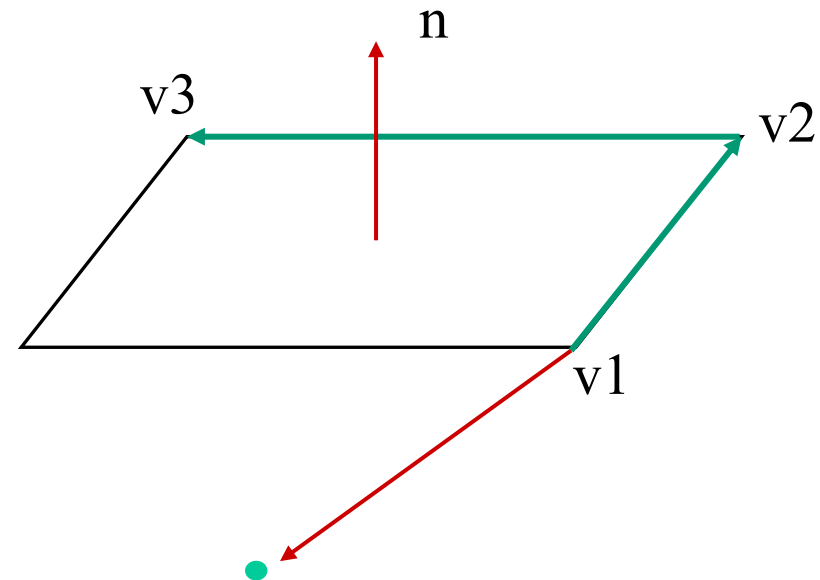
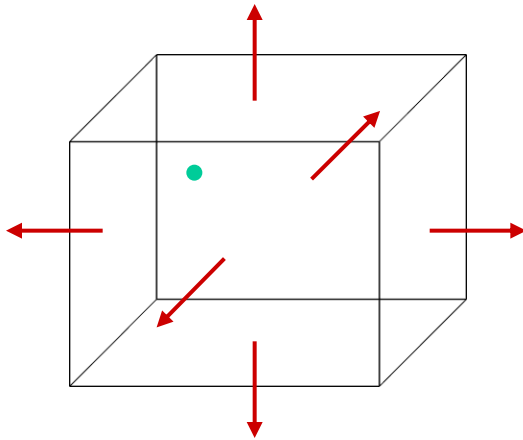


Se añade un objeto cubo dentro del octaedro



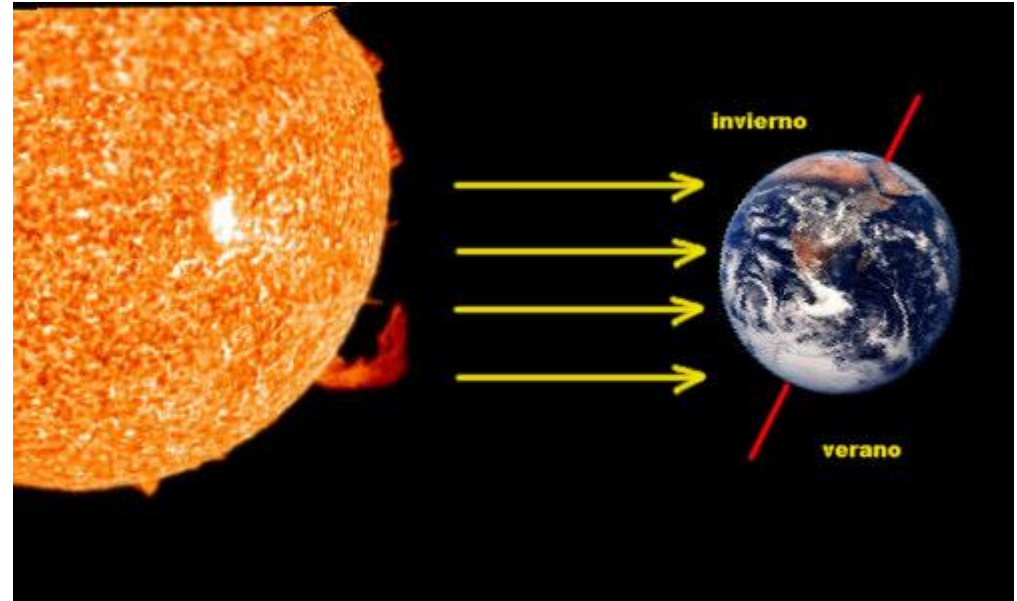
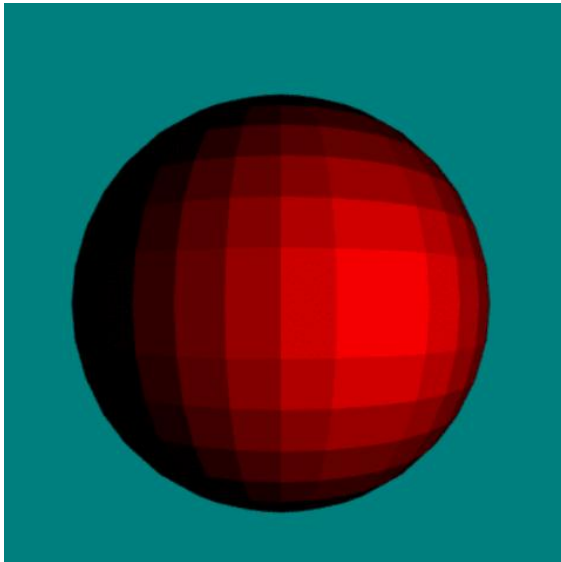
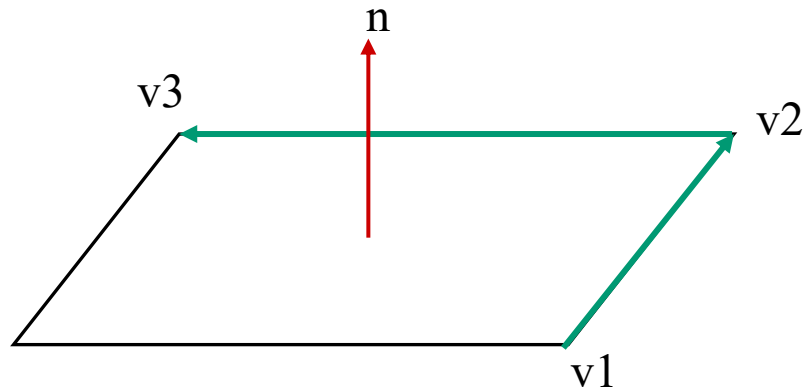
3. Objetos (mallas)

- ! Usos de la normal a una cara: contención de un punto en un objeto cerrado formado por caras convexas.



3. Objetos (mallas)

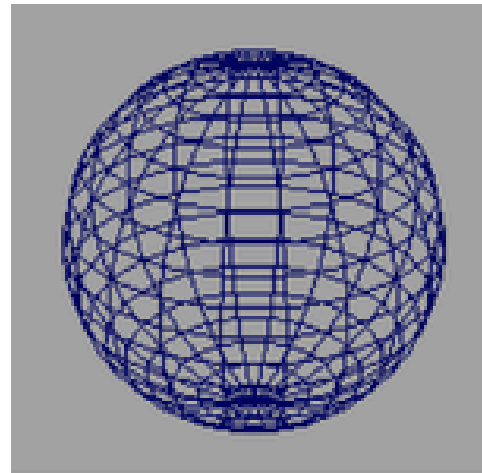
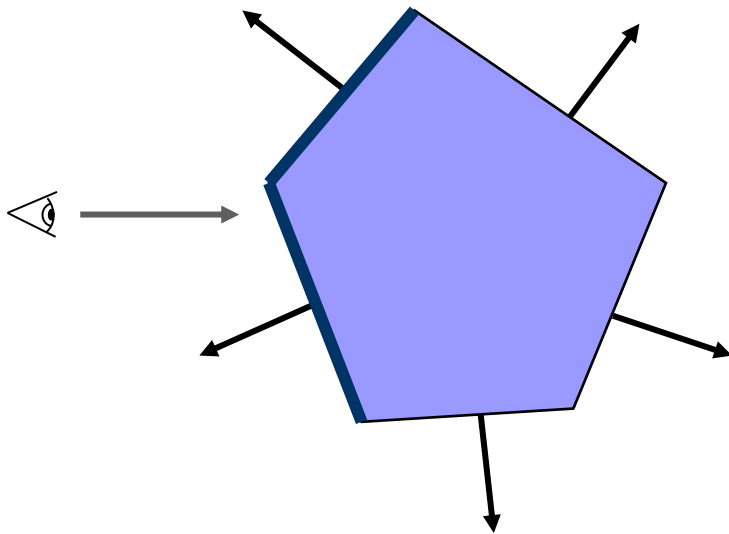
! Usos de la normal a una cara: iluminación (color) de una cara



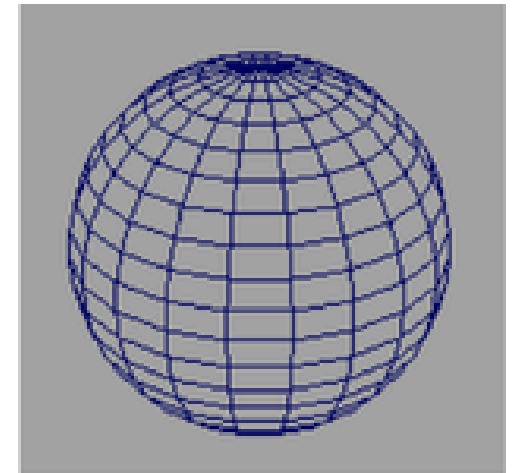
3. Objetos (mallas)

! Usos de la normal a una cara: optimización mediante eliminación de la cara trasera (**Back-face culling**)

! Activado por defecto



Backfaces



No backfaces

3. Objetos (mallas)

! Problema con el uso de **Back-Face Culling**



3. Objetos (mallas)

- ! Malla creada con **ArrayMesh**. Lista de triángulos aislados
- ! Iluminación plana, una normal por cada cara

```
...
func _ready():
    var octaedro = MeshInstance3D.new()
    var mesh = ArrayMesh.new()
    var st = SurfaceTool.new()
    var material = StandardMaterial3D.new()

    # Se indica un color para toda la figura
    material.albedo_color = Color(0.9, 0.4, 0.1)

    # Coordenadas de la figura
    ...
    # se indica triángulos sólidos
    st.begin(Mesh.PRIMITIVE_TRIANGLES)

    # parte superior
    st.set_normal(Plane(p1,p2,p5).normal)
    st.add_vertex(p1)
    st.add_vertex(p2)
    st.add_vertex(p5)

    st.set_normal(Plane(p2,p3,p5).normal)
    st.add_vertex(p2)
    st.add_vertex(p3)
    st.add_vertex(p5)
    ...
    mesh=st.commit()
    octaedro.mesh = mesh
    octaedro.material_override=material
    add_child(octaedro)
```

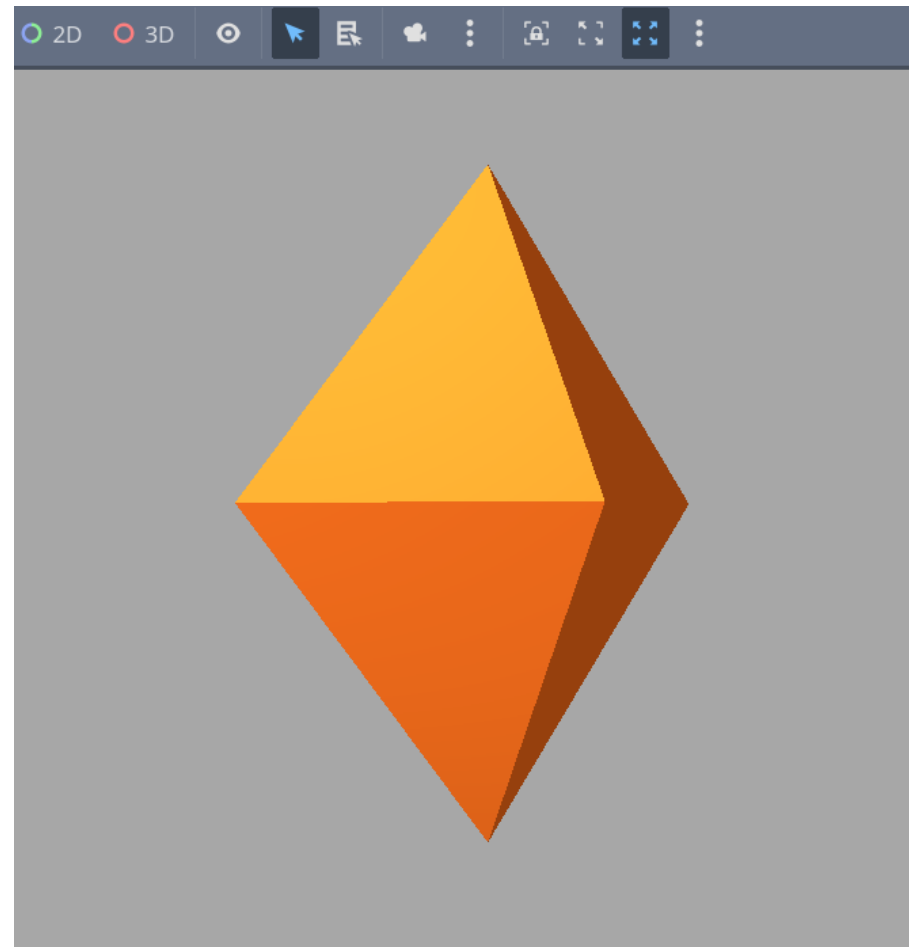


3. Objetos (mallas)

- ! Malla creada con [ArrayMesh](#). Lista de triángulos aislados
- ! Iluminación plana, una normal por cada cara

```
(0.0, 0.447214, -0.894427)  
(0.894427, 0.447214, 0.0)  
(0.0, 0.447214, 0.894427)  
(-0.894427, 0.447214, 0.0)  
(0.0, -0.447214, -0.894427)  
(0.894427, -0.447214, 0.0)  
(0.0, -0.447214, 0.894427)  
(-0.894427, -0.447214, 0.0)
```

Normales a las caras,
las superiores con la componente “y” positiva,
las inferiores con la componente “y” negativa



3. Objetos (mallas)

! Aislando la malla de la visualización

- Se crea un nodo que englobe diversas funciones para emplearlas en otros nodos
- Las funciones se definen como **static**
- Se usa **load(ruta al nodo)** en los nodos que las utilizan

```
#operaciones.gd
```

```
extends Node
```

```
static func sumar(a, b):  
    return a + b
```

```
static func multiplicar(a, b):  
    return a * b
```

```
extends Node
```

```
var calcular =  
load("res://operaciones.gd")
```

```
func _ready():  
    var resultado = calcular.sumar(3, 4)  
    print("Resultado:", resultado)
```



3. Objetos (mallas)

! Aislando la malla de la visualización

! Ejemplo, se crea un Node3D que contenga funciones para distintos modos de visualización que llamo **modos_malla**

extends Node3D

función para crear un sólido visualizado sin iluminación

pasamos un vector de caras y otro de colores para cada cara

static func malla_solido(caras: Array, colores: Array,
material: StandardMaterial3D)->ArrayMesh:

var st = SurfaceTool.new()

se desactiva la iluminación

material.shading_mode = BaseMaterial3D.SHADING_MODE_UNSHADED

material.vertex_color_use_as_albedo = true

st.begin(Mesh.PRIMITIVE_TRIANGLES)

for i in caras.size():

st.set_color(colores[i])

st.add_vertex(caras[i][0])

st.add_vertex(caras[i][1])

st.add_vertex(caras[i][2])

return st.commit()



3. Objetos (mallas)

! Aislando la malla de la visualización

! Ejemplo, se crea un Node3D que contenga funciones para distintos modos de visualización que llamo **modos_malla**

función para crear un sólido visualizado con iluminación plana

```
static func malla_iluminacion(caras: Array, albedo: Color,  
                             material: StandardMaterial3D)->ArrayMesh:
```

```
var st = SurfaceTool.new()
```

```
# desactivar la iluminación ambiental para caras a las que no llegue luz directa
```

```
material.disable_ambient_light=true
```

```
material.albedo_color=albedo
```

```
st.begin(Mesh.PRIMITIVE_TRIANGLES)
```

```
for i in caras.size():
```

```
    st.set_normal(Plane(caras[i][0], caras[i][1], caras[i][2]).normal)
```

```
    st.add_vertex(caras[i][0])
```

```
    st.add_vertex(caras[i][1])
```

```
    st.add_vertex(caras[i][2])
```

```
return st.commit()
```

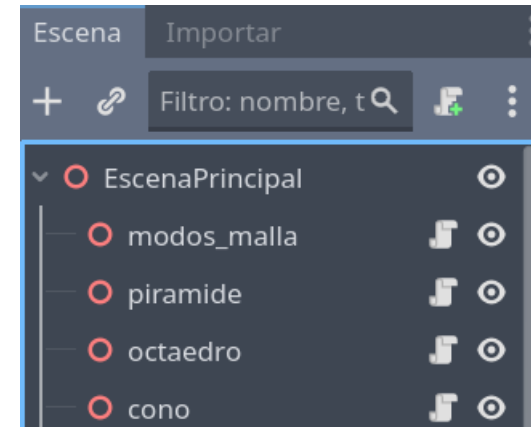


3. Objetos (mallas)

! Aislando la malla de la visualización

! Ejemplo de un octaedro usando el nodo **modos_malla**

```
# figura octaedro
extends Node3D
@export var v_superior: float = 1.0
@export var v_inferior: float = -1.0
@export var lado: float = 1.0
# variables globales al script
var octaedro = MeshInstance3D.new()
var material = StandardMaterial3D.new()
# cargar el nodo externo
var modo = load("res://modos_malla.gd")
# crear los puntos
var p1 = Vector3(-lado/2.0, 0, -lado/2.0)
var p2 = Vector3( lado/2.0, 0, -lado/2.0)
var p3 = Vector3( lado/2.0, 0, lado/2.0)
var p4 = Vector3(-lado/2.0, 0, lado/2.0)
var p5 = Vector3(0, v_superior, 0)
var p6 = Vector3(0, v_inferior, 0)
```



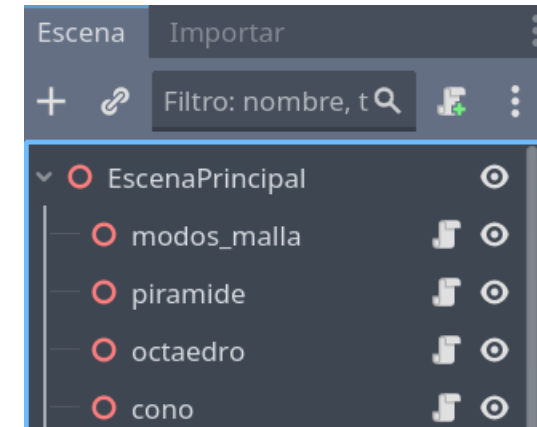
3. Objetos (mallas)

! Aislando la malla de la visualización

! Ejemplo de un octaedro usando el nodo **modos_malla**

```
# figura octaedro
# crear un vector de caras como triángulos aislados
var caras= [[p1, p2, p5], [p2, p3, p5], [p3, p4, p5], [p4, p1, p5],
            [p2, p1, p6], [p3, p2, p6], [p4, p3, p6], [p1, p4, p6]]
# crear un vector de colores
var colores=[Color(0.0, 0.0, 0.5, 0.5), Color(0.0, 1.0, 0.0, 0.5),
            Color(1.0, 0.0, 0.5, 0.5), Color(0.5, 1.0, 0.0, 0.5),
            Color(0.2, 1.0, 0.5, 0.5), Color(1.0, 1.0, 0.0, 0.5),
            Color(0.2, 0.2, 1.0, 0.5), Color(0.0, 1.0, 1.0, 0.5)]
# color para la iluminación
var albedo=Color(0.9,0.4,0.1, 1.0)

func _ready() -> void:
    # visualizado inicialmente como sólido con caras de distinto color
    octaedro.mesh = modo.malla_solido(caras, colores, material)
    octaedro.material_override=material
    add_child(octaedro)
```



3. Objetos (mallas)

! Aislando la malla de la visualización

! Ejemplo de un octaedro usando el nodo **modos_malla**

- Se añade al nodo octaedro un evento para cambiar el modo

función para manejar un evento

func _input(event):

se cambia el material y se rehace la malla,

lo que permite cambiar el modo de visualización al ejecutar

var nuevo_material = StandardMaterial3D.new()

if event is InputEventKey and event.pressed:

match event.keycode:

KEY_1:

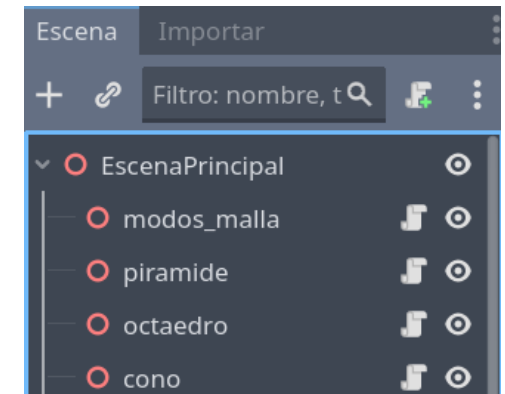
octaedro.mesh = modo.malla_solido(caras, colores, nuevo_material)

octaedro.material_override=nuevo_material

KEY_2:

octaedro.mesh = modo.malla_iluminacion(caras, albedo, nuevo_material)

octaedro.material_override=nuevo_material

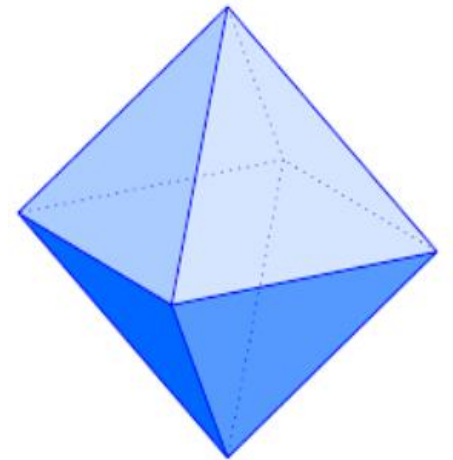


3. Objetos (mallas)

- ! Malla creada con **ArrayMesh**. Listas de vértices y caras. Malla aislada de la visualización (ejemplo octaedro)

```
extends Node3D # se crea en un node_3D
...
# variables globales al script
var octaedro = MeshInstance3D.new()
var material = StandardMaterial3D.new()
var modo = load("res://modos_malla_2.gd")

# lista de vértices
var vértices = [Vector3(-lado/2.0, 0, -lado/2.0), # 0
                Vector3( lado/2.0, 0, -lado/2.0), # 1
                Vector3( lado/2.0, 0, lado/2.0),  # 2
                Vector3(-lado/2.0, 0, lado/2.0),   # 3
                Vector3(0, v_superior, 0),         # 4
                Vector3(0, v_inferior, 0)]          # 5
```



3. Objetos (mallas)

! Malla creada con **ArrayMesh**. Listas de vértices y caras. Malla aislada de la visualización (ejemplo octaedro)

lista de caras

```
var caras = [[0, 1, 4], [1, 2, 4], [2, 3, 4], [3, 0, 4],  
             [1, 0, 5], [2, 1, 5], [3, 2, 5], [0, 3, 5]]
```

lista de colores

```
var colores = [Color(0.0, 0.0, 0.5, 0.5), Color(0.0, 1.0, 0.0, 0.5),  
              Color(1.0, 0.0, 0.5, 0.5), Color(0.5, 1.0, 0.0, 0.5),  
              Color(0.2, 1.0, 0.5, 0.5), Color(1.0, 1.0, 0.0, 0.5),  
              Color(0.2, 0.2, 1.0, 0.5), Color(0.0, 1.0, 1.0, 0.5)]
```

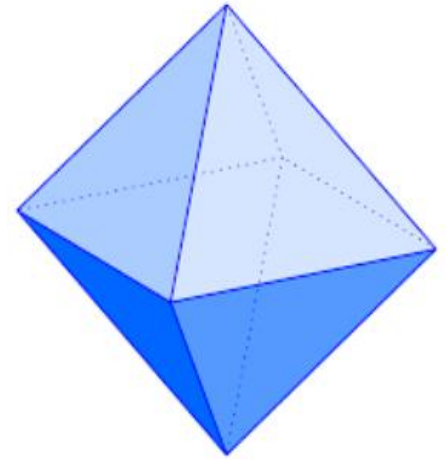
```
var albedo = Color(0.9,0.4,0.1, 1.0)
```

```
func _ready() -> void:
```

```
    octaedro.mesh =modo.malla_solido(vértices, caras, colores, material)
```

```
    octaedro.material_override=material
```

```
    add_child(octaedro)
```



3. Objetos (mallas)

! Malla creada con **ArrayMesh**. Listas de vértices y caras. Malla aislada de la visualización

! Cambios en las funciones de visualización por uso de estas listas

extends Node3D

función para crear un sólido visualizado de forma plana

pasamos un vector de vértices, un vector de caras y otro de colores para cada cara

**static func malla_solido(vértices: Array, caras: Array, colores: Array,
material: StandardMaterial3D)->ArrayMesh:**

var st = SurfaceTool.new()

se desactiva la iluminación

material.shading_mode = BaseMaterial3D.SHADING_MODE_UNSHADED

material.vertex_color_use_as_albedo = true

st.begin(Mesh.PRIMITIVE_TRIANGLES)

for i in caras.size():

 st.set_color(colores[i])

 # se accede a los vértices usando como índices los valores contenidos en la lista de caras

st.add_vertex(vértices[caras[i][0]])

st.add_vertex(vértices[caras[i][1]])

st.add_vertex(vértices[caras[i][2]])

return st.commit()



3. Objetos (mallas)

! Malla creada con **ArrayMesh**. Listas de vértices y caras. Malla aislada de la visualización

! Cambios en las funciones de visualización por uso de estas listas

extends Node3D

función para crear un sólido visualizado con iluminación plana

**static func malla_iluminacion(vértices: Array, caras: Array, albedo: Color,
material: StandardMaterial3D)->ArrayMesh:**

var st = SurfaceTool.new()

desactivar la iluminación ambiental para caras a las que no llegue luz directa

material.disable_ambient_light=true

material.albedo_color=albedo

st.begin(Mesh.PRIMITIVE_TRIANGLES)

for i in caras.size():

se accede a los vértices usando como índices los valores contenidos en la lista de caras

**st.set_normal (Plane(vértices[caras[i][0]], vértices[caras[i][1]],
vértices[caras[i][2]]).normal)**

st.add_vertex(vértices[caras[i][0]])

st.add_vertex(vértices[caras[i][1]])

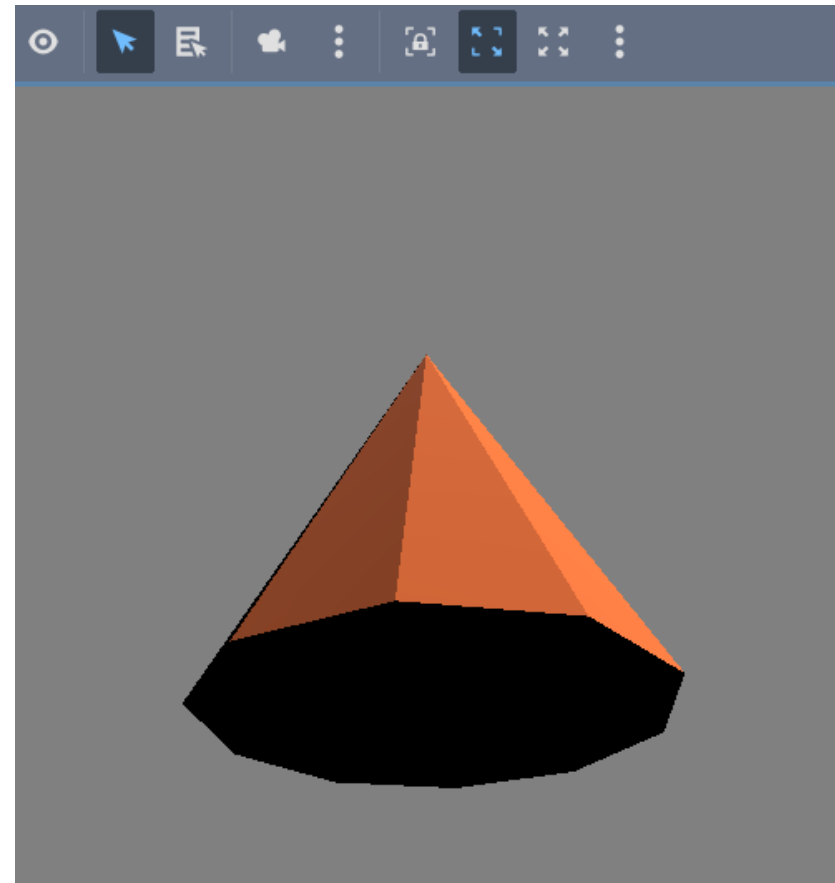
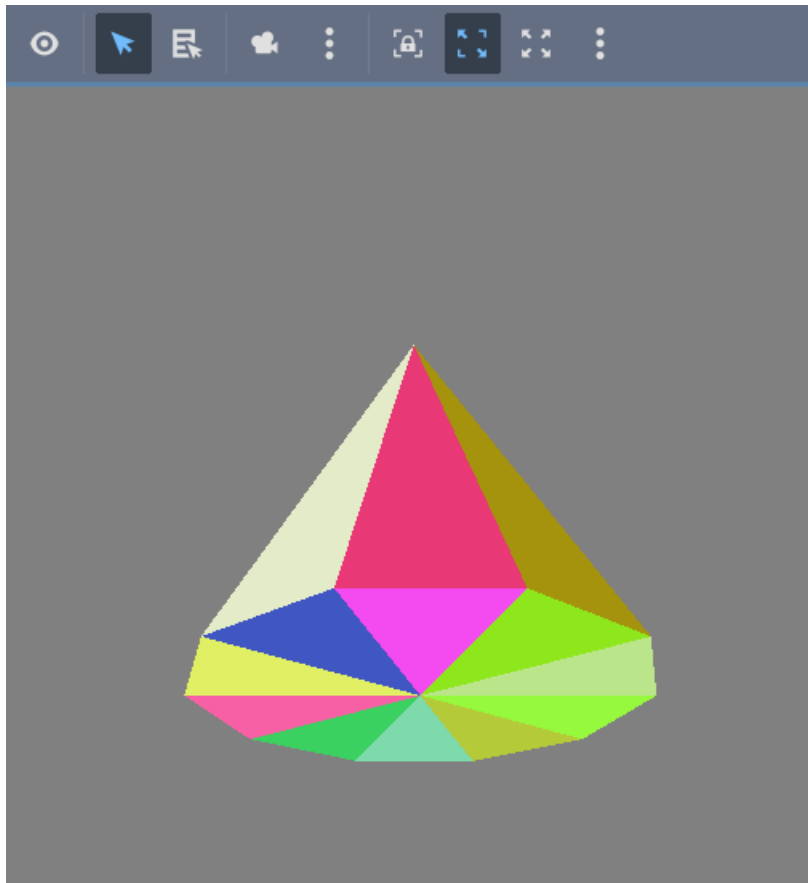
st.add_vertex(vértices[caras[i][2]])

return st.commit()



3. Objetos (mallas)

- ! Malla creada con [ArrayMesh](#). Listas de vértices y caras. Malla aislada de la visualización
- ! Ejemplo cono (objeto por revolución)



3. Objetos (mallas)

- ! Malla creada con **ArrayMesh**. Listas de vértices y caras. Malla aislada de la visualización. Ejemplo cono (objeto por revolución)

```
# cono.gd
extends Node3D # se crea en un node_3D
@export var altura: float =1.0
@export var radio: float =0.5
@export var lados: int =10
# variables globales al script
var cono =MeshInstance3D.new()
var material =StandardMaterial3D.new()
var mesh = ArrayMesh.new()
var modo = load("res://modos_malla_2.gd")

# matrices de vértices y caras inicialmente vacías
var vértices = [ ]
# caras
var caras= [ ]
# colores
var colores=[ ]
var albedo=Color(0.9,0.4,0.1, 1.0)
```



3. Objetos (mallas)

! Malla creada con **ArrayMesh**. Listas de vértices y caras. Malla aislada de la visualización. Ejemplo cono (objeto por revolución)

```
func _ready() -> void: # continuación de cono.gd
    var i=0
    # vértices
    var px; var pz
    while i<lados:
        px=radio*cos(i*2*PI/lados)
        pz=-radio*sin(i*2*PI/lados)
        vértices.append(Vector3(px, 0, pz))
        i+=1
    # caras superiores
    vértices.append(Vector3(0, altura, 0))
    ...
    # caras en la base
    vértices.append(Vector3(0, 0, 0))
    ...
    # colores aleatorios
    for j in caras.size():
        colores.append(Color(randf(), randf(), randf()))
```



3. Objetos (mallas)

! Pintando ejes.

```
extends Node3D
```

```
@export var tam:=10
```

```
func _ready() -> void:
```

```
    var ejes = MeshInstance3D.new()
```

```
    var material = StandardMaterial3D.new()
```

```
    var st = SurfaceTool.new()
```

```
    material.vertex_color_use_as_albedo = true
```

```
    st.begin(Mesh.PRIMITIVE_LINES) # se pintan líneas
```

```
    st.set_color(Color(1.0, 0.0, 0.0, 1.0)) # eje X
```

```
    st.add_vertex(Vector3(tam, 0, 0))
```

```
    st.add_vertex(Vector3(-tam, 0, 0))
```

```
    st.set_color(Color(0.0, 1.0, 0.0, 1.0)) # eje Y
```

```
    st.add_vertex(Vector3(0, tam, 0))
```

```
    st.add_vertex(Vector3(0, -tam, 0))
```

```
    st.set_color(Color(0.0, 0.0, 1.0, 1.0)) # eje Z
```

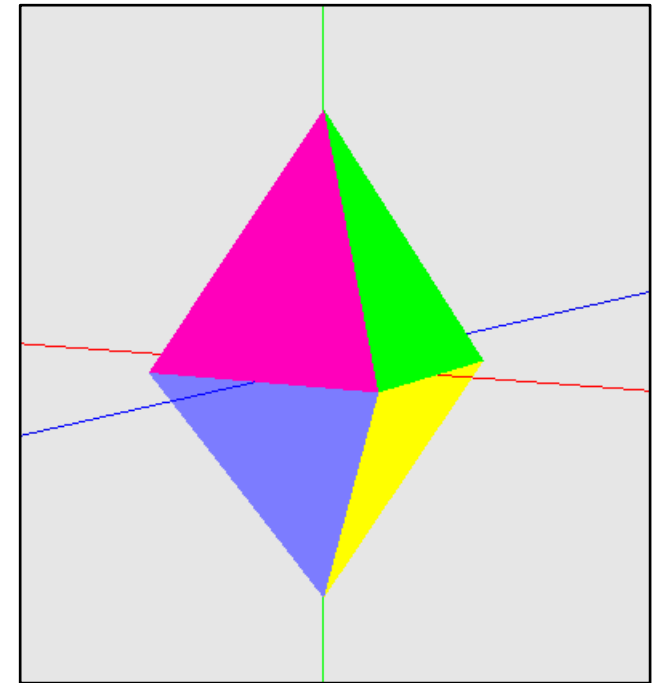
```
    st.add_vertex(Vector3(0,0,tam))
```

```
    st.add_vertex(Vector3(0,0,-tam))
```

```
    ejes.mesh = st.commit()
```

```
    ejes.material_override=material
```

```
    add_child(ejes)
```



4. Transformaciones geométricas

- ! En Godot las matrices de transformación en 3D son **4x4** y multiplican por la izquierda a los vértices.

$$\begin{pmatrix} m_{00} & m_{01} & m_{02} & m_{03} \\ m_{10} & m_{11} & m_{12} & m_{13} \\ m_{20} & m_{21} & m_{22} & m_{23} \\ m_{30} & m_{31} & m_{32} & m_{33} \end{pmatrix} \begin{pmatrix} v_{0x} & \dots & v_{nx} \\ v_{0y} & \dots & v_{ny} \\ v_{0z} & \dots & v_{ny} \\ 1 & 1 & 1 \end{pmatrix}$$

- ! Ejemplo con una matriz de traslación

$$\begin{pmatrix} 1 & 0 & 0 & T_x \\ 0 & 1 & 0 & T_y \\ 0 & 0 & 1 & T_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} v_{0x} & \dots & v_{nx} \\ v_{0y} & \dots & v_{ny} \\ v_{0z} & \dots & v_{ny} \\ 1 & 1 & 1 \end{pmatrix}$$



4. Transformaciones geométricas

- ! Concatenando varias transformaciones.
- ! La multiplicación de matrices de transformación se realiza por la izquierda del vector de vértices de un objeto

$$T \times R \times S \times \begin{pmatrix} v_{0x} & \dots & v_{nx} \\ v_{0y} & \dots & v_{ny} \\ v_{0z} & \dots & v_{ny} \\ 1 & 1 & 1 \end{pmatrix} = M \times \begin{pmatrix} v_{0x} & \dots & v_{nx} \\ v_{0y} & \dots & v_{ny} \\ v_{0z} & \dots & v_{ny} \\ 1 & 1 & 1 \end{pmatrix}$$



4. Transformaciones geométricas

! Hay dos formas de realizar las transformaciones, usando una clase y como propiedades y métodos de **Node3D**.

! Clase **Transform3D** formada por dos propiedades:

- Una matriz **3x4** de rotación y escalado **Basis**
- Un vector de traslación o posición **origin**

Transform3D (Basis, origin)

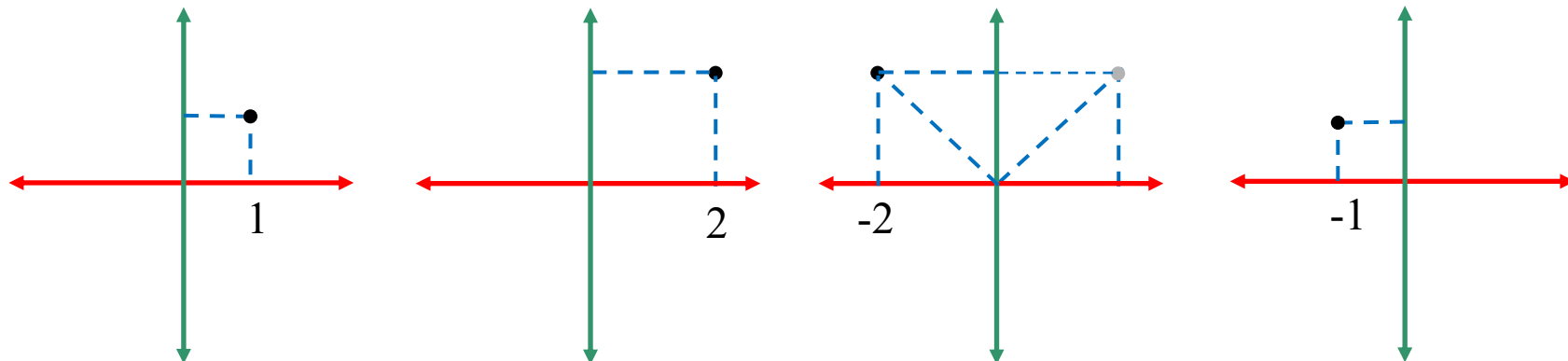
- Métodos:
 - **rotated**(axis: Vector3, angulo: float))
 - **scaled**(scale: Vector3)
 - **translate**(offset: Vector3)



4. Transformaciones geométricas

! Clase **Transform3D**, ejemplo:

```
func _ready() -> void:  
    var punto = Vector3(1, 1, 0)  
    var escala = Transform3D (Basis().scaled(Vector3(2, 2, 2)))  
    # Rotación con respecto al eje Z, ángulo en radianes  
    var rotacion = Transform3D(Basis(Vector3(0,0,1), PI/2.0))  
    var traslacion = Transform3D(Basis(), Vector3(1, -1, 0))  
    var transformacion_total = traslacion * rotacion * escala  
    var resultado = transformacion_total*punto  
    print("Punto final: ", resultado) # (-1.0, 1.0, 0.0)
```



4. Transformaciones geométricas

! Transformaciones como propiedades y métodos de **Node3D**

! Propiedades:

- **position** (Vector3D), position = Vector3D(...)
- **scale** (Vector3D), scale = Vector3D(...)
- **global_position** (Vector3D), global_position = Vector3D(...)

! Métodos

- **rotate**(axis: Vector3, angulo: float)
- **rotate_x**(angulo: float), **rotate_y** (angulo: float),
rotate_z(angulo: float)
- **translate**(offset: Vector3)
- **set_identity**(), matriz identidad



4. Transformaciones geométricas

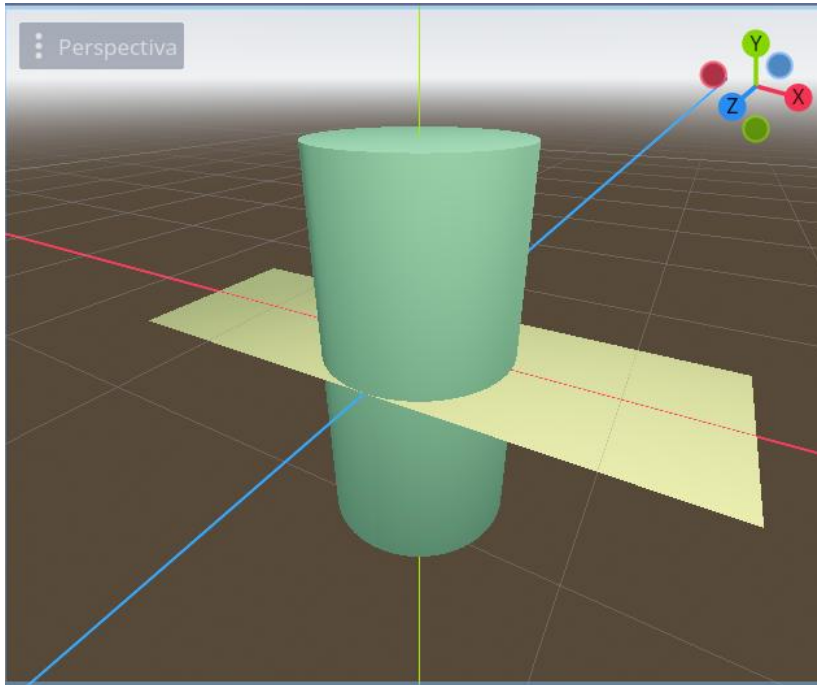
! Transformaciones como propiedades y métodos de **Node3D**,
ejemplo con una **PrimitiveMesh** :

Cilindro, por defecto tiene $\text{radio} = 1$
y $\text{altura} = 2$, centrado en el origen,
con tapas activadas



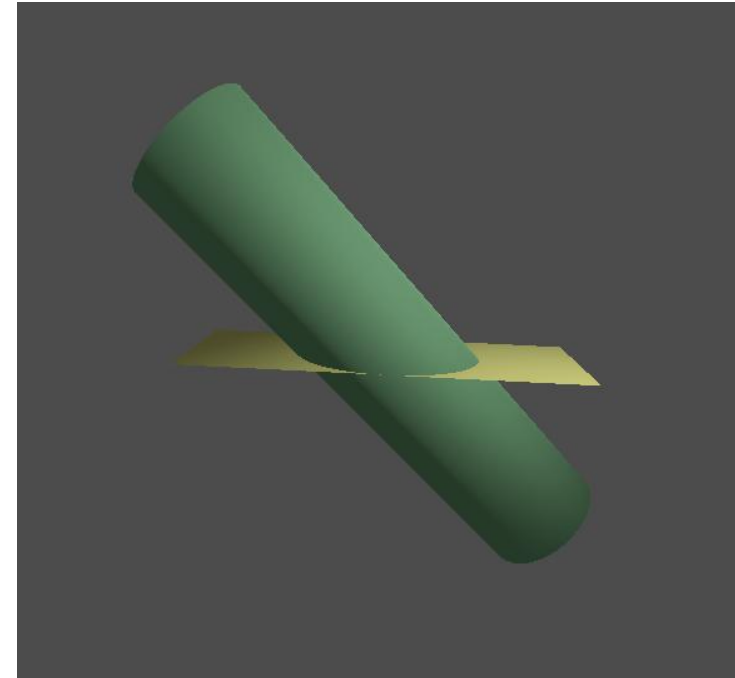
4. Transformaciones geométricas

! Ejemplo con una **PrimitiveMesh** :



El orden de aplicación de las transformaciones de abajo hacia arriba.

En el ejemplo, primero se escala y luego se rota



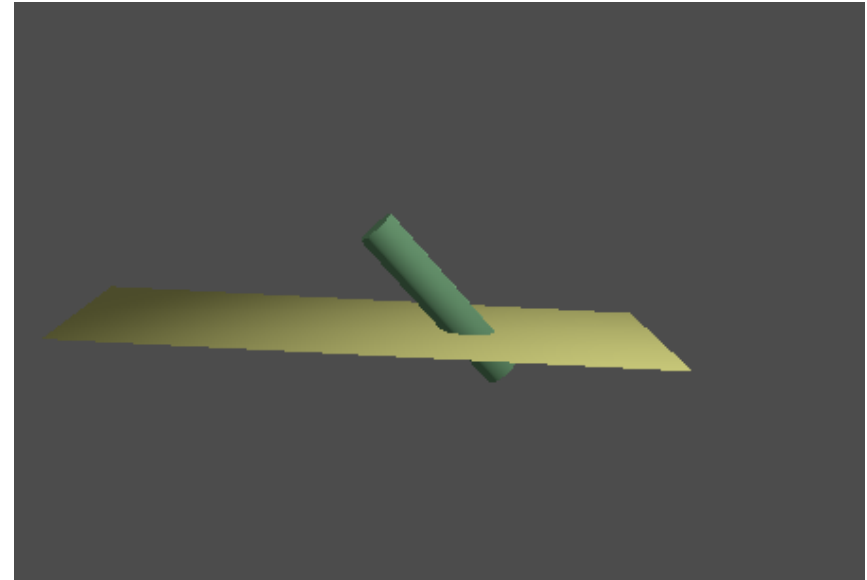
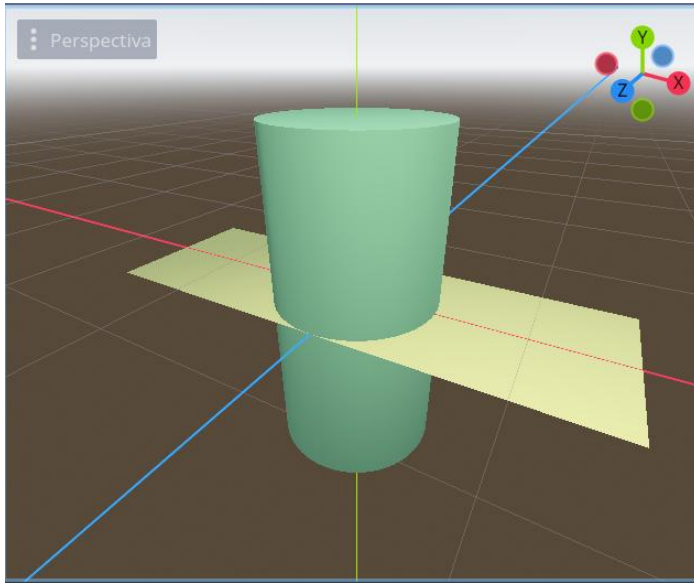
```
extends MeshInstance3D
```

```
func _ready():  
    rotate_z(PI/4.0)  
    scale=Vector3(1, 2, 1)
```



4. Transformaciones geométricas

! Transformar con respecto a un punto pivote:



extends MeshInstance3D

func _ready():



translate(Vector3(0,0.5,0))

rotate_z(PI/4.0)

translate(Vector3(0,-0.5,0))

scale=Vector3(0.2,0.5,0.2)

Punto pivote (0, 0.5, 0)



4. Transformaciones geométricas

! Ejemplo. Con un cubo y 4 pirámides se crea una estrella

```
extends MeshInstance3D
```

```
extends Node3D
```

```
@export var altura: float = 1.5
```

```
func _ready():
```

```
    var piramide = crear_piramide(altura)
```

```
    var mesh_instance = MeshInstance3D.new()
```

```
    var material = StandardMaterial3D.new()
```

```
    mesh_instance.mesh = piramide
```

```
    material.albedo_color = Color(1.0, 0.1, 0.2, 0.5)
```

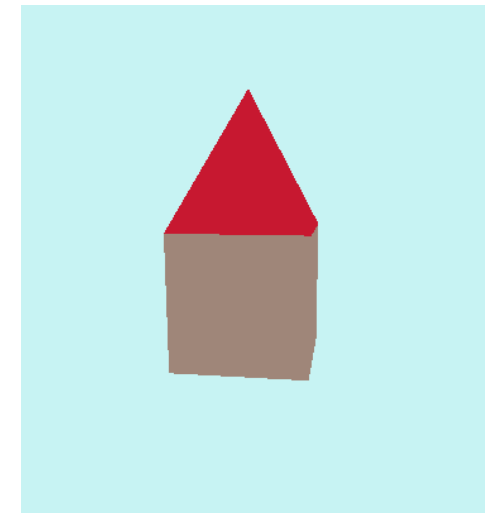
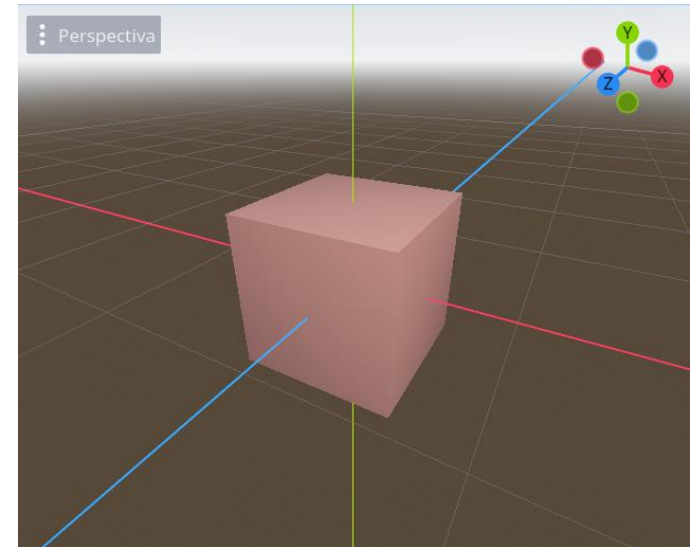
```
    mesh_instance.material_override = material
```

```
    add_child(mesh_instance)
```

```
    translate(Vector3(0, 0.5, 0))
```

```
func crear_piramide(h: float) -> ArrayMesh:
```

```
    ...
```



4. Transformaciones geométricas

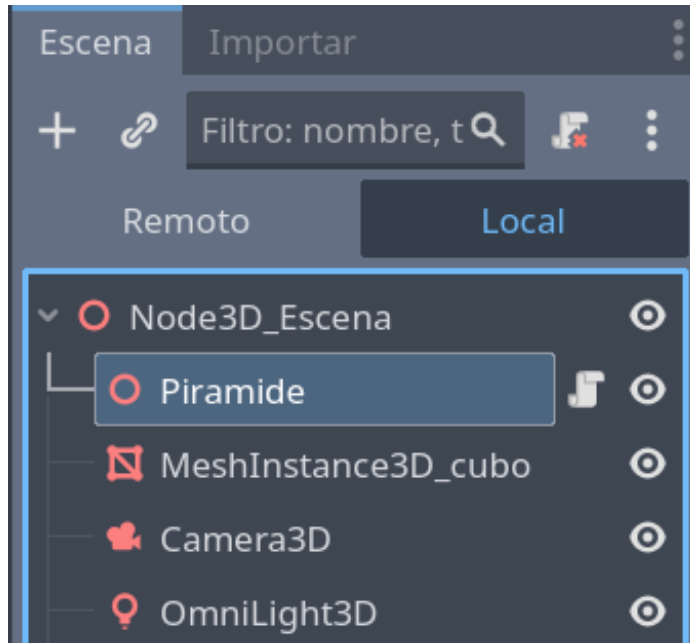
! Ejemplo. Con un cubo y 4 pirámides se crea una estrella

```
func _ready():  
    var piramide = crear_piramide(1)  
    var material = StandardMaterial3D.new()  
    material.albedo_color = Color(1.0, 0.1, 0.2, 1.0) # Naranja claro  
    # se pone el mismo material para las pirámides  
  
    var pos=[Vector3(0.0,0.5,0.0), Vector3(-0.5,0.0,0.0),  
            Vector3(0.0,-0.5,0.0), Vector3(0.5,0.0,0.0)]  
    for i in 4:  
        var mesh_instance = MeshInstance3D.new()  
        mesh_instance.mesh = piramide  
        mesh_instance.material_override = material  
        # en este caso se usa la clase Transform3D  
        var tras = Transform3D()  
        tras.basis = Basis(Vector3(0,0,1), i*PI/2).scaled(Vector3(1,1,1))  
        tras.origin=pos[i]  
        mesh_instance.transform = tras  
        add_child(mesh_instance) # se crean cuatro pirámides
```

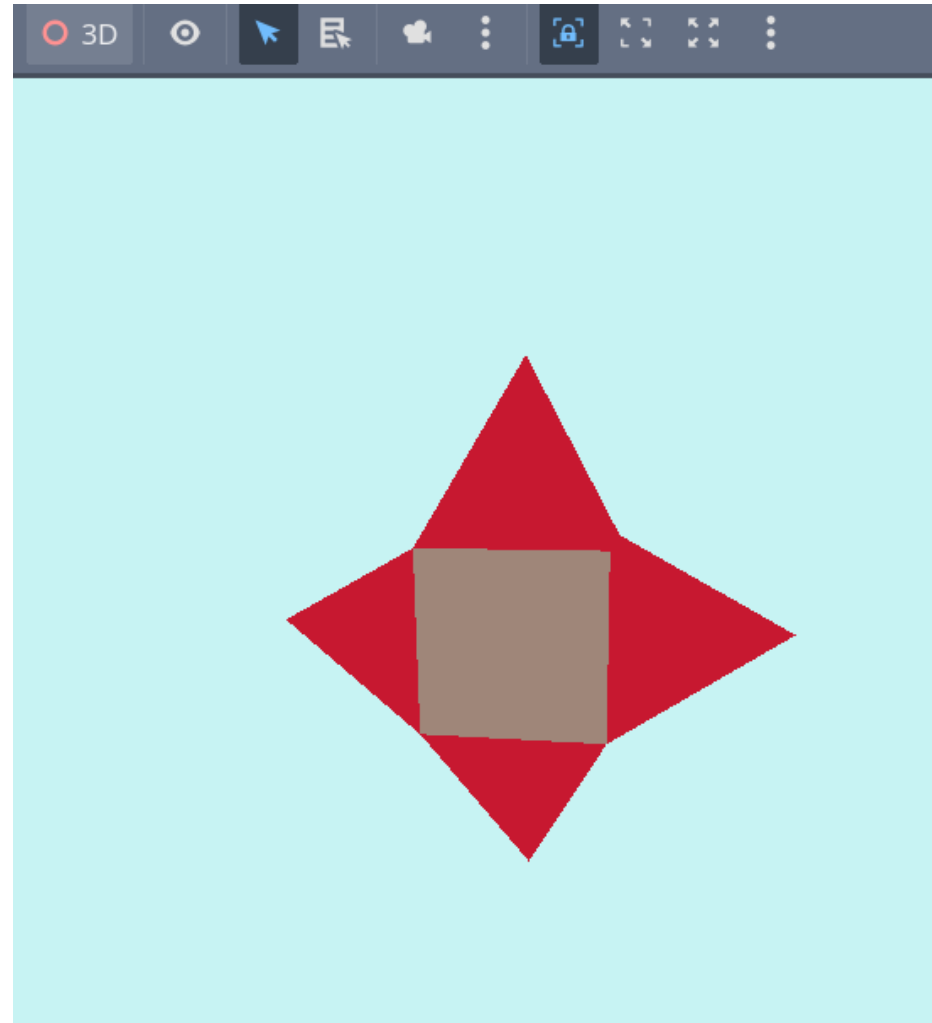


4. Transformaciones geométricas

! Ejemplo. Con un cubo y 4 pirámides se crea una estrella



Jerarquía de la escena



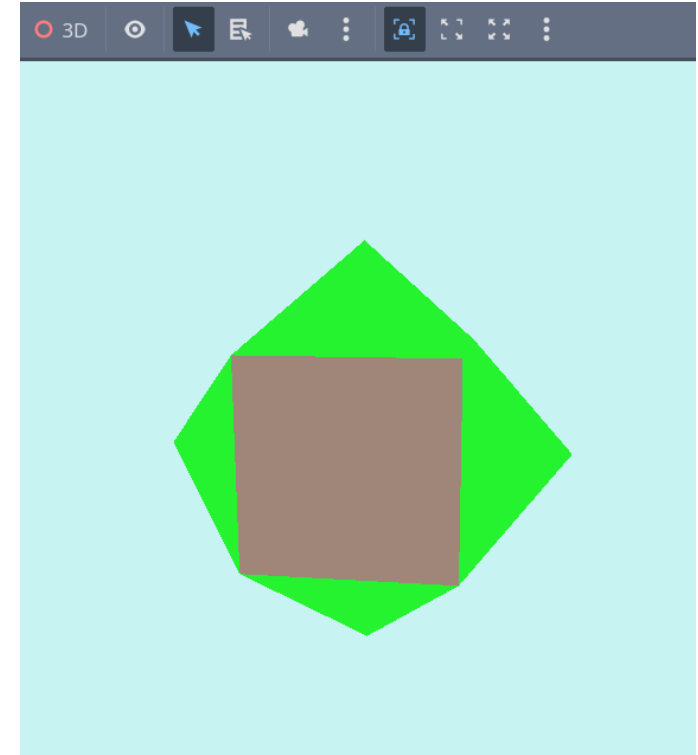
4. Transformaciones geométricas

! Ejemplo. Pirámides por referencia

```
func _ready():  
    var piramide = crear_piramide(0.5)  
    var material = StandardMaterial3D.new()  
    material.albedo_color = Color(0.2, 1.0, 0.2, 1.0)  
  
    var pos=[Vector3(0.0,0.5,0.0), Vector3(-0.5,0.0,0.0),  
            Vector3(0.0,-0.5,0.0), Vector3(0.5,0.0,0.0)]  
    for i in 4:  
        var mesh_instance = MeshInstance3D.new()  
        mesh_instance.mesh = piramide  
        mesh_instance.material_override = material  
        var tras = Transform3D()  
        tras.basis = Basis(Vector3(0,0,1), i*PI/2).scaled(Vector3(1,1,1))  
        tras.origin=pos[i]  
        mesh_instance.transform = tras  
        add_child(mesh_instance) # se crean cuatro pirámides
```

```
func crear_piramide(h: float) -> ArrayMesh:
```

```
...
```



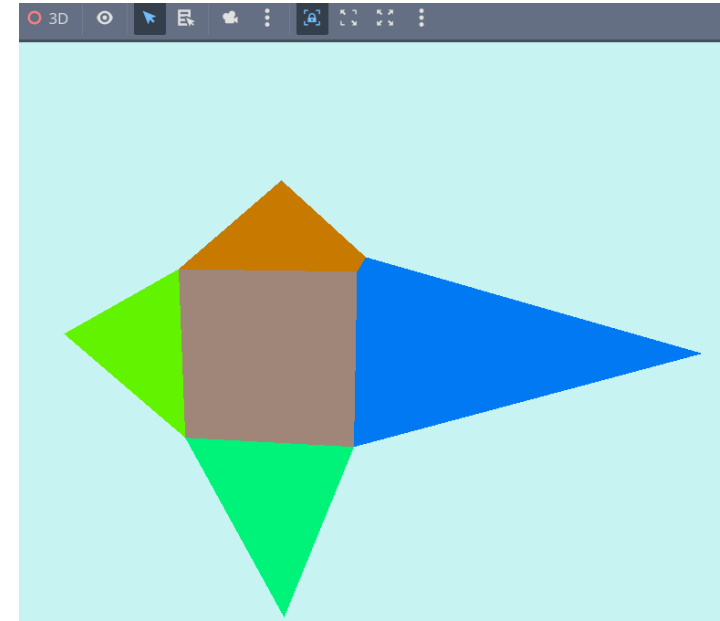
4. Transformaciones geométricas

! Ejemplo. Pirámides por copia

```
func _ready():  
    var altura=[0.5, 1, 1.5, 2]  
    var color= [Color(1.0,0.5,0.0,1.0), Color(0.5,1.0,0.0,1.0),  
                Color(0.0,1.0,0.5,1.0), Color(0.0,0.5,1.0,1.0)]  
    var pos=[Vector3(0.0,0.5,0.0), Vector3(-0.5,0.0,0.0),  
            Vector3(0.0,-0.5,0.0), Vector3(0.5,0.0,0.0)]  
    for i in 4:  
        var mesh_instance = MeshInstance3D.new()  
        var material = StandardMaterial3D.new()  
        mesh_instance.mesh = crear_piramide(altura[i])  
        material.albedo_color = color[i]  
        mesh_instance.material_override = material  
        var tras = Transform3D()  
        tras.basis = Basis(Vector3(0,0,1), i*PI/2).scaled(Vector3(1,1,1))  
        tras.origin=pos[i]  
        mesh_instance.transform = tras  
        add_child(mesh_instance) # se crean cuatro pirámides
```

```
func crear_piramide(h: float) -> ArrayMesh:
```

```
...
```



4. Transformaciones geométricas

! Ejemplo. Pirámides repetidas en nodos (hay que copiar primero el script)

```
extends Node3D
```

```
@export var altura: float = 1
```

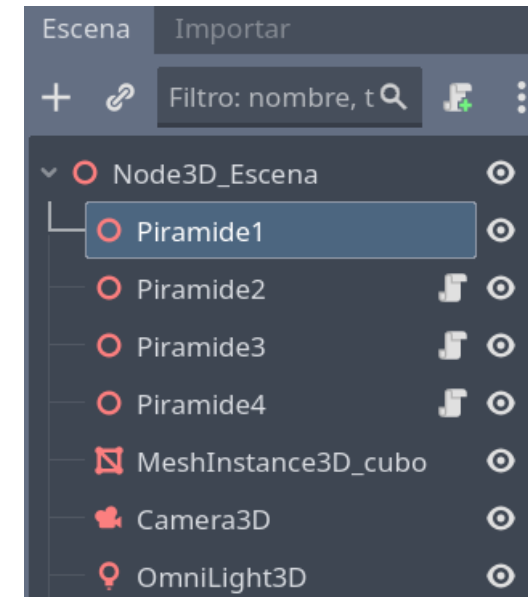
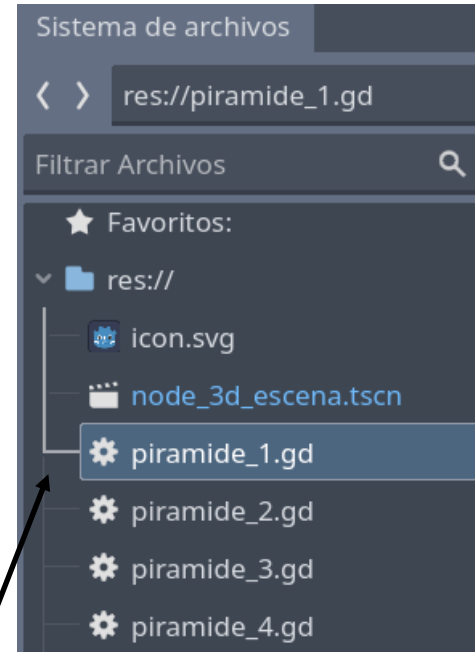
```
func _ready():
```

```
    var piramide = crear_piramide(altura)
    var mesh_instance = MeshInstance3D.new()
    var material = StandardMaterial3D.new()
    mesh_instance.mesh = piramide
    material.albedo_color = Color(1.0, 0.5, 0.0, 1.0)
    mesh_instance.material_override = material
    add_child(mesh_instance)
    translate(Vector3(0,0.5,0))
```

```
func crear_piramide(h: float) -> ArrayMesh:
```

```
    ...
```

piramide_1.gd



4. Transformaciones geométricas

! Ejemplo. Pirámides repetidas en nodos (hay que copiar primero el script)

```
func _ready():  
    ...  
    add_child(mesh_instance)  
    translate(Vector3(-0.5,0,0))  
    rotate_z(PI/2.0)
```

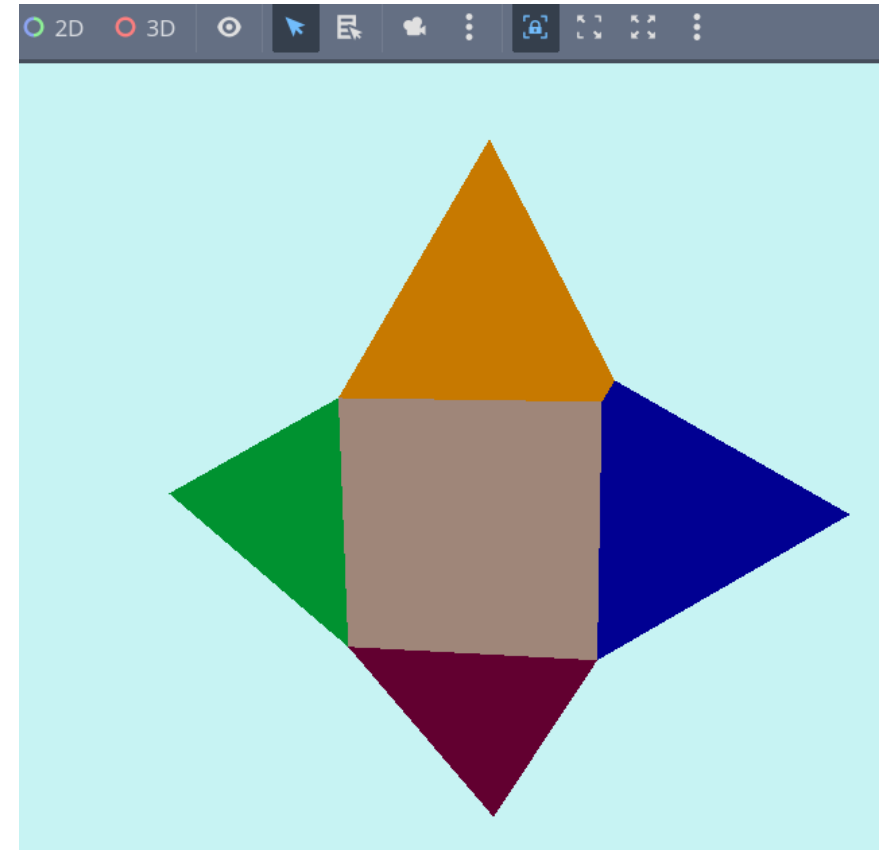
piramide_2.gd

```
func _ready():  
    ...  
    add_child(mesh_instance)  
    translate(Vector3(0,-0.5,0))  
    rotate_z(PI)
```

piramide_3.gd

```
func _ready():  
    ...  
    add_child(mesh_instance)  
    translate(Vector3(0.5,0,0))  
    rotate_z(3.0*PI/2.0)
```

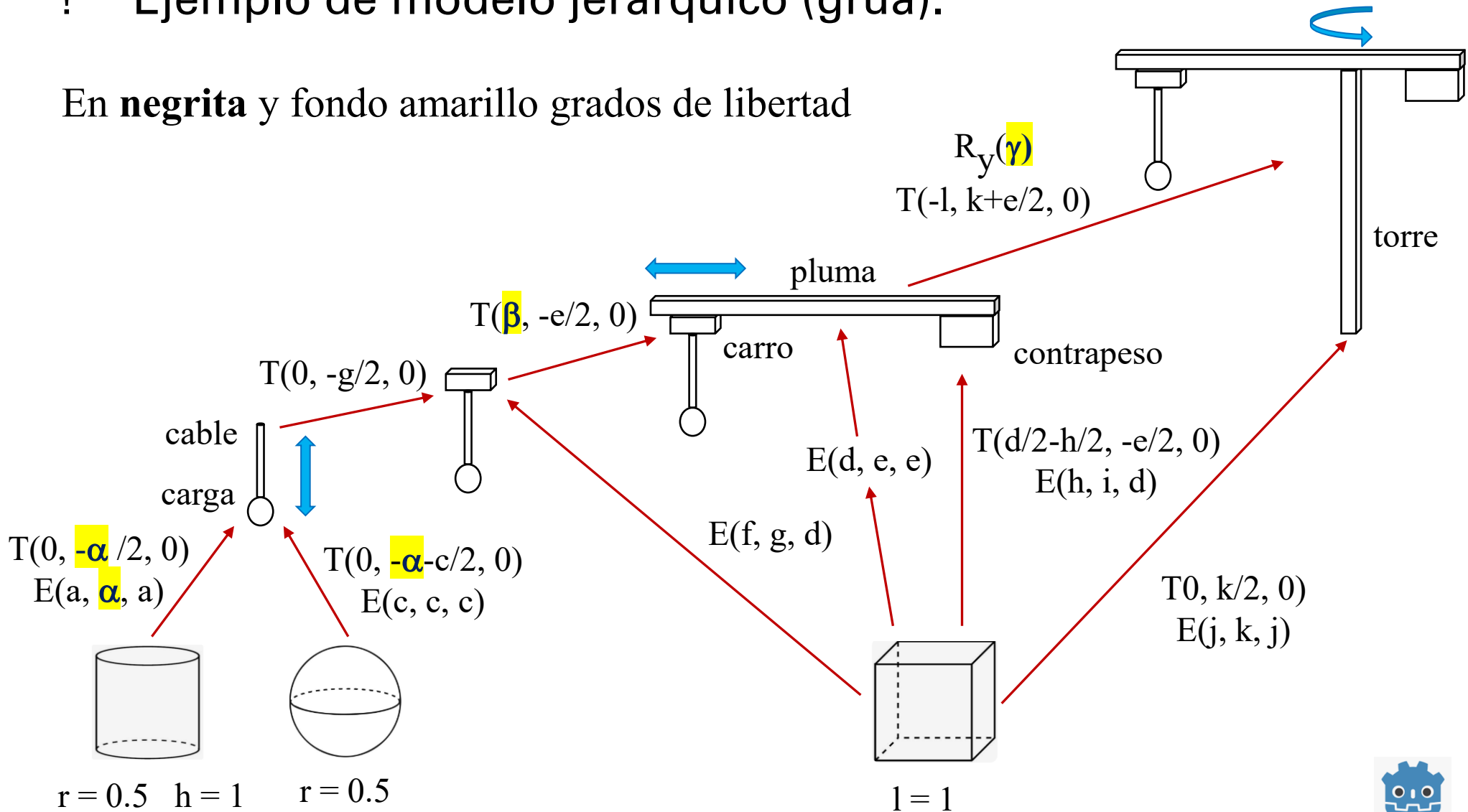
piramide_4.gd



4. Transformaciones geométricas

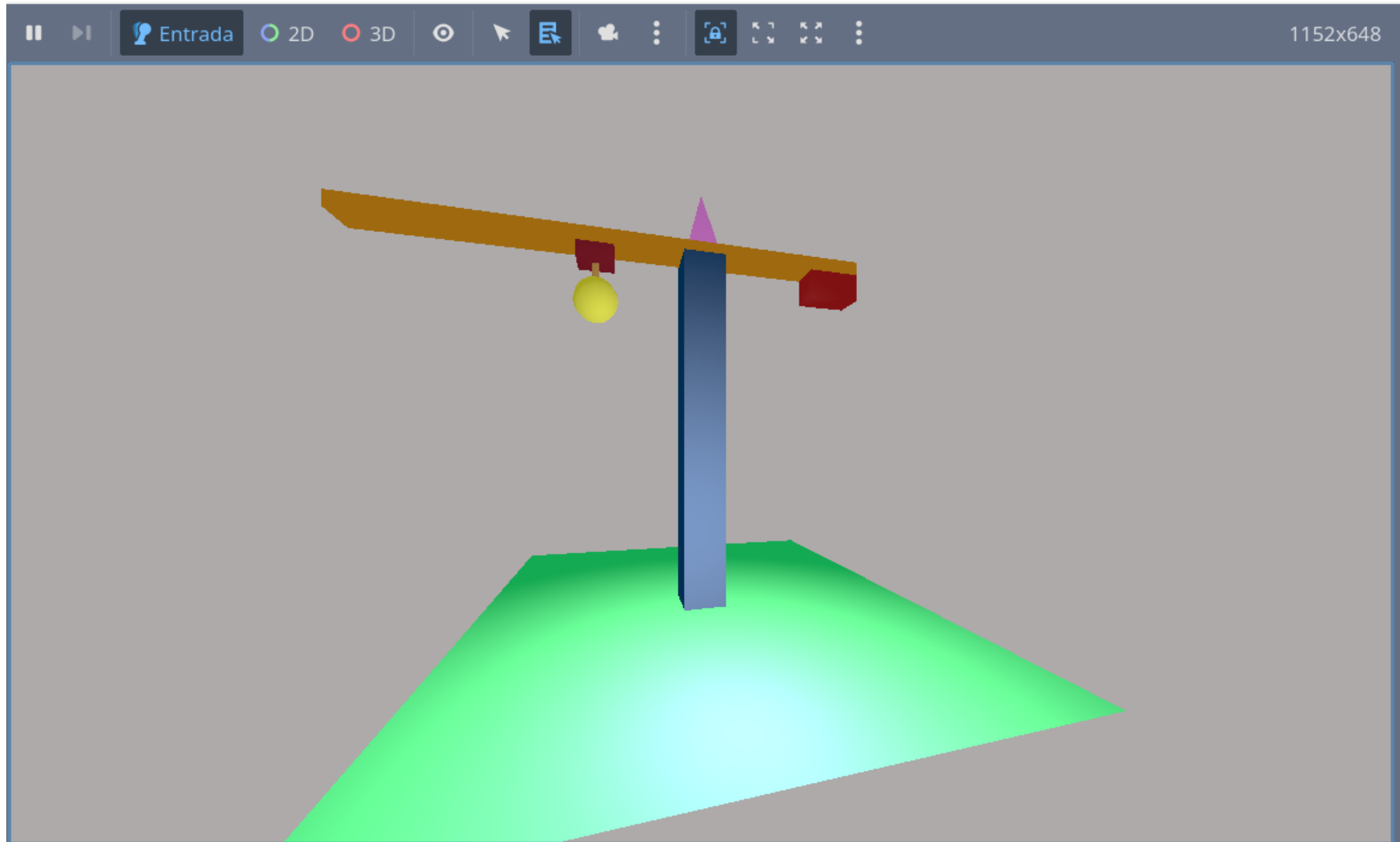
! Ejemplo de modelo jerárquico (grúa).

En **negrita** y fondo amarillo grados de libertad



4. Transformaciones geométricas

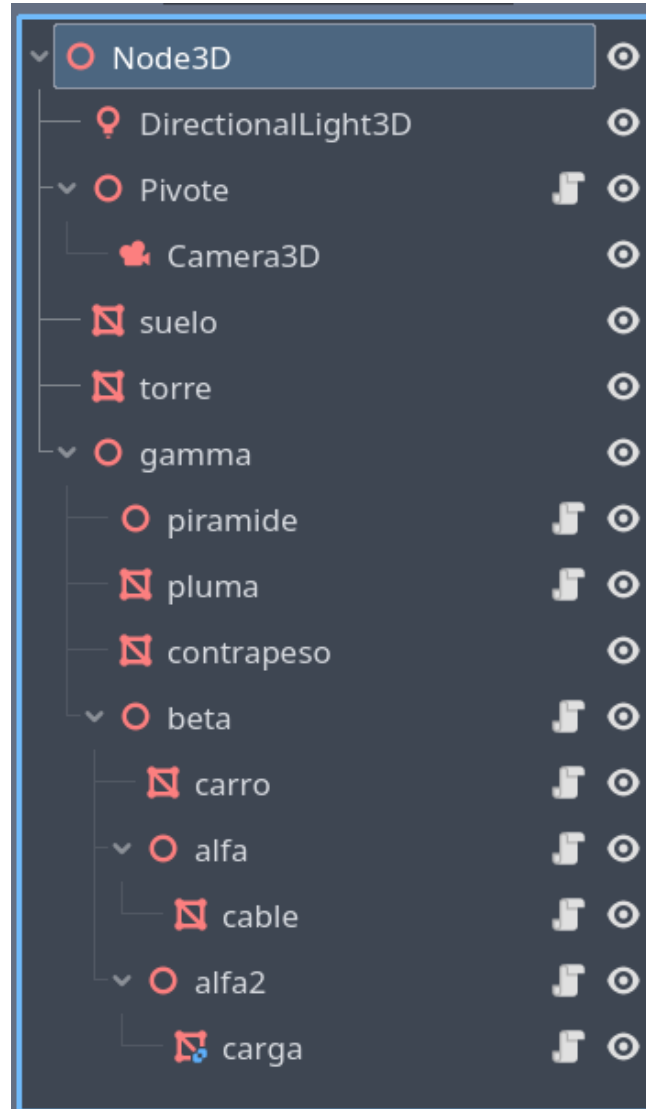
! Ejemplo de modelo jerárquico (grúa).



4. Transformaciones geométricas

! Ejemplo de modelo jerárquico (grúa).

Nodos para
grados de
libertad

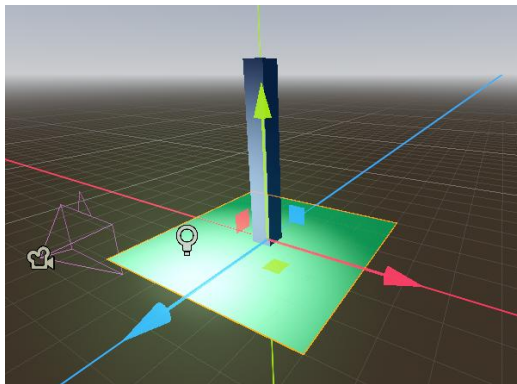
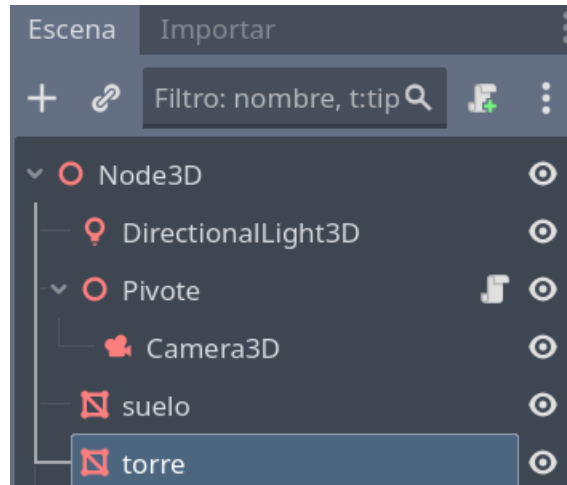


Jerarquía final

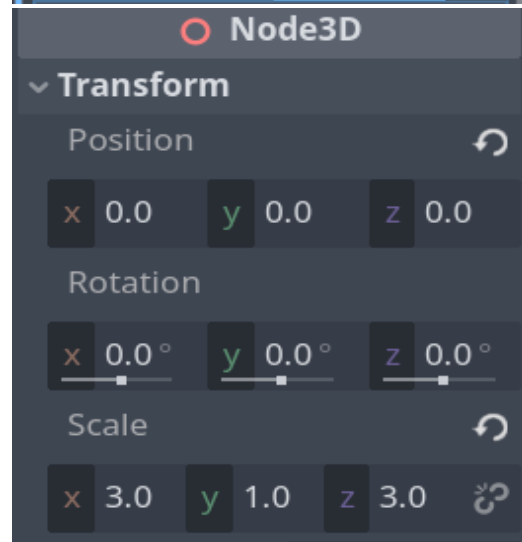
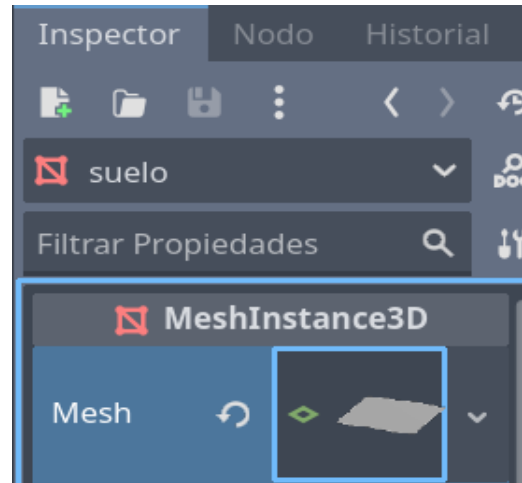


4. Transformaciones geométricas

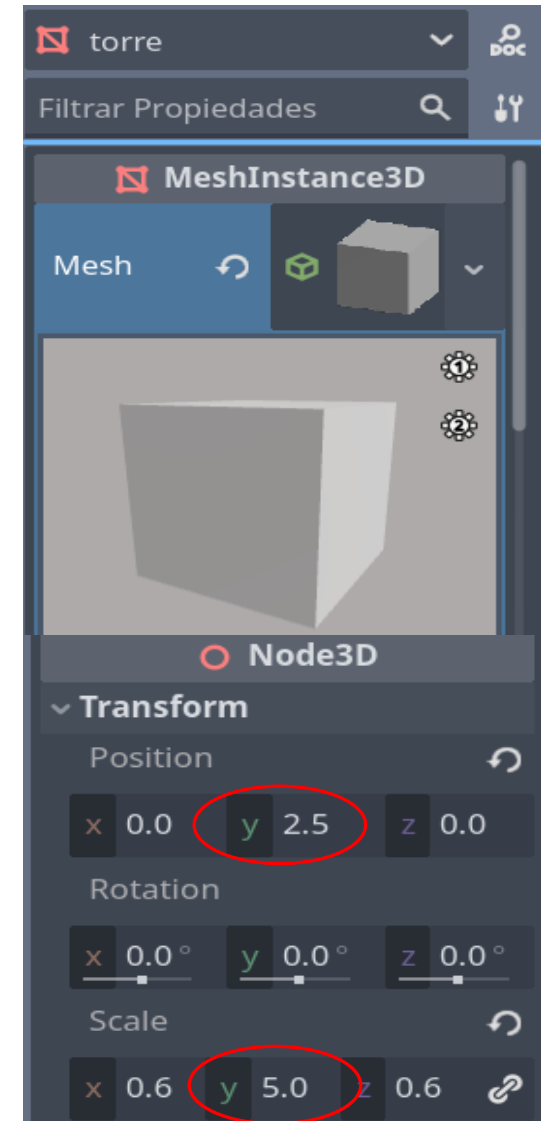
! Ejemplo de modelo jerárquico (grúa).



suelo: tipo **PlaneMesh**



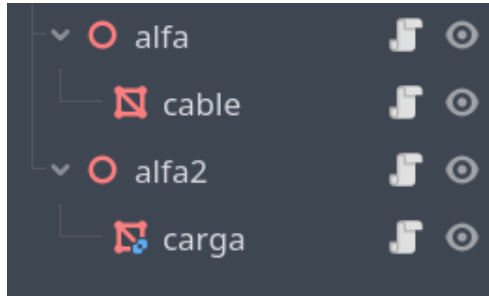
torre: tipo **BoxMesh**



4. Transformaciones geométricas

! Ejemplo de modelo jerárquico (grúa).

cable: tipo **CylinderMesh**



Jerarquía cable y carga



```
# cable.gd
```

```
extends MeshInstance3D
```

```
func _ready() -> void:
```

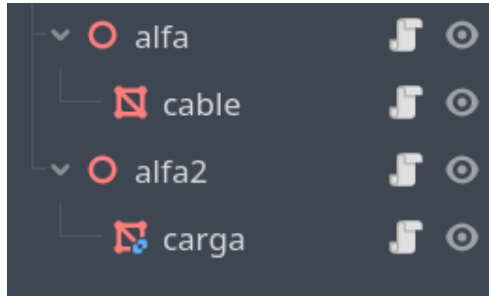
```
    translate(Vector3(0, -0.5, 0))
```

```
    scale = (Vector3(0.1, 0.5, 0.1))
```



4. Transformaciones geométricas

! Ejemplo de modelo jerárquico (grúa).



Jerarquía cable y carga

```
# alfa.gd
extends Node3D
@export var cambio := 0.02
@export var escalar = 0.25 # tamaño inicial cable
```

```
func _ready():
    translate(Vector3(-1.5, -escalar/2.0-0.1, 0.0))
    _process(cambio)
```

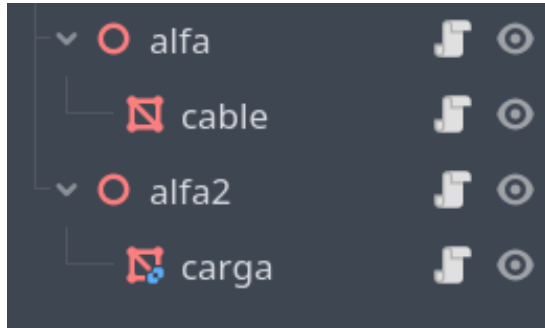
```
func _process(cambio):
    if Input.is_action_pressed("bajar_cable"):
        escalar += cambio
        if escalar > 4.3: escalar = 4.3 # restricciones
    elif Input.is_action_pressed("subir_cable"):
        escalar -= cambio
        if escalar < 0.1: escalar = 0.1 # restricciones
    scale=Vector3(1.0, escalar, 1.0)
```



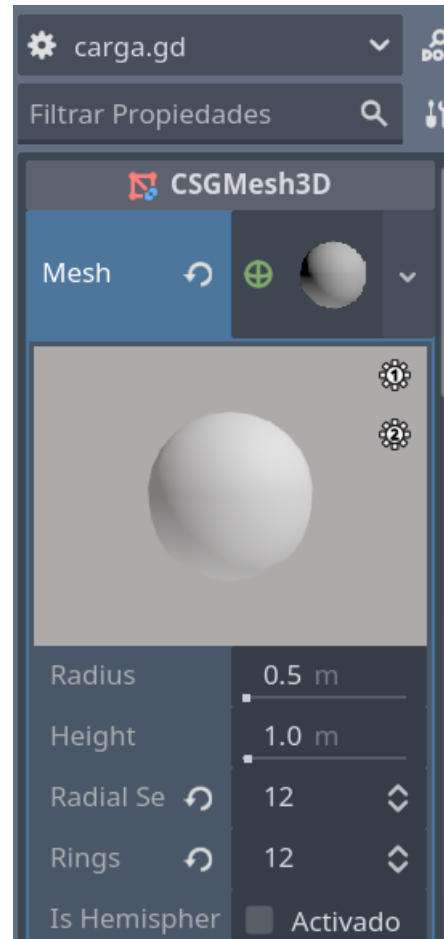
4. Transformaciones geométricas

! Ejemplo de modelo jerárquico (grúa).

carga: tipo **CSGMesh3D**



Jerarquía cable y carga



```
# carga.gd
```

```
extends CSGMesh3D
```

```
func _ready() -> void:
```

```
    translate(Vector3(0, -0.3, 0))
```

```
    scale = Vector3(0.6, 0.6, 0.6)
```



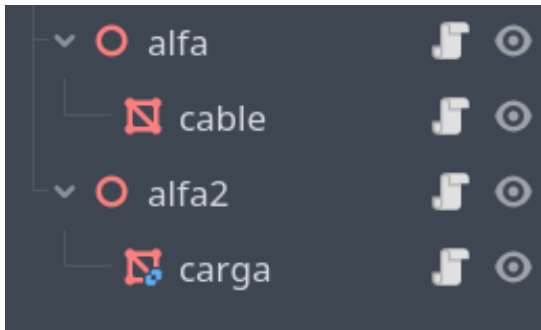
4. Transformaciones geométricas

! Ejemplo de modelo jerárquico (grúa).

```
# alfa2.gd
extends Node3D
@export var cambio := 0.02
@export var escalar = 0.25
```

```
func _ready():
    _process(cambio)
```

```
func _process(cambio):
    # para no acumular traslaciones se fija siempre
    # la posición inicial de la carga
    position=Vector3(-1.5, -0.25, 0.0)
    if Input.is_action_pressed("bajar_cable"):
        escalar += cambio
        if escalar > 4.3: escalar = 4.3 # restricciones
    elif Input.is_action_pressed("subir_cable"):
        escalar -= cambio
        if escalar < 0.1: escalar = 0.1 # restricciones
    translate(Vector3(0,-escalar, 0))
```



Jerarquía cable y carga



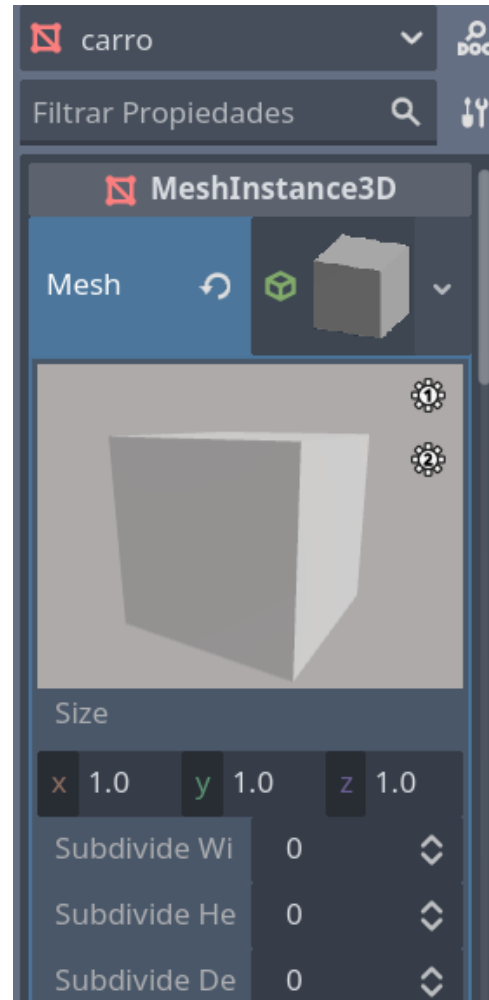
4. Transformaciones geométricas

! Ejemplo de modelo jerárquico (grúa).

carro: tipo **BoxMesh**



Jerarquía cable y carga
con el carro



carro.gd

extends Node3D

func _ready() -> void:

translate(Vector3(-1.5, -0.2, 0))

scale=Vector3(0.5, 0.2, 0.4)



4. Transformaciones geométricas

! Ejemplo de modelo jerárquico (grúa).

```
# beta.gd  
extends Node3D  
@export var incremento := 0.1  
var trasladar := 0.0
```



Jerarquía cable y carga
con el carro

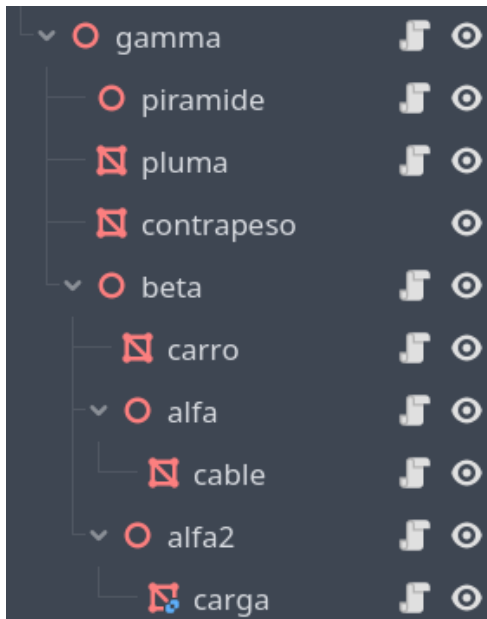
```
func _ready():  
    _process(incremento)  
  
func _process(incremento):  
    # para no acumular traslaciones se fija siempre  
    # la posición inicial del carro  
    position=Vector3(-0.5, 0.0, 0.0)  
    if Input.is_action_pressed("mover_izquierda"):  
        trasladar -= incremento  
        if trasladar < -1.2 : trasladar = -1.2 # restricciones  
    elif Input.is_action_pressed("mover_derecha"):  
        trasladar += incremento  
        if trasladar > 2.2 : trasladar = 2.2 # restricciones  
    translate(Vector3(trasladar, 0, 0))
```



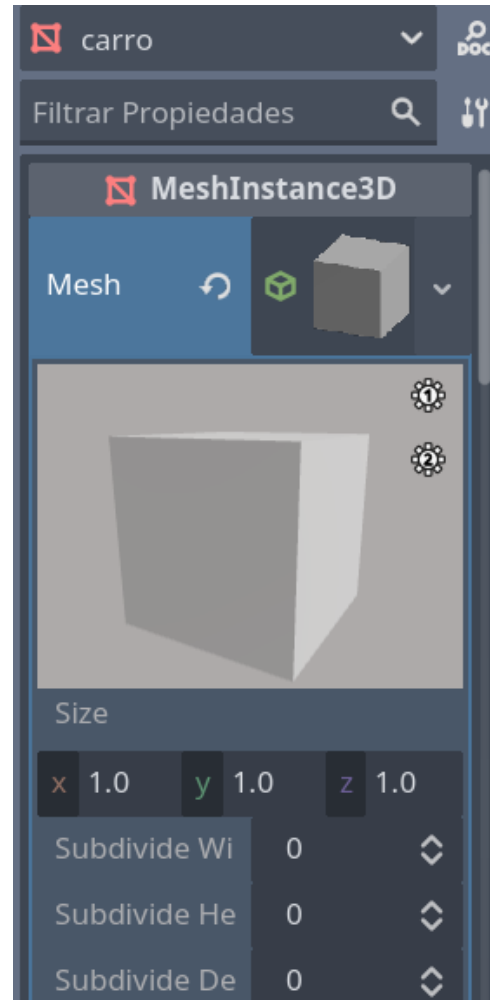
4. Transformaciones geométricas

! Ejemplo de modelo jerárquico (grúa).

pluma: tipo **BoxMesh**



Jerarquía final parte articulada



```
# pluma.gd
extends MeshInstance3D

func _ready():
    scale=Vector3(7.0, 0.2, 0.4)
```

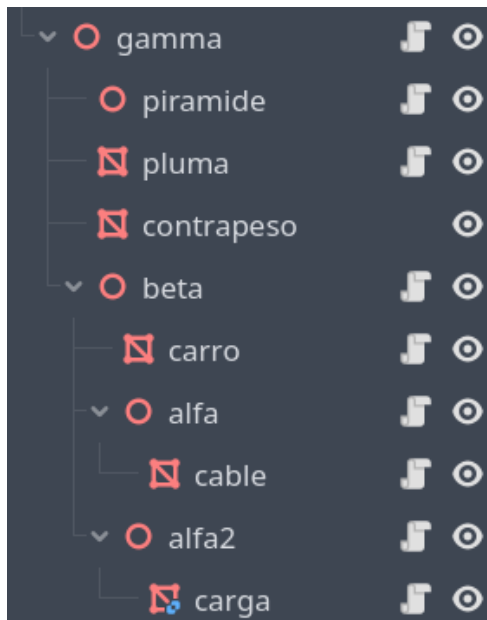


4. Transformaciones geométricas

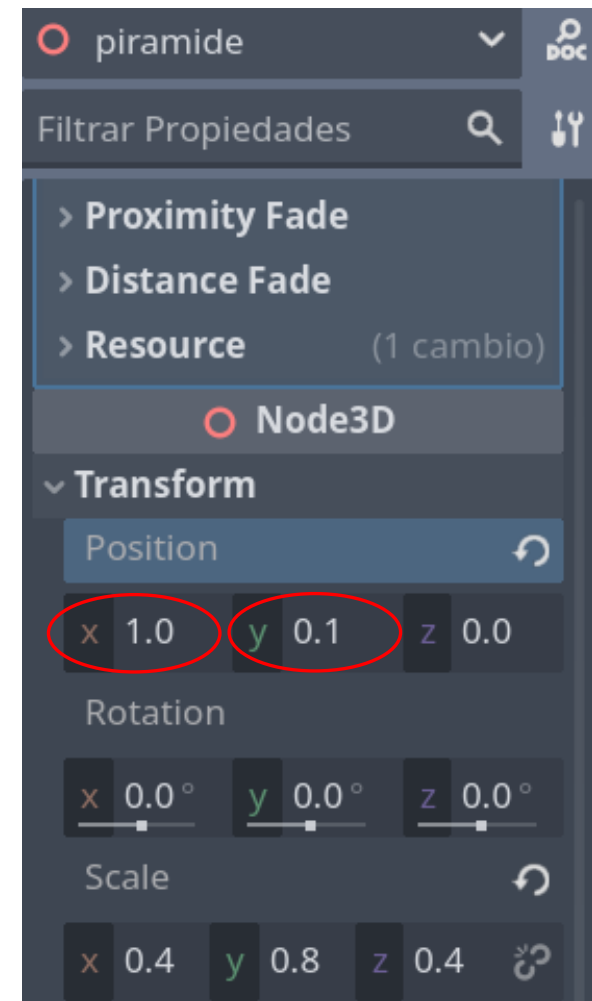
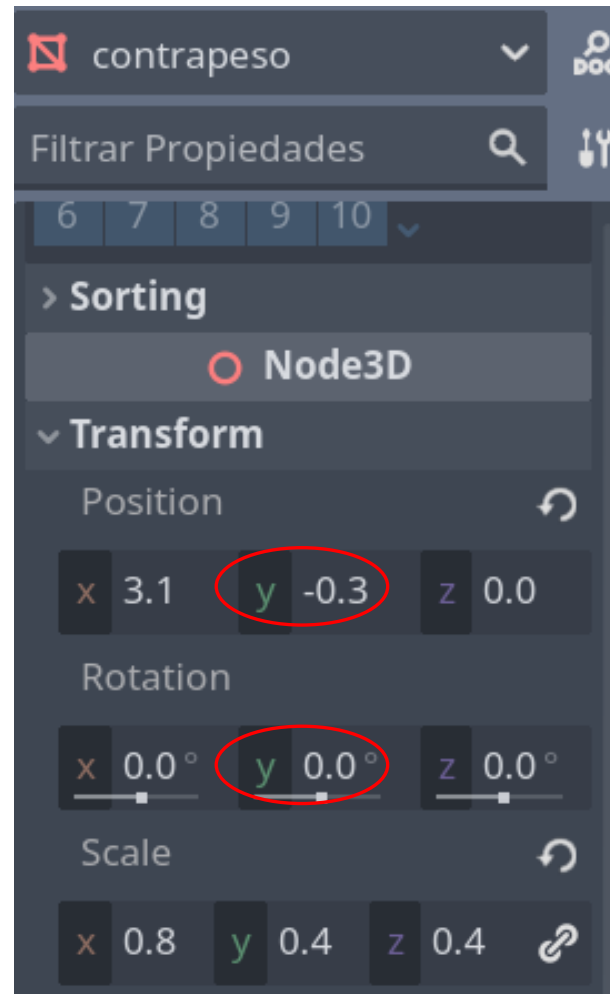
! Ejemplo de modelo jerárquico (grúa).

contrapeso: tipo **BoxMesh**

pirámide: tipo **Node3D**

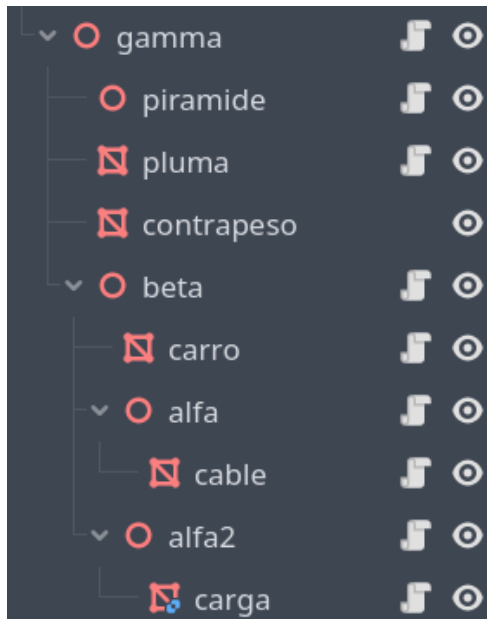


Jerarquía final parte articulada



4. Transformaciones geométricas

! Ejemplo de modelo jerárquico (grúa).



Jerarquía final parte articulada

```
# gamma.gd
```

```
extends Node3D
```

```
@export var incremento := 0.2 # en radianes
```

```
var angulo := 0.0
```

```
func _ready():
```

```
    _process(incremento)
```

```
func _process(incremento):
```

```
    # posición final de la pluma y elementos dependientes
```

```
    position = Vector3(-1.0, 5.1, 0.0)
```

```
    rotation = Vector3(0.0, 0.0, 0.0)
```

```
    if Input.is_action_pressed("girar_izquierda"):
```

```
        angulo -= incremento
```

```
        if angulo < -PI/2.0 : angulo = -PI/2.0 # restricciones
```

```
    elif Input.is_action_pressed("girar_derecha"):
```

```
        angulo += incremento
```

```
        if angulo > PI/2.0 : angulo = PI/2.0 # restricciones
```

```
    translate(Vector3(1.0, 0.0, 0.0))
```

```
    rotate_y(angulo) # rotación con respecto a (1, 0, 0)
```

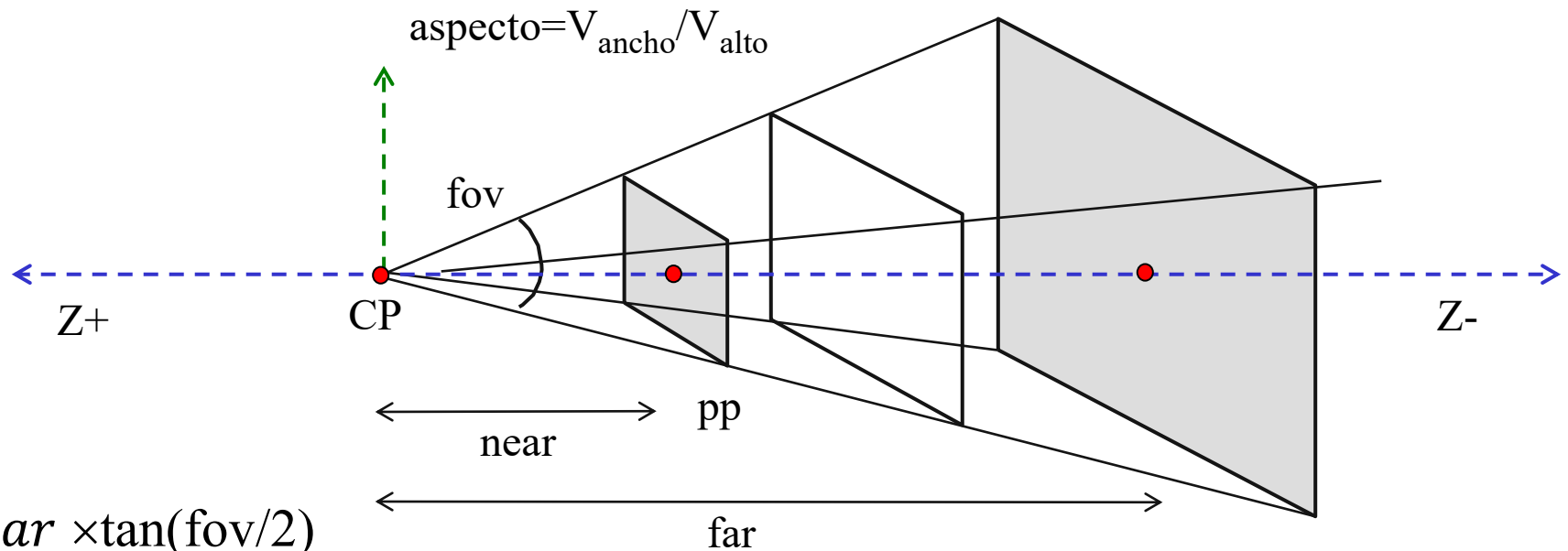
```
    translate(Vector3(-1, 0.0, 0.0))
```



5. Transformaciones de proyección

! Proyección en perspectiva.

Por defecto, la cámara está situada en el origen apuntando a Z negativo. Volumen de vista especificando el ángulo del campo de visión en el plano X-Y (fov), relacionado con la proporción ancho/alto del viewport ($\text{aspecto} = V_{\text{ancho}} / V_{\text{alto}}$) y los planos de recorte *near* y *far*. El plano de proyección coincide con *near*



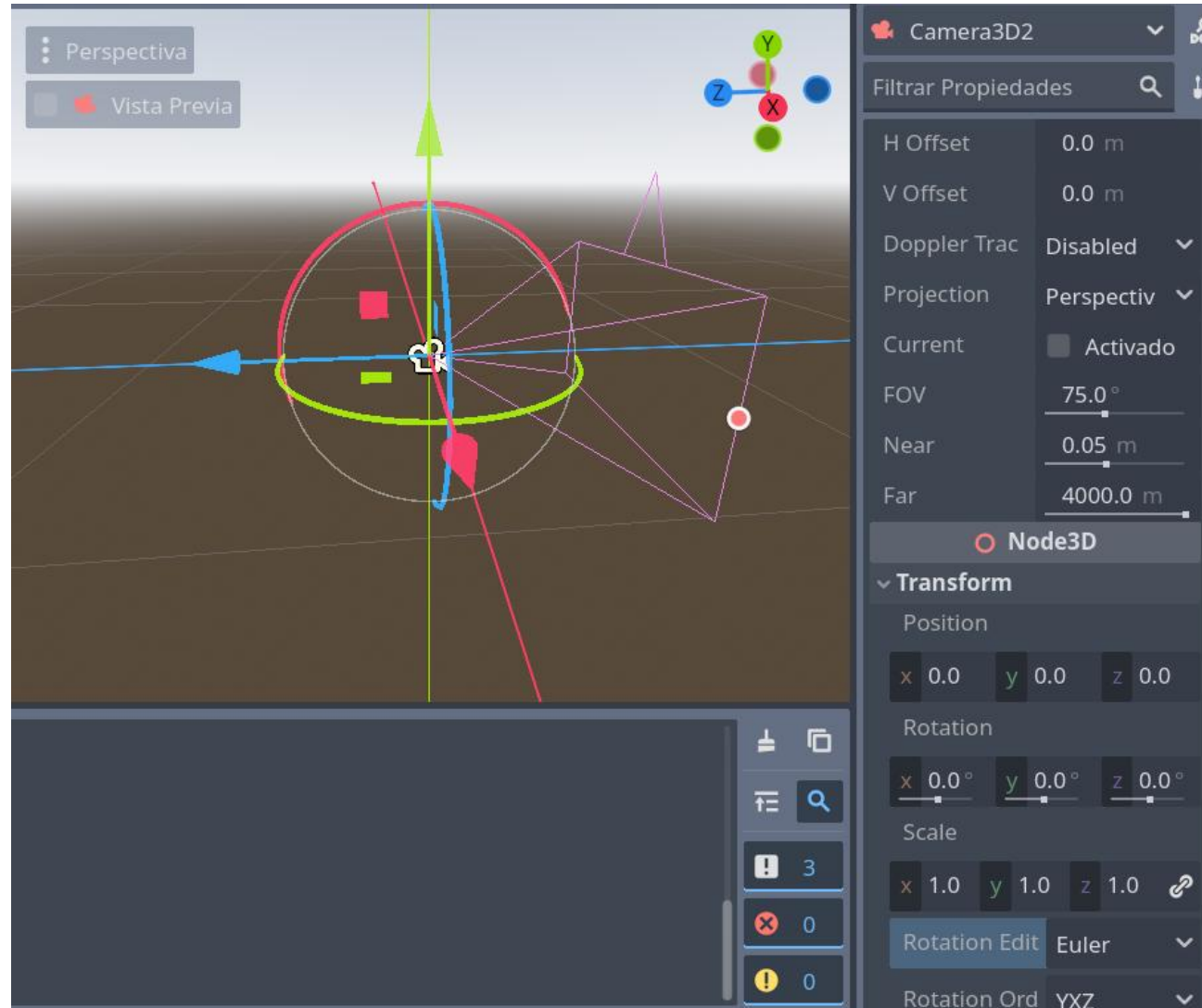
$$W_{\text{alto}} = 2 \times \text{near} \times \tan(\text{fov}/2)$$

$$W_{\text{ancho}} = W_{\text{alto}} \times \frac{V_{\text{ancho}}}{V_{\text{alto}}}$$



5. Transformaciones de proyección

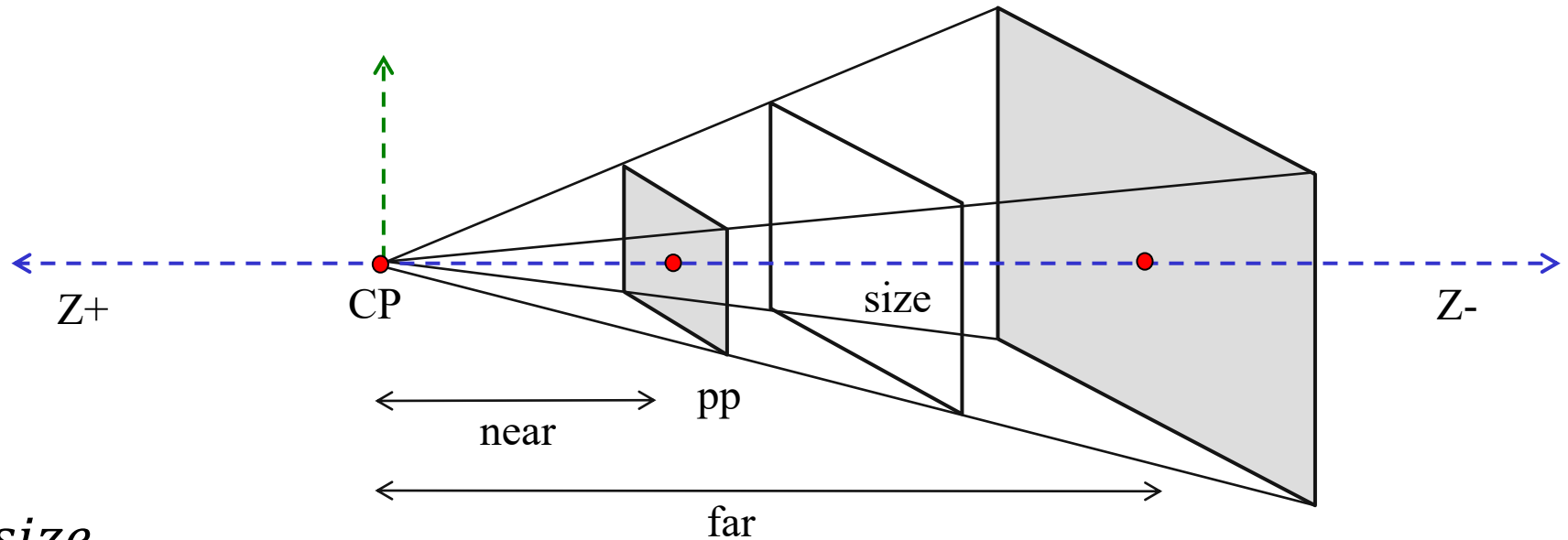
! Proyección en perspectiva.



5. Transformaciones de proyección

! Proyección frustrum

Cámara 3D también situada en el origen apuntando a Z negativo. Volumen de vista especificando los planos de recorte **near** y **far**. El parámetro **size** controla la altura del plano de proyección que coincide con **near**



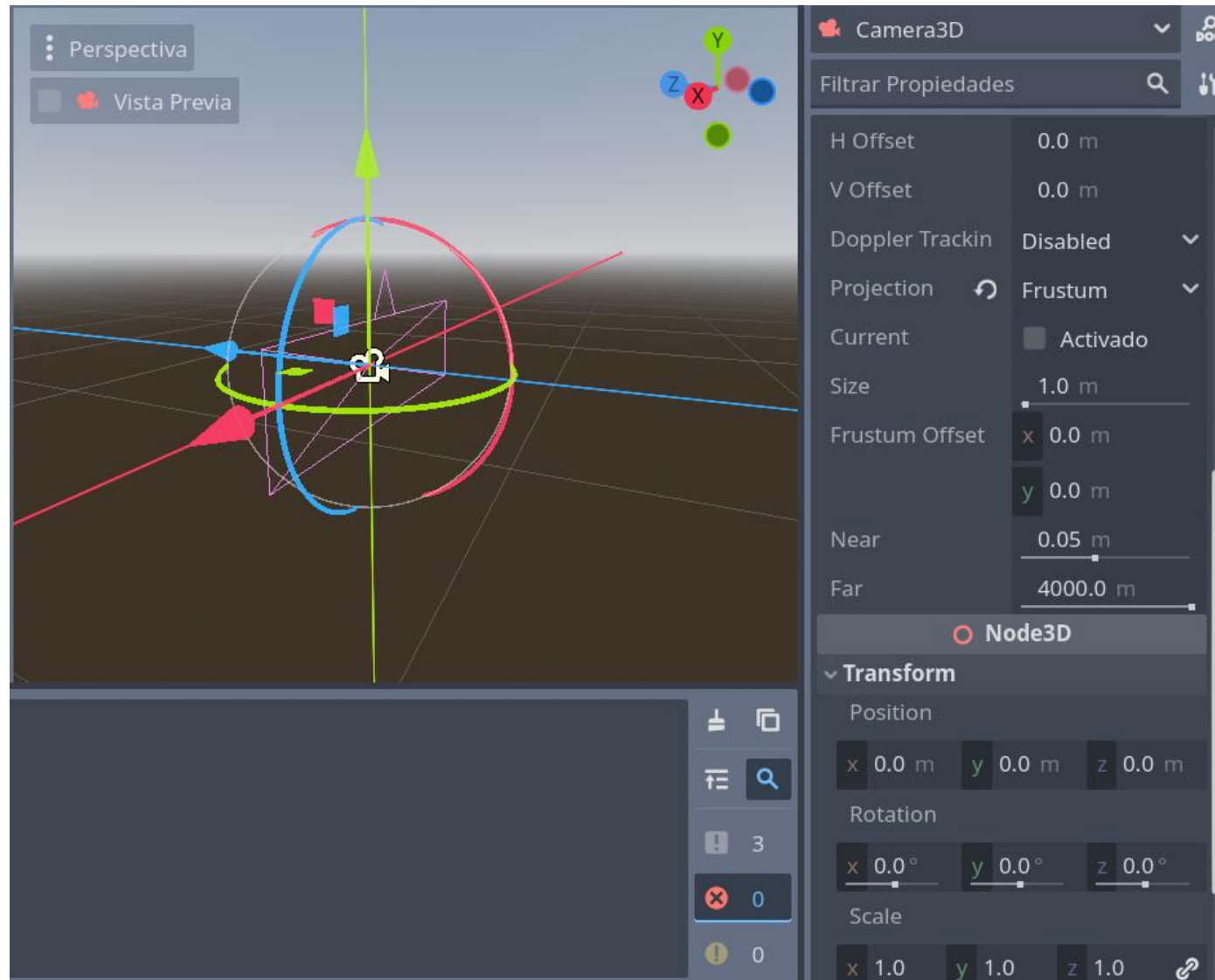
$$W_{alto} = size$$

$$W_{ancho} = size \times \frac{V_{ancho}}{V_{alto}}$$



5. Transformaciones de proyección

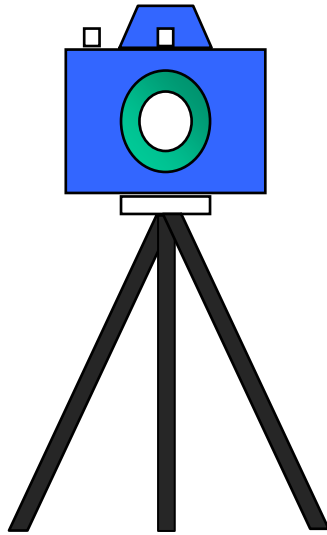
! Proyección frustrum



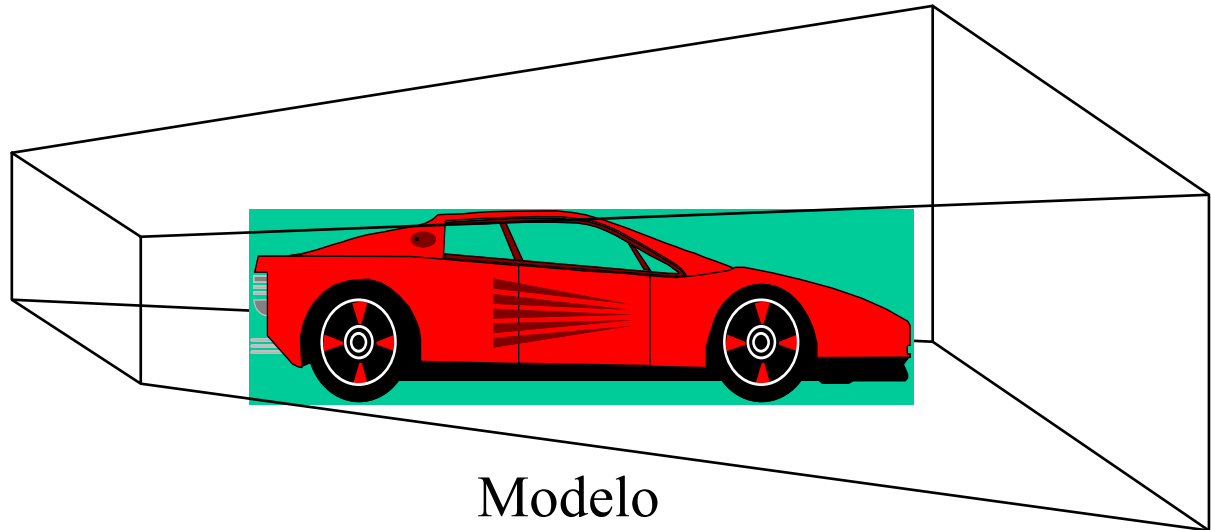
5. Transformaciones de proyección

! Atención a la colocación de los objetos con respecto al volumen de vista.

Posición de la cámara



Volumen de vista

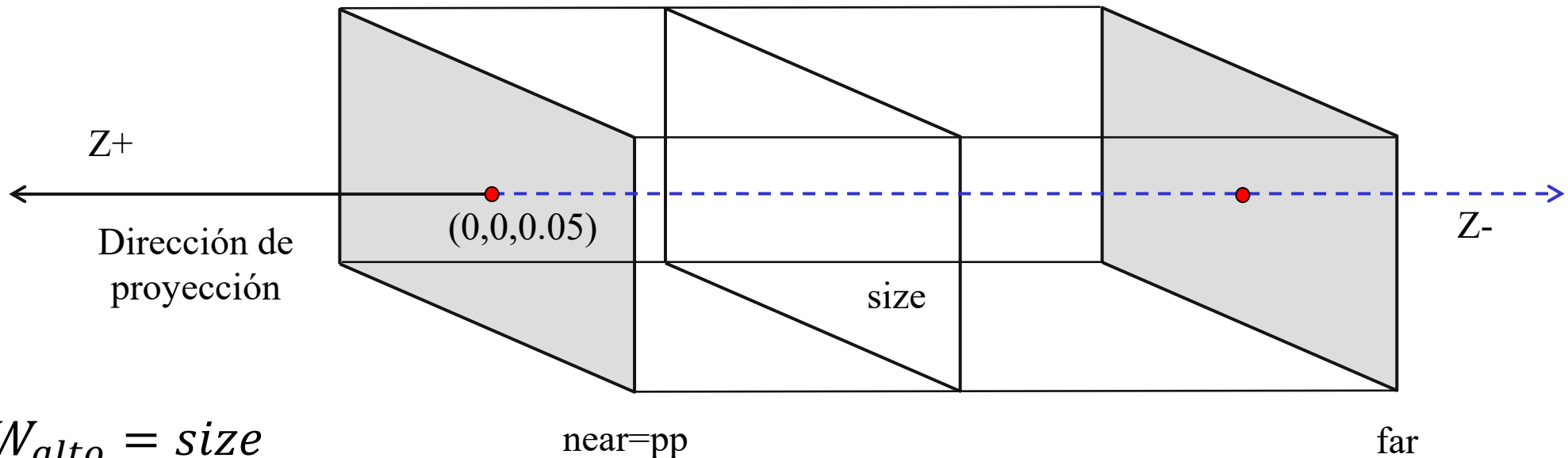


Modelo

5. Transformaciones de proyección

! Proyección ortográfica

Se delimita un volumen de vista para una proyección ortográfica en alzado hacia Z negativo. **Near** y **far** definen los planos de recorte de dicho volumen, **size** la altura del plano de proyección. La dirección de proyección es paralela al eje **Z**.



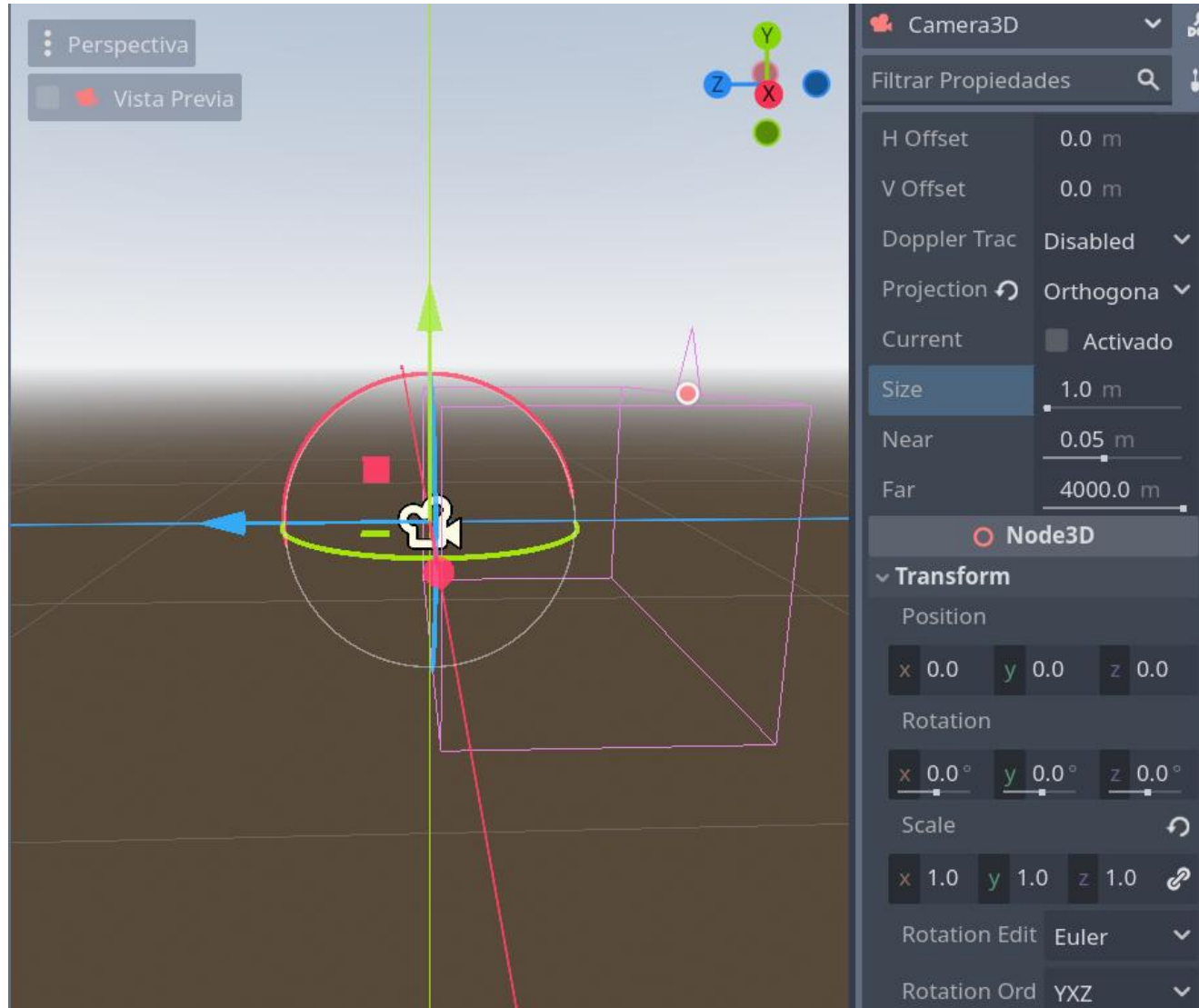
$$W_{alto} = size$$

$$W_{ancho} = size \times \frac{V_{ancho}}{V_{alto}}$$



5. Transformaciones de proyección

! Proyección ortográfica



5. Transformaciones de proyección

! Cámara orbital (zoom no aplicable a una ortográfica)

```
# pivote.gd
extends Node3D
@export var incremento := 0.02 # velocidad angular en rad/s
@export var trasladar := 5.0 # posición inicial en z de la cámara
@export var incremento2 := 0.05 # velocidad del zoom de la cámara
@export var umbral := 0.02 # evita valores negativos del zoom
var angulo := 0.0 # rotación en X
var angulo2 := 0.0 # rotación en y
var acciones := ["orbitar_izquierda", "orbitar_derecha",
                 "orbitar_abajo", "orbitar_arriba", "alejar", "acercar"]

func _ready():
    var delta = 0
    _process(delta)

func _process(delta):
    # se inicializa la rotación y la traslación de la cámara
    rotation=Vector3(0,0,0.0)
    position=Vector3(0,0,0.0)
```



5. Transformaciones de proyección

! Cámara orbital (zoom no aplicable a una ortográfica)

```
func _process(delta): # pivote.gd  continuación
...
    for action in acciones:
        if Input.is_action_pressed(action):
            match action: # match equivale a un switch case
                # para rotar en Y
                "orbitar_izquierda": angulo2 -= incremento
                "orbitar_derecha":  angulo2 += incremento
                # para rotar en X
                "orbitar_abajo":    angulo -= incremento
                "orbitar_arriba":   angulo += incremento

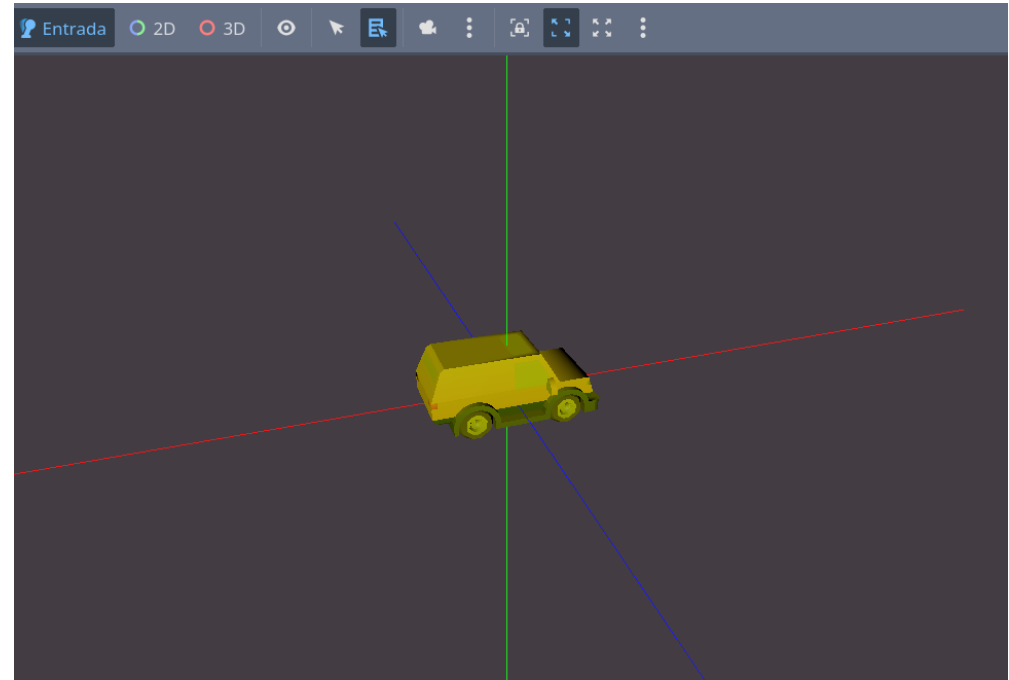
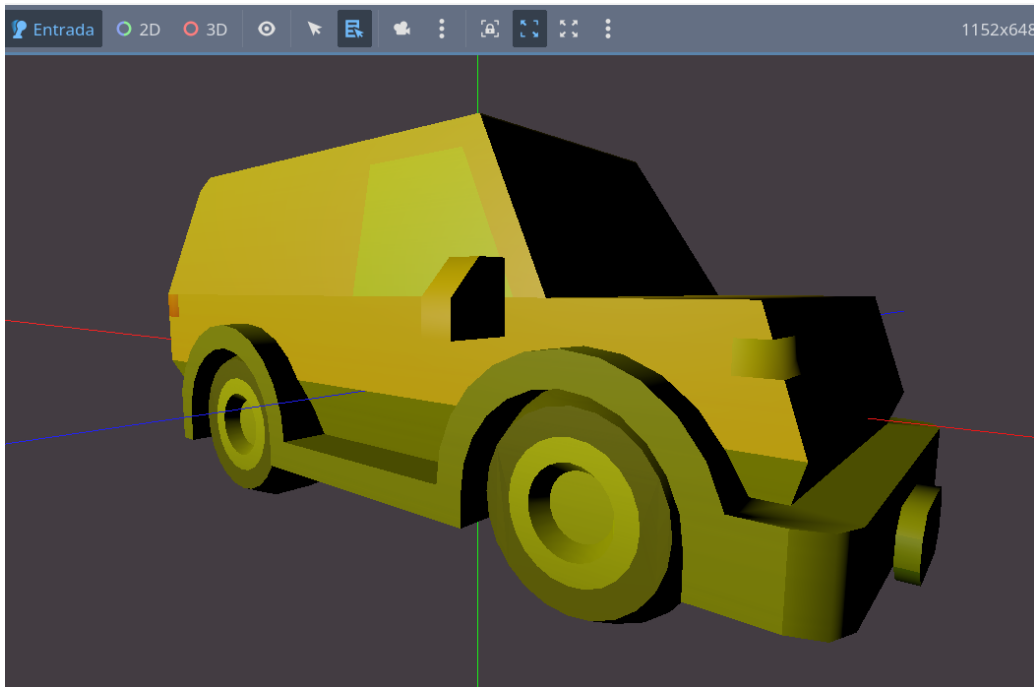
                "alejar":  trasladar *= 1.05
                "acercar":
                    trasladar -= incremento2
                    if trasladar < umbral : trasladar = umbral

            rotate_y(angulo2)
            rotate_x(angulo)
            translate(Vector3(0, 0, trasladar)) # zoom trasladando la cámara
```



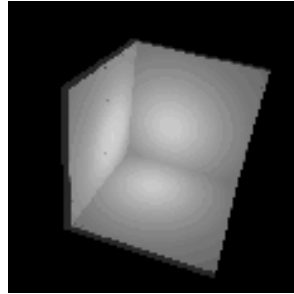
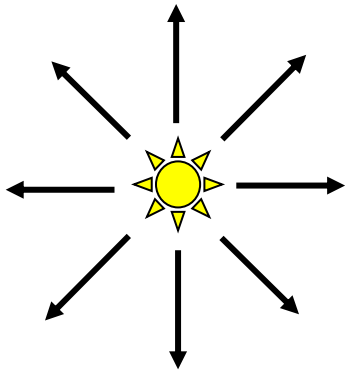
5. Transformaciones de proyección

! Cámara orbital (zoom no aplicable a una ortográfica)

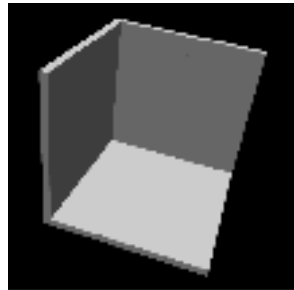
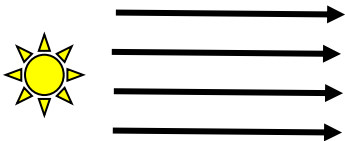


6. Iluminación

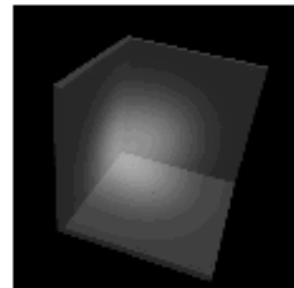
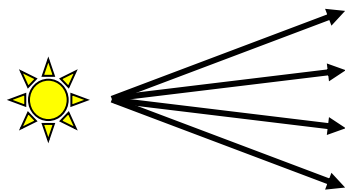
! Tipos (nodos) de luces en una escena.



Puntual: **OmniLight3D**



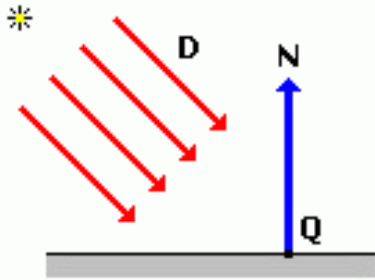
Direccional: **DirectionalLight3D**



Focal: **SpotLight3D**

6. Iluminación

! Tipos (nodos) de luces en una escena.



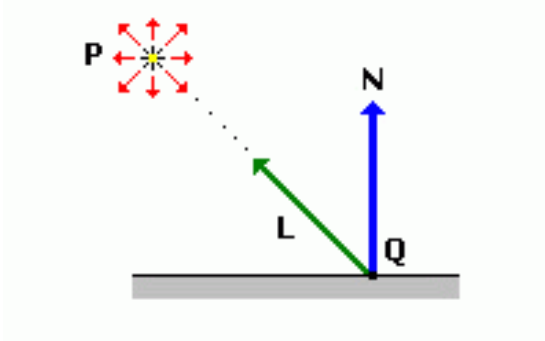
Escena exterior



Escena interior

6. Iluminación

! Tipos (nodos) de luces en una escena.



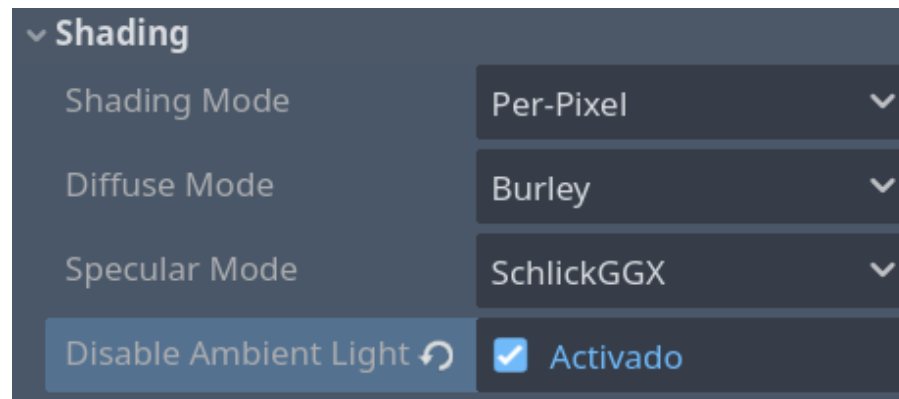
Escena interior



Escena exterior

6. Iluminación

- ! Luz ambiental, simula la iluminación indirecta.
- ! Se aplica por defecto una luz ambiental a los objetos tanto en las caras traseras como en las delanteras.
- ! Color de la luz (0.5, 0.5, 0.5)
- ! Intensidad 1.0
- ! Se puede controlar creando un nodo **WorldEnvironment**
- ! A nivel de mallas se activa o desactiva para cada una



6. Iluminación

! Propiedades de **WorldEnvironment**.

- **Background** cambia el color de fondo usado al mostrar la escena. **Custom Color**
- **Ambient Light** para cambiar la luz ambiental con un color definido por el usuario. Con **Background** o **Sky** usan los colores de fondo o del cielo respectivamente
- **Reflected Light** procedencia de la luz rebotada, si es del fondo es más uniforme
- **Tonemap** cuando hay saturación de color (valores de RGB superiores a 1, como por exceso de luces en una escena) se pueden reescalar. Con **Linear<1** hay corrección pero con valores >1 aumenta la saturación

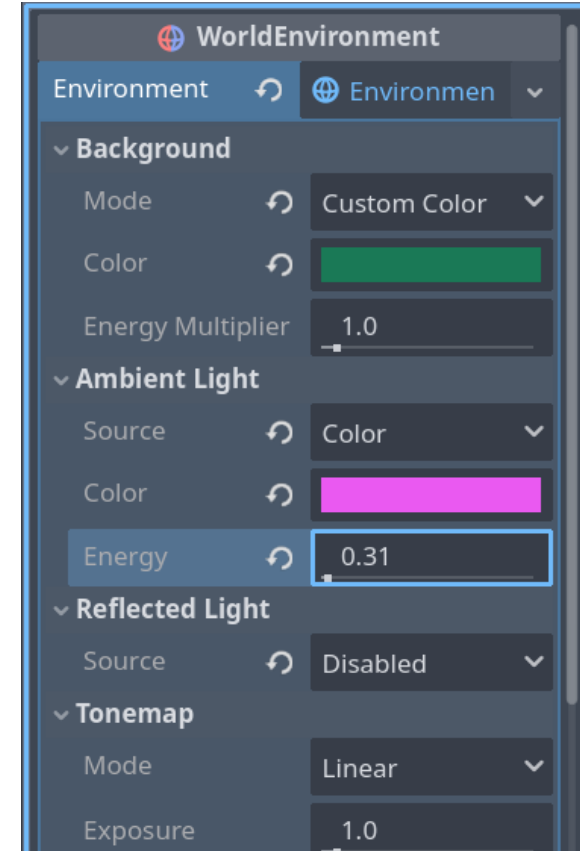
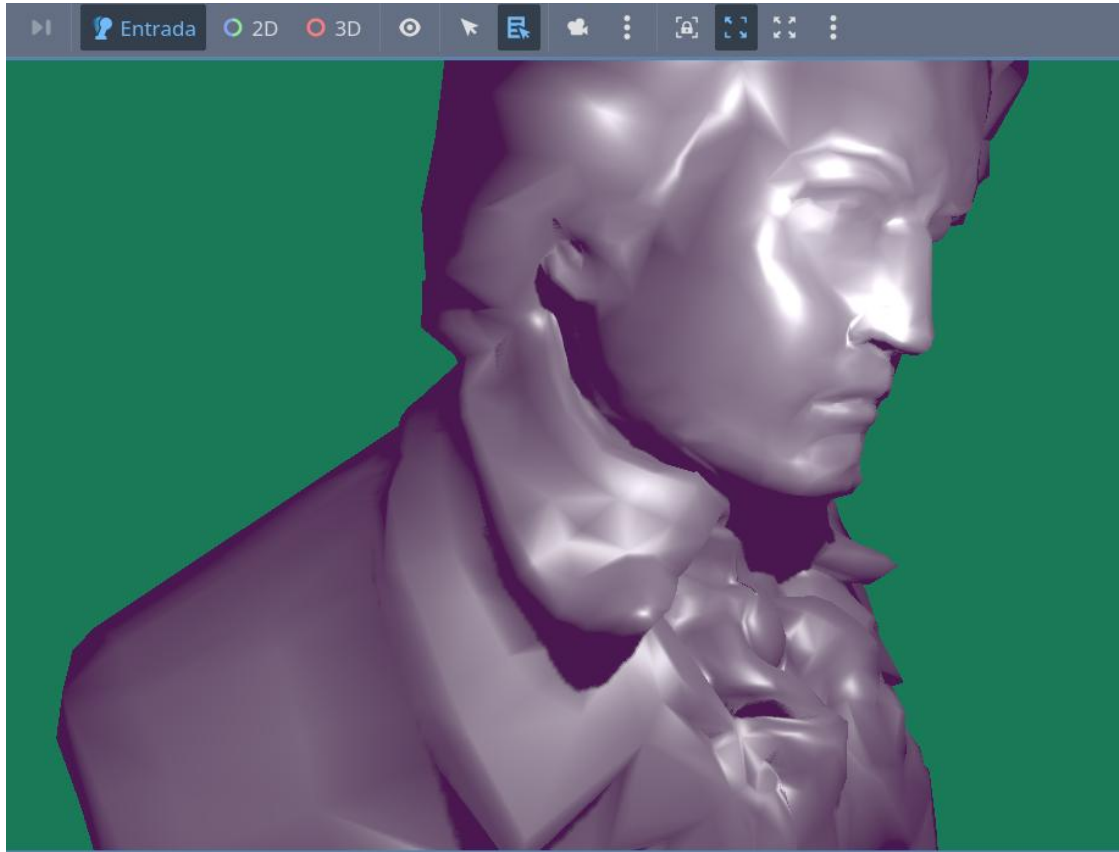


6. Iluminación

! Propiedades de **WorldEnvironment**.



Objeto gris



6. Iluminación

! Propiedades genéricas de las luces **Light3D**.

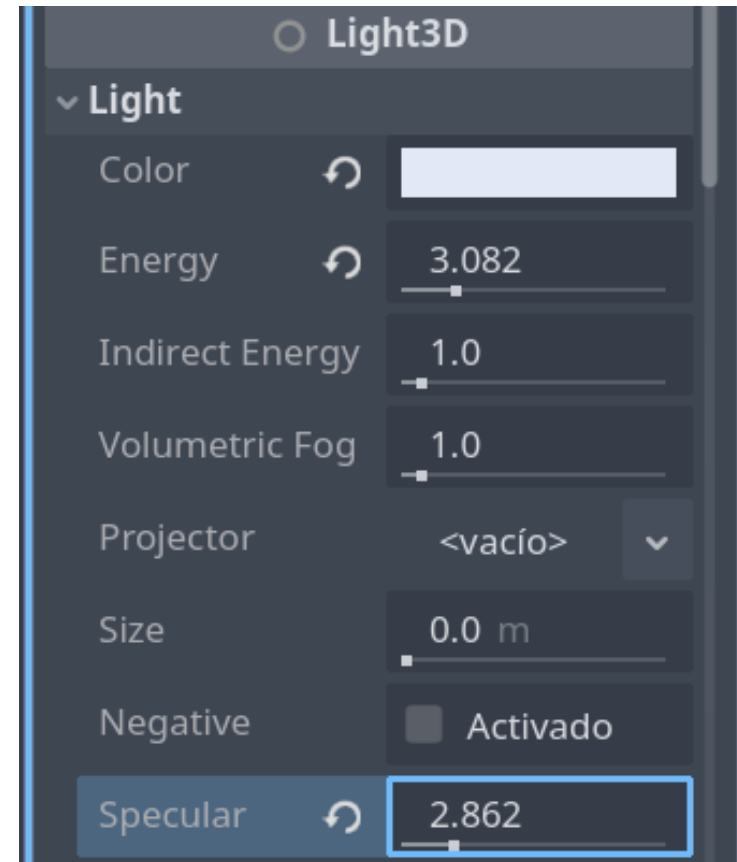
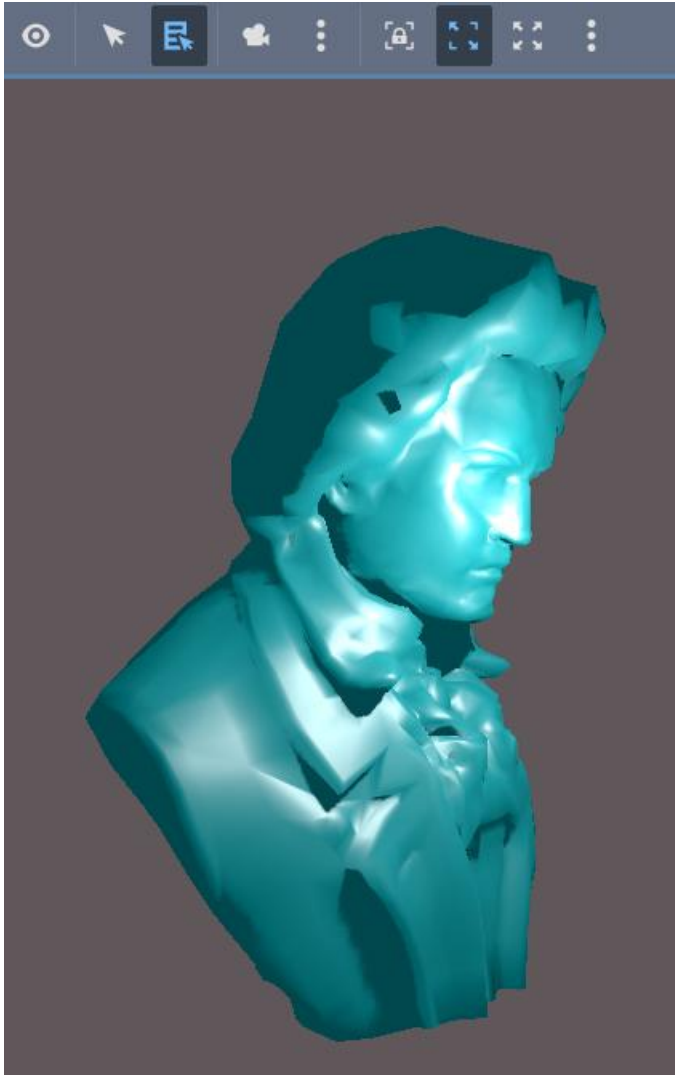
- **Energy** intensidad [0 , 16], no tiene relación con unidades físicas
- **Color** por defecto el blanco
- **Indirect Energy** cantidad de energía de la luz que contribuye a la iluminación global [0, 16]. Son los rebotes de la luz en otros objetos, se usa sobre todo en escenas interiores (valores >1)
- **Volumetric Fog Energy** como de brillante se quiere ver una luz al activar la niebla [0, 16]. Con valores >1 más intensa en niebla
- **Specular** controla el brillo que es el reflejo especular de la luz en los objetos [0,16]
- **Bake Mode** (luz horneada) Mejora el rendimiento en los cálculos de iluminación ya que estos se precaculan y se guarda en estructuras en vez de calcularlos en tiempo real para cada frame



6. Iluminación

! Propiedades genéricas de las luces **Light3D**.

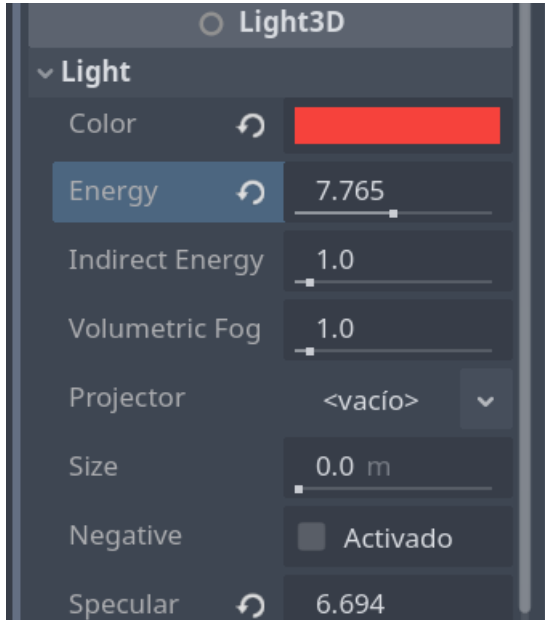
Objeto azul
cian



Luz blanca, brillo color de la luz

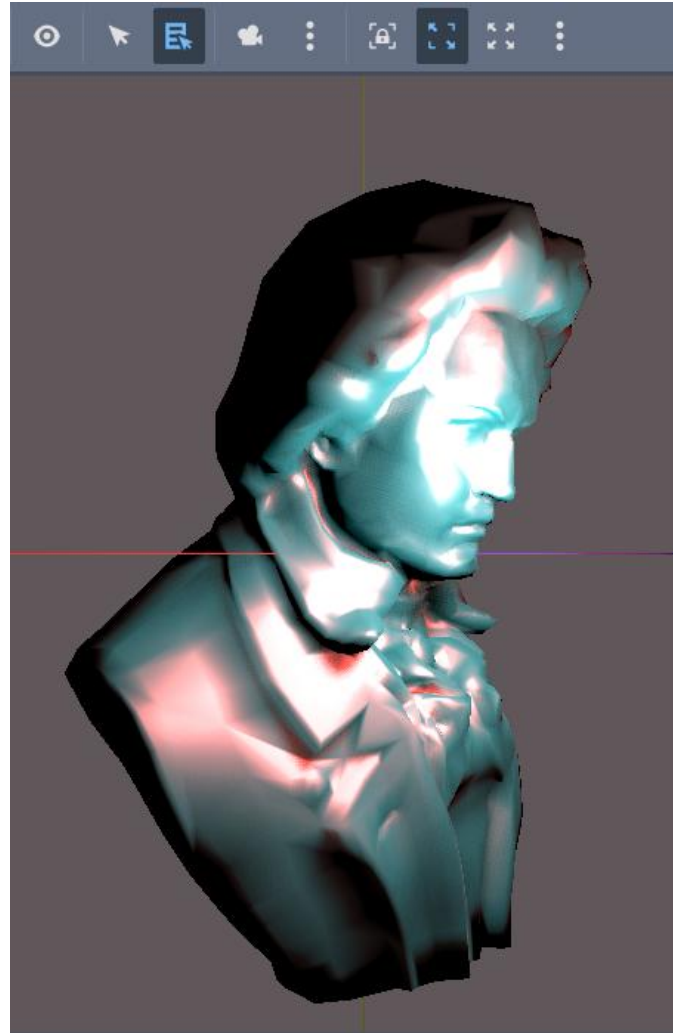
6. Iluminación

! Propiedades genéricas de las luces **Light3D**.



Luz roja. Se aprecia en los
brillos de la luz blanca

Objeto azul cian,
sin luz ambiental



Luz blanca

6. Iluminación

! Propiedades genéricas de las luces **Light3D**. Sombras

- **Bias** desplazamiento en relación al **Shadow Map** para tratar artificios en las sombras [0 , 10]. Valores bajos producen shadow acne, valores altos peter-panning
- **Normal Bias** desplazamiento en relación a la normal de las caras. También para tratar los artificios en las sombras [1,10]. Cuando los objetos son complicados o tienen caras inclinadas
- **Reverse cull face** revierte la cara trasera, se usa cuando se quiere añadir sombras a ambos lados de una cara (objetos no cerrados)
- **Transmittance bias** disminuye las auto-sombras con objetos translucidos [-16, 16] (típicos [-2, 2]. Con valores bajos riesgo de artificios, con valores altos menos visible la luz transmitida
- **Opacity** controla la opacidad [0, 1]
- **Blur** desenfoque en las sombras, las hace más suaves [0, 10]



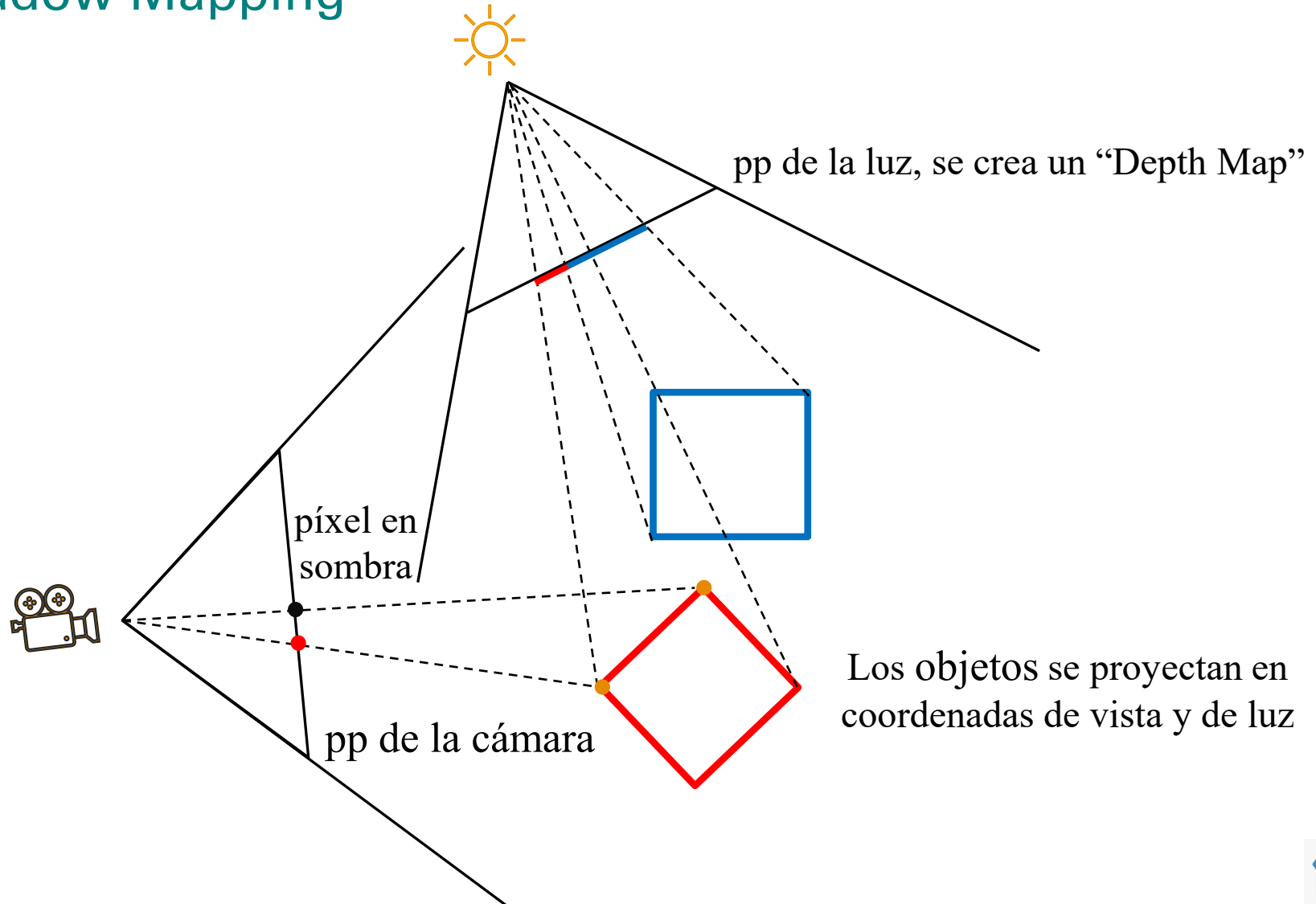
6. Iluminación

- ! Para la sombra se usa **Shadow Mapping** (algoritmo inicial por Lance Williams en 1978)
- ! La luz se considera igual que el observador (cámara). Se calcula un **Depth Map**, imagen de la escena en un plano de proyección asociado a la luz
- ! Dos pasadas:
 - Calcular primero el **Depth Map**, guardando para cada pixel la distancia a la superficie más cercana a la luz
 - Calcular la imagen para la cámara. Para cada pixel comprobar si está en sombra: para el punto que se proyecta en el píxel, sus coordenadas cartesianas se transforman al sistema cartesiano de la luz; la coordenada **z** así obtenida se compara con la coordenada **z** almacenada en el **Depth Map**; si es mayor, el pixel en la imagen estará en sombra.



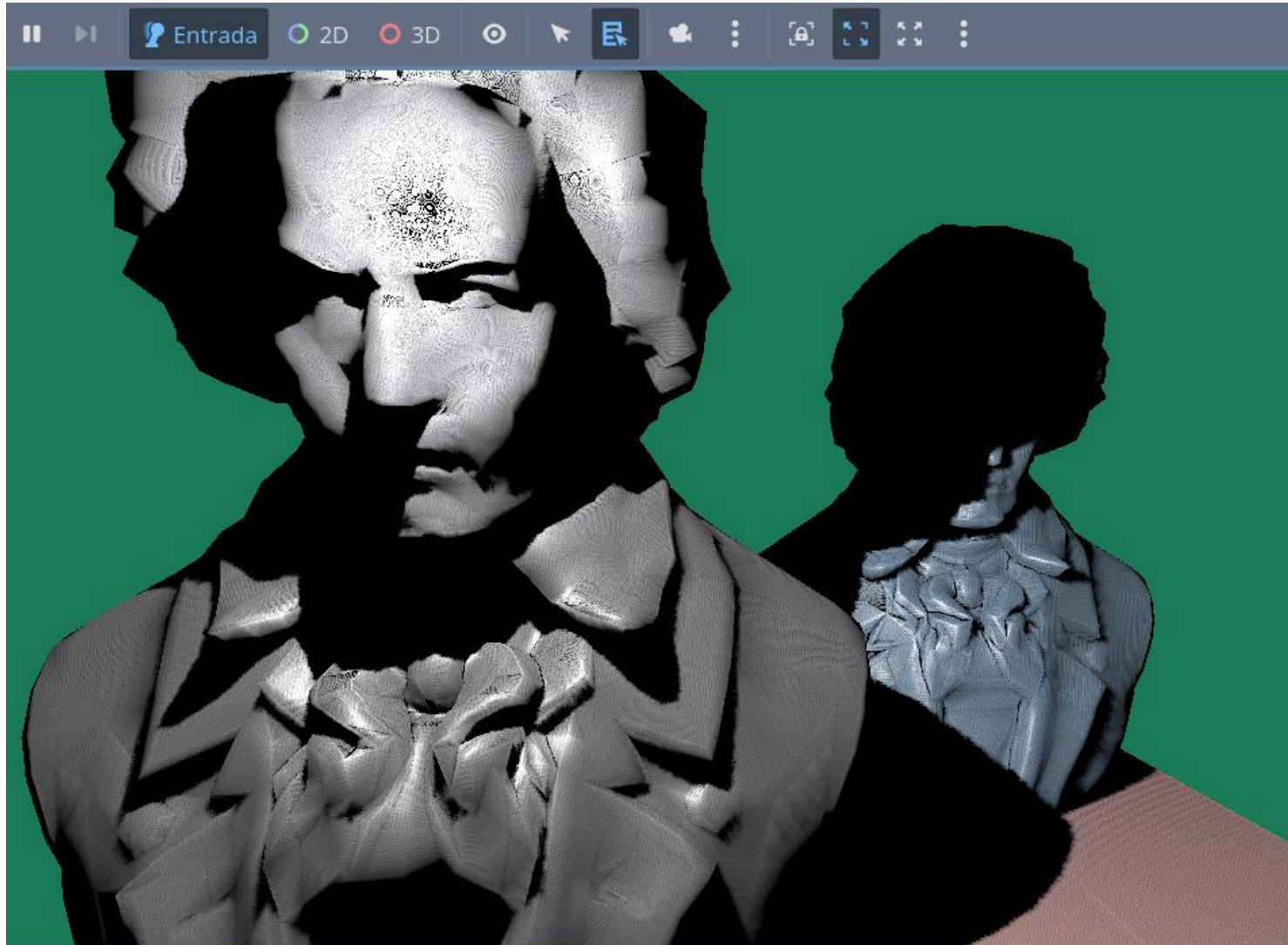
6. Iluminación

! Shadow Mapping



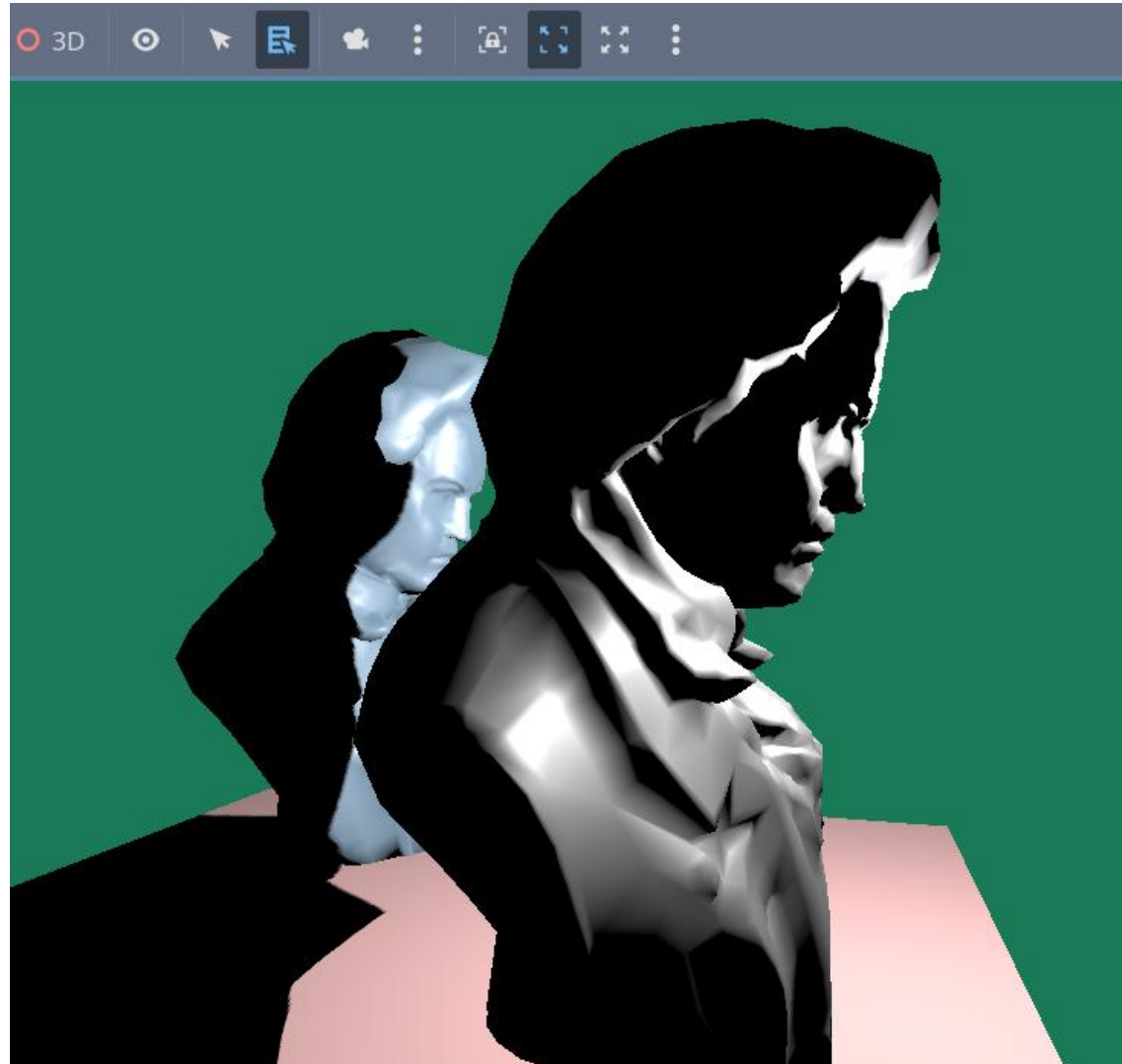
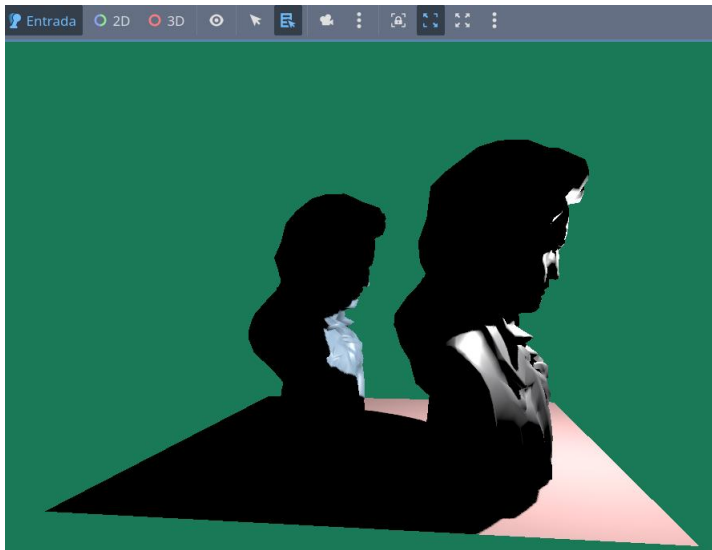
6. Iluminación

! Artificios en las sombras, shadow acne



6. Iluminación

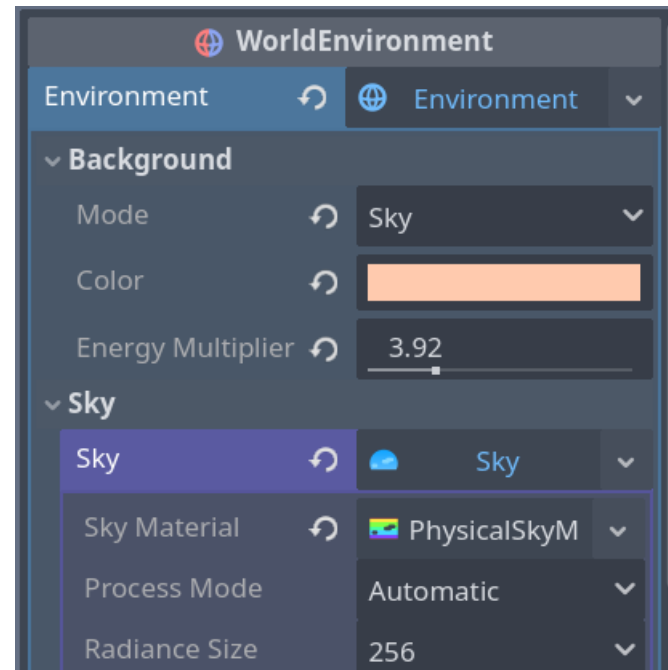
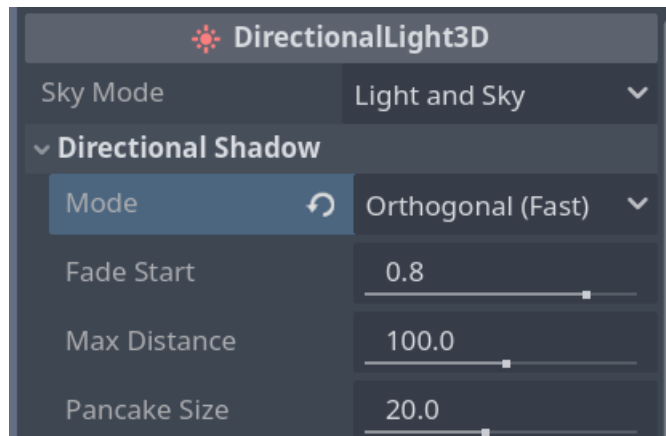
! Artificios en las sombras, peter-panning



6. Iluminación

! Luz direccional DirectionalLight3D

- Sky Mode: **Light and Sky, Light Only, Sky Only**, si la luz afecta a la escena y al cielo, solo a la escena o solo al cielo.
- Para que tenga efecto se necesita crear un nodo **WorldEnvironment** con **Background → Sky** y un **Sky material → PhysicalSky**



6. Iluminación

! Luz direccional `DirectionalLight3D`

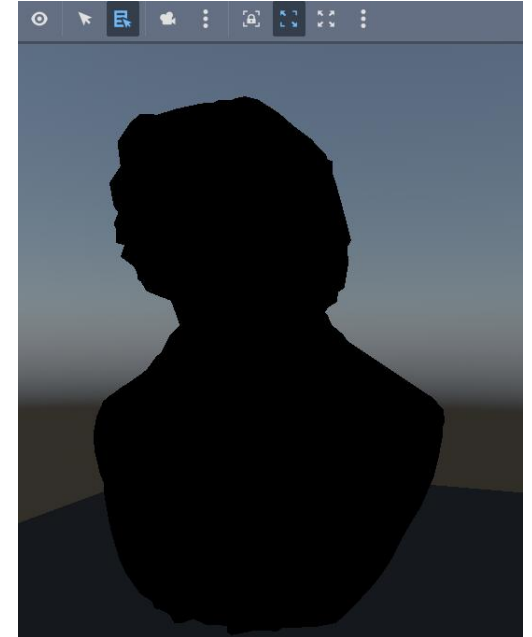
- Sky Mode



Light and Sky



Light Only

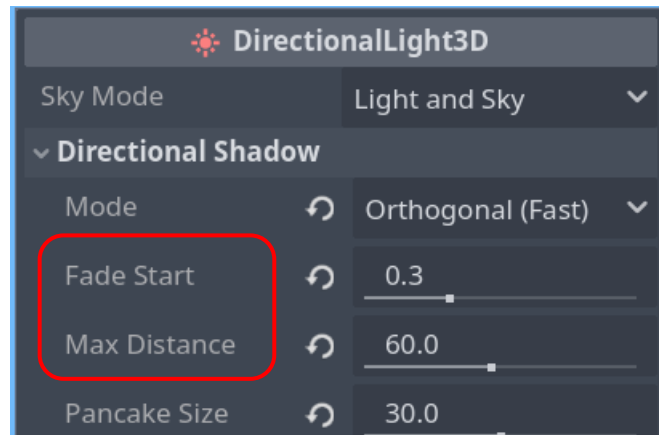
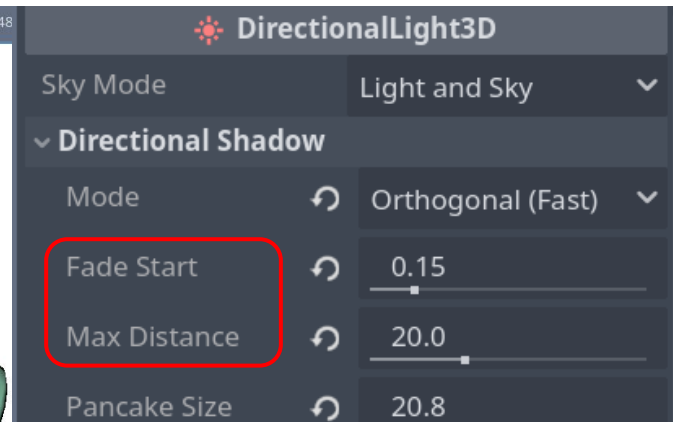
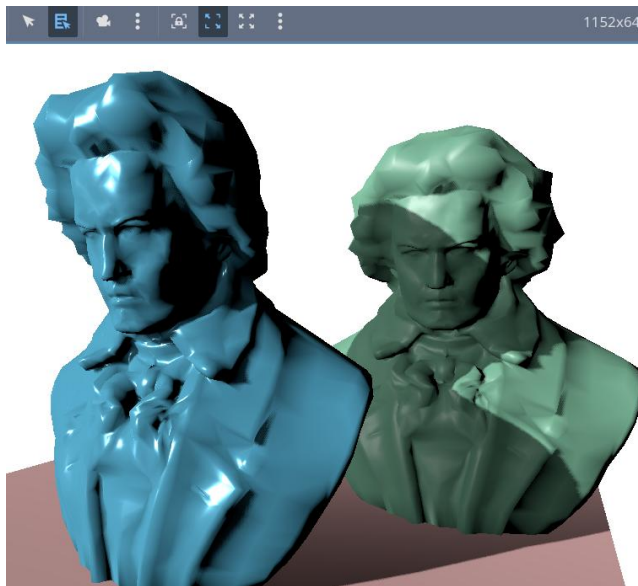


Sky Only

6. Iluminación

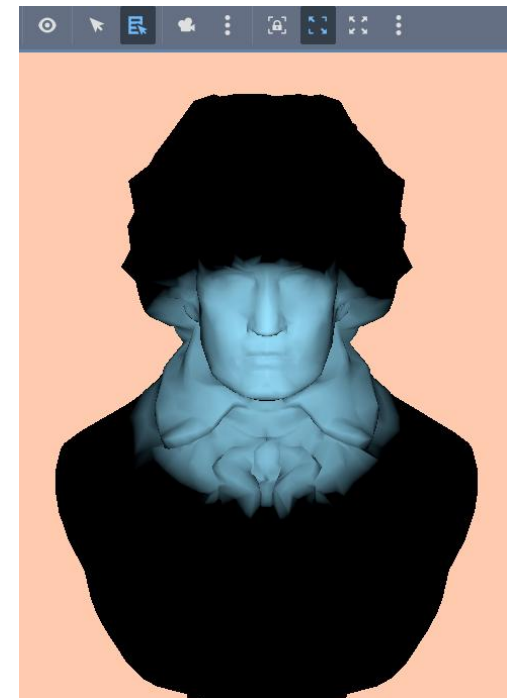
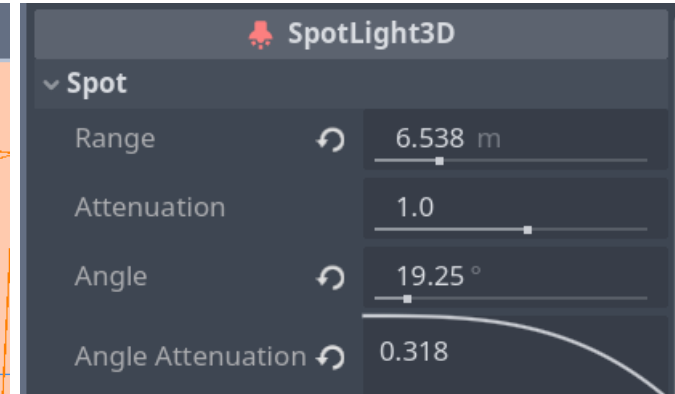
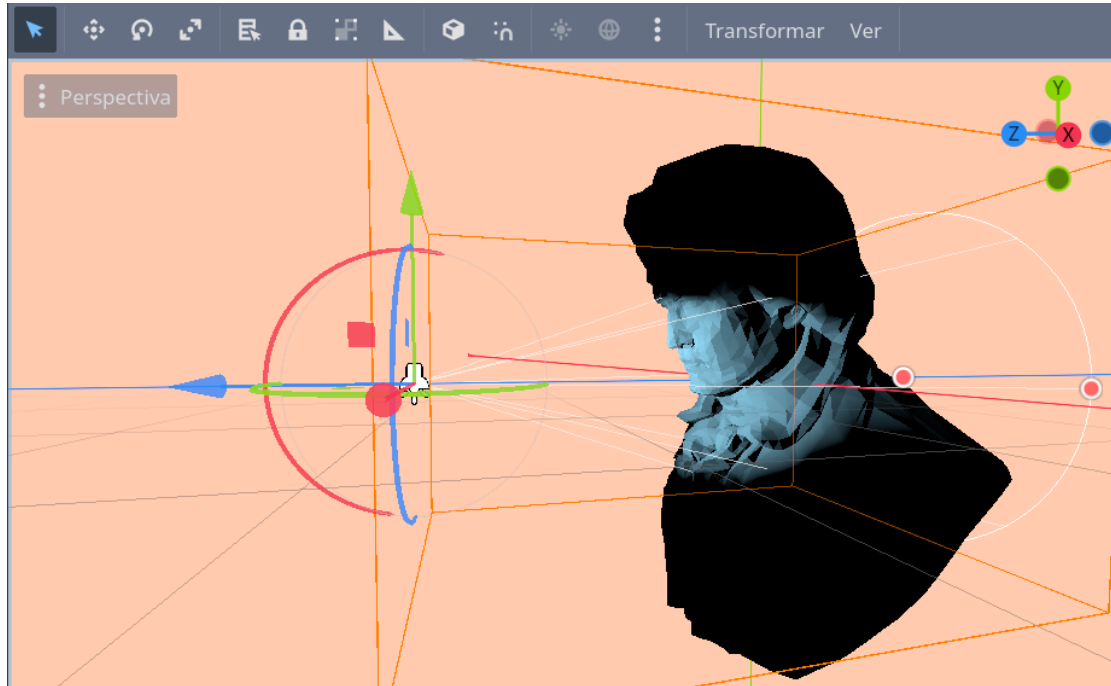
! Luz direccional **DirectionalLight3D**

- Directional Shadow, **Orthogonal** (se necesitan al menos dos objetos para que uno proyecte sombra sobre otro)



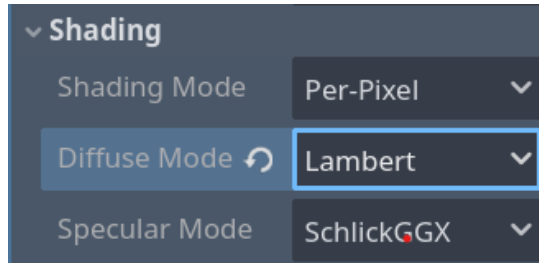
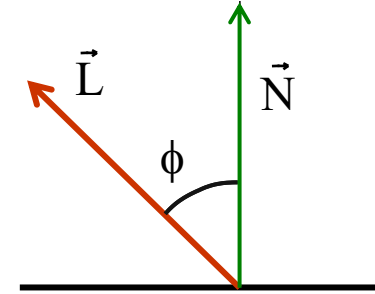
6. Iluminación

! Luz focal **SpotLight3D**



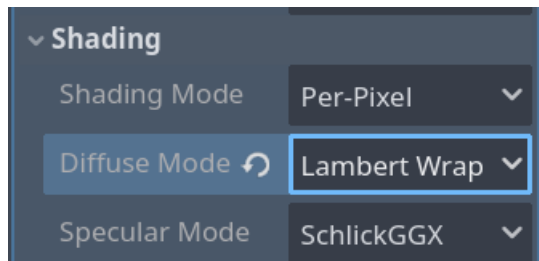
6. Iluminación

! Material, **shading** (parte difusa)



$$I = I_L K_d (\cos \Phi)_+ = I_L K_d (\vec{N} \cdot \vec{L})_+$$

$$(x)_+ = \max(0, x)$$



$$I = I_L K_d \left(\frac{\vec{N} \cdot \vec{L} + w}{1 + w} \right)_{[0,1]}$$

$$(x)_{[0,1]} = \text{clamp}(x, 0, 1) = \min(\max(x, 0), 1)$$

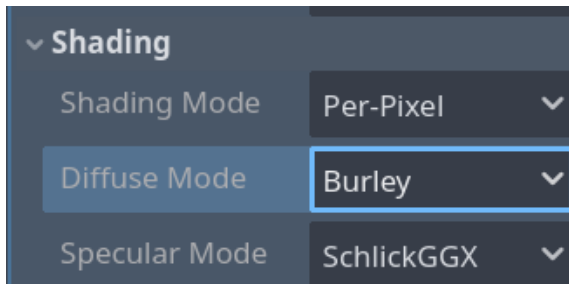
$w=0.3$, valor por defecto



6. Iluminación

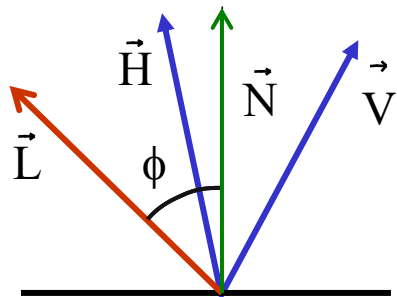
! Material, **shading** (parte difusa)

- El modelo de **Burley** (2012) es tanto empírico como basado en la física o PBR (Physically Based Rendering)
- Para luces no extensas se tiene la ecuación siguiente:



$$I \propto I_L (f_d + f_s)$$

$$f_d = \frac{k_d}{\pi} \left(1 - (F_{D90} - 1)(1 - \vec{N} \vec{L})^5 \right) \times \left(1 - (F_{D90} - 1)(1 - \vec{N} \vec{V})^5 \right)$$

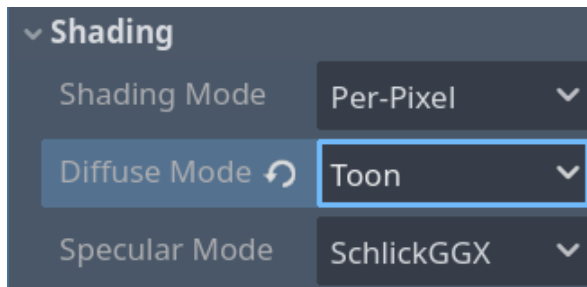


$$F_{D90} = 0.5 + 2 \times \text{rugosidad} \times (\vec{H} \vec{L})^2$$

6. Iluminación

! Material, **shading** (parte difusa)

- El modelo de **Toon** (cartoon o dibujos animados) implementa una reflexión difusa discretizada (Gooch et al. 1998). Puede tener un suavizado opcional



$$I = I_L K_d (\vec{N} \vec{L})_{\text{Toon}}$$

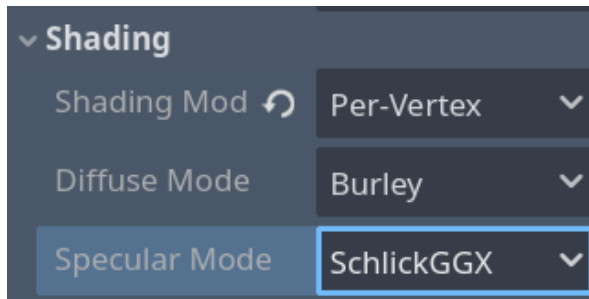
$$(x)_{\text{Toon}} = \frac{x \cdot \text{step}}{\text{step} - 1}, \quad x = \max(0, \vec{N} \vec{L})$$

step=3, valor por defecto

6. Iluminación

! Material, **shading** (parte especular)

- El modelo de **SchlickGGX** implementa una reflexión especular que combina tres distribuciones (Walter et al. 2007): la GGX (Ground Glass with X distribution), la de Fresnel con la aproximación de Schlick y la geometría de Smith



$$I \propto I_L (f_d + f_s)$$

$$f_s = \frac{D_{\text{GGX}}(\vec{H}) F_{\text{Schlick}}(\vec{V}, \vec{H}) G_{\text{Smith}}(\vec{N}, \vec{L}, \vec{V})}{4(\vec{N} \cdot \vec{L})(\vec{N} \cdot \vec{V})}$$

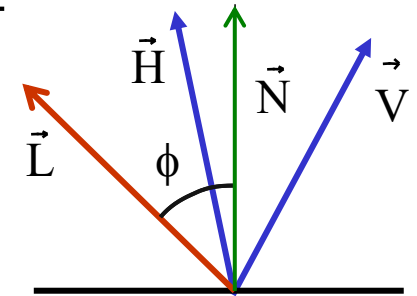


6. Iluminación

! Material, **shading** (parte especular)

- Distribución GGX ("Ground Glass with X distribution o Gaussian Generalized X) o de Trowbridge-Reitz

$$D_{\text{GGX}}(\vec{H}) = \frac{\text{rugosidad}^2}{\pi \left((\vec{N} \cdot \vec{H})^2 (\text{rugosidad}^2 - 1) + 1 \right)^2}$$



- Distribución de Fresnel con la aproximación de Schlick

$$F_{\text{Schlick}}(\vec{V}, \vec{H}) = F_0 + (1 - F_0) \left(1 - (\vec{V} \cdot \vec{H}) \right)^5$$

F_0 reflexión a incidencia normal (rasante), depende del material

$$F_0 = \text{mix}(0.04, \text{albedo}, \text{metallic})$$



6. Iluminación

! Material, **shading** (parte especular)

- Distribución de geometría de Smith o factor de visibilidad (versión simplificada)

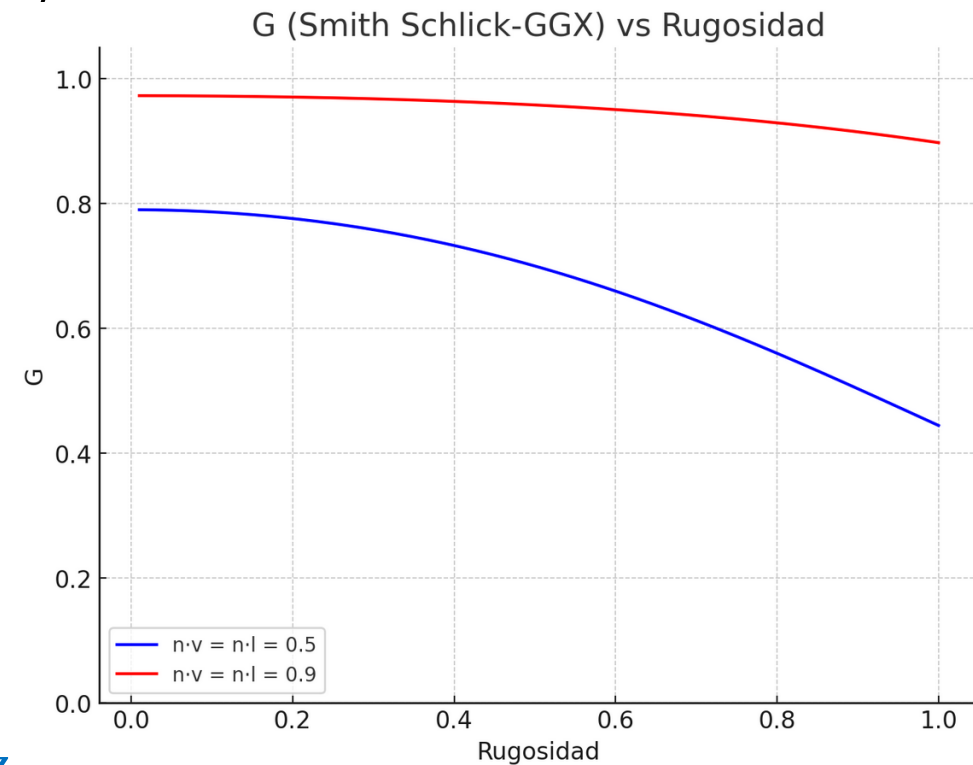
$$G_{\text{Smith}}(\vec{N}, \vec{L}, \vec{V}) = G_1(\vec{N} \vec{L}) G_1(\vec{N} \vec{V})$$

$$G_1(x) = \frac{x}{x(1-k) + k}$$

$$k = \frac{(\text{rugosidad}^2 + 1)^2}{8}$$

$G_1(\vec{N} \vec{L})$ microfacetas visibles a la luz

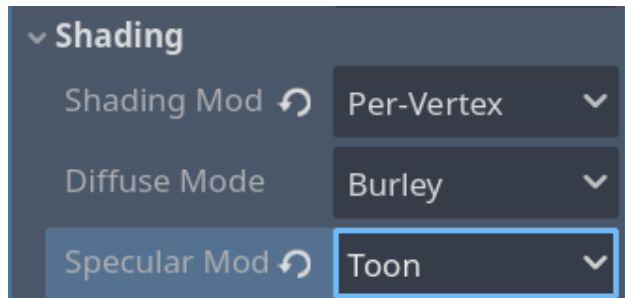
$G_1(\vec{N} \vec{V})$ microfacetas visibles al observador



6. Iluminación

! Material, **shading** (parte especular)

- El modelo de **Toon** se calcula a partir del modelo de **Phong-Blinn** que se discretiza

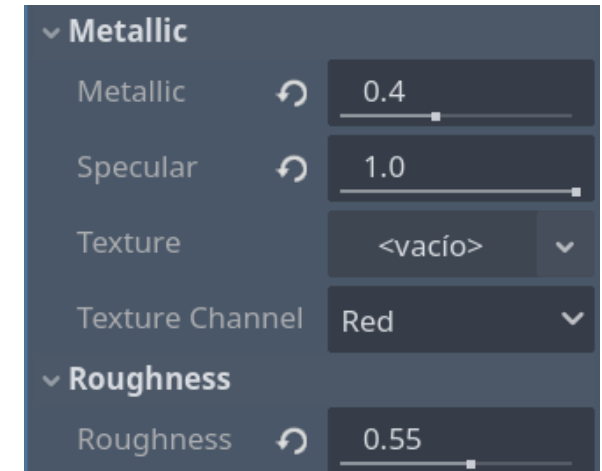
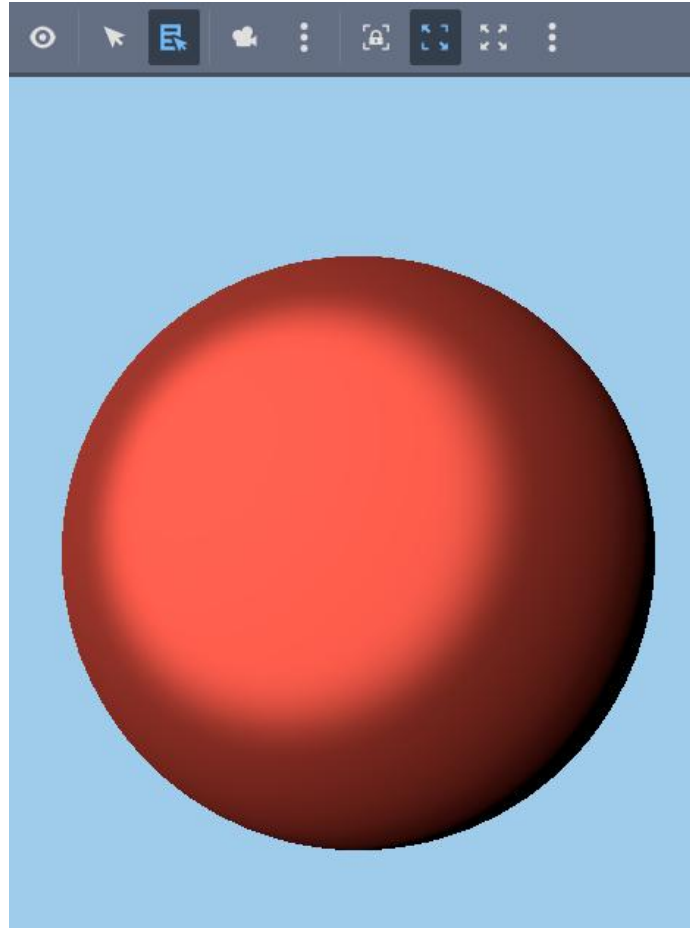
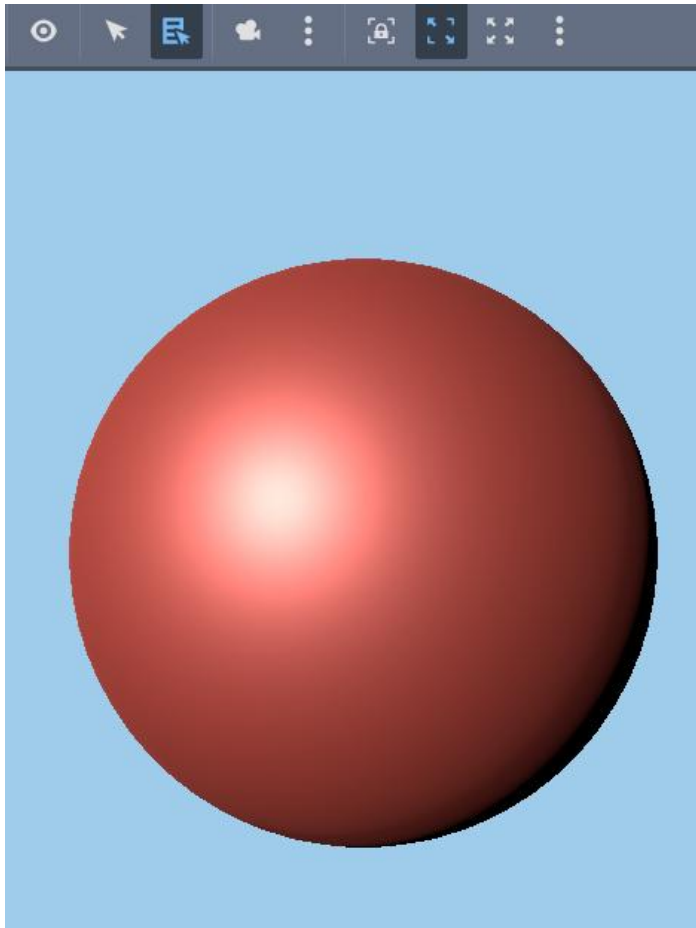


$$I_S = I_L K_s \left((\vec{N} \cdot \vec{H})^n \right)_{\text{Toon}}$$

$$(x)_{\text{Toon}} = \frac{x.\text{step}}{\text{step}-1}, \quad x = \max\left(0, (\vec{N} \cdot \vec{H})^n\right)$$

6. Iluminación

! Material, **shading** (parte especular)



6. Iluminación

! Base de datos de valores de materiales y luces
<https://physicallybased.info/>

PHYSICALLYBASED

A database of physically based values for CG artists

Engine: **Godot** Color Space: **sRGB** Light Units: **Photometric**

Search:

Materials

Name	Color	Specular	Density
Aluminum	0.962 0.965 0.965	-	2700 kg/m3

Banana

Organic



Shading

Albedo: 0.818 0.756 0.367

Metallic: 0

Misc

Relative Luminance: 0.523

References

[Jakubiec, J. A., "Spectral Materials Database", \(Accessed:...](#)



7. Texturas

- ! Especificar la correspondencia entre las coordenadas de la imagen usada como textura y los vértices de los objetos.
- ! Las coordenadas se nombran como UV y están normalizadas

`extends Node3D`

`# cargar imagen de textura`

`@export var imagen: Texture2D`

`...`

`var st = SurfaceTool.new()`

`st.begin(Mesh.PRIMITIVE_TRIANGLES)`

`# primer triángulo`

`st.set_normal (Vector3(0, 0, 1))`

`st.set_uv (Vector2(0, 0))`

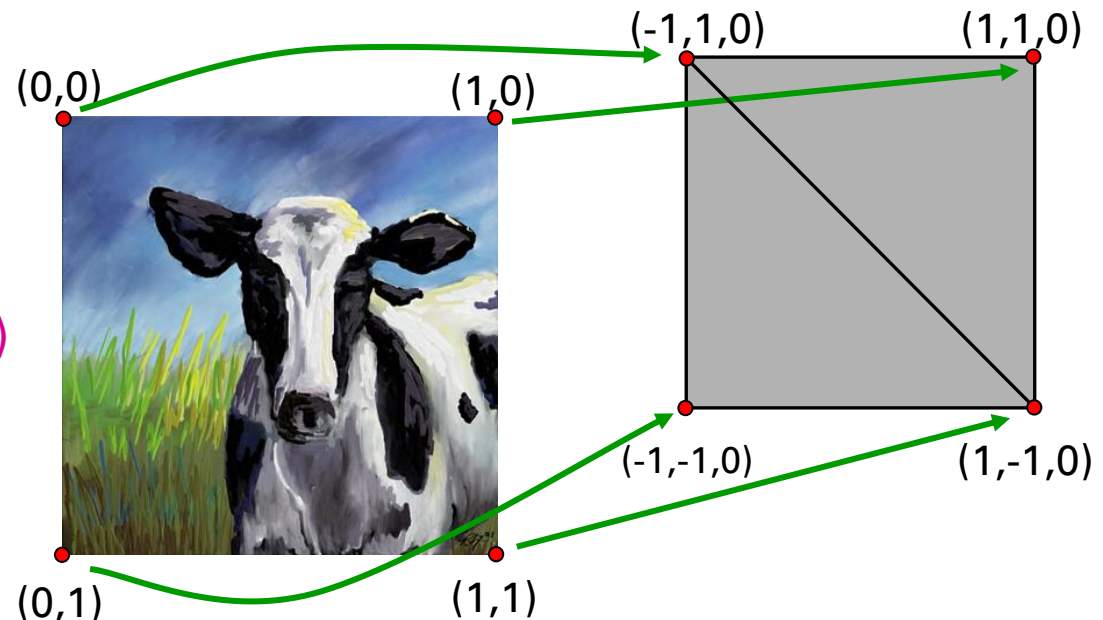
`st.add_vertex (Vector3(-1, 1, 0))`

`st.set_uv (Vector2(1, 1))`

`st.add_vertex (Vector3(1, -1, 0))`

`st.set_uv (Vector2(0, 1))`

`st.add_vertex (Vector3(-1, -1, 0))`

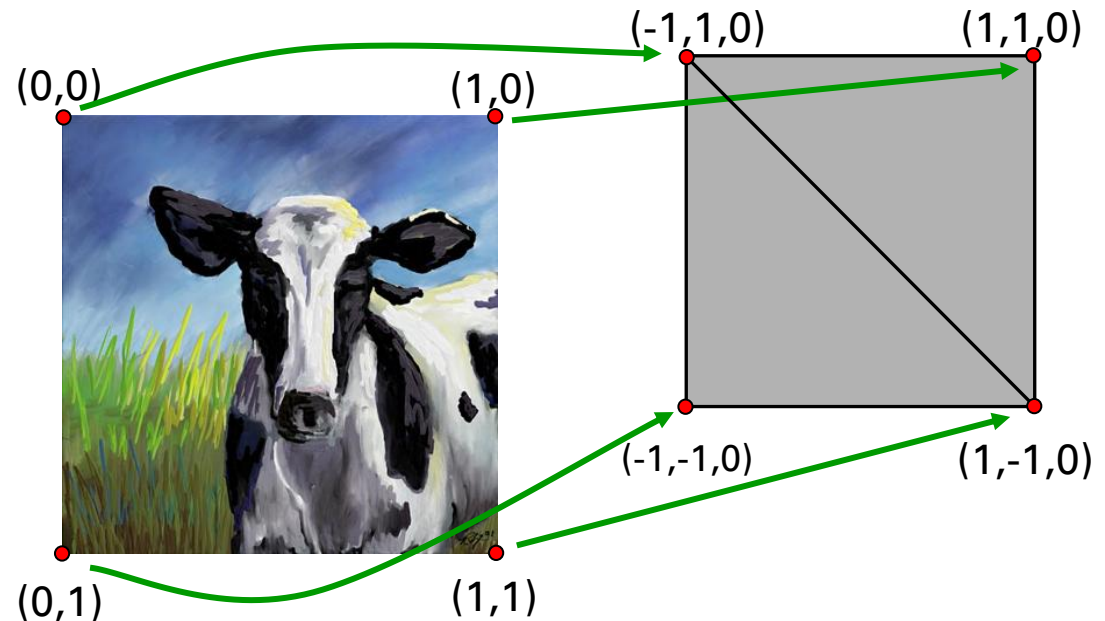


7. Texturas

! Continuación ejemplo.

```
# segundo triángulo triángulo
st.set_normal (Vector3(0, 0, 1))
st.set_uv (Vector2(0, 0))
st.add_vertex (Vector3(-1, 1, 0))
st.set_uv (Vector2(1, 0))
st.add_vertex (Vector3(1, 1, 0))
st.set_uv (Vector2(1, 1))
st.add_vertex (Vector3(1, -1, 0))

var mesh = st.commit()
var mi = MeshInstance3D.new()
mi.mesh = mesh
# crear material con textura
var mat = StandardMaterial3D.new()
mat.albedo_texture = imagen
mat.uv1_offset = Vector3.ZERO
mi.material_override = mat
add_child(mi)
```



7. Texturas

! Malla creada con **ArrayMesh**. Listas de vértices y caras. Malla aislada de la visualización. Añadiendo función para texturas

extends Node3D

función para crear un sólido visualizado con iluminación plana y textura

**static func malla_textura (vértices: Array, caras: Array,
 coord_tex: Array)->ArrayMesh:**

```
var st = SurfaceTool.new()
st.begin(Mesh.PRIMITIVE_TRIANGLES)
for i in caras.size():
```

```
# se accede a los vértices y coordenadas de textura usando como índices
```

```
# los valores contenidos en la lista de caras
```

```
    st.set_normal (Plane(vértices[caras[i][0]], vértices[caras[i][1]],  
                          vértices[caras[i][2]]).normal)
```

```
    for j in range(0, 3):
        st.set_uv( coord_tex[caras[i][j]] )
        st.add_vertex( vértices[caras[i][j]] )
```

```
return st.commit()
```



3. Objetos (mallas)

- ! Malla creada con **ArrayMesh**. Listas de vértices y caras. Malla aislada de la visualización (ejemplo octaedro)

```
extends Node3D # se crea en un node_3D
```

```
@export var imagen: Texture2D
```

```
...
```

```
# lista de vértices
```

```
...
```

```
# lista de caras
```

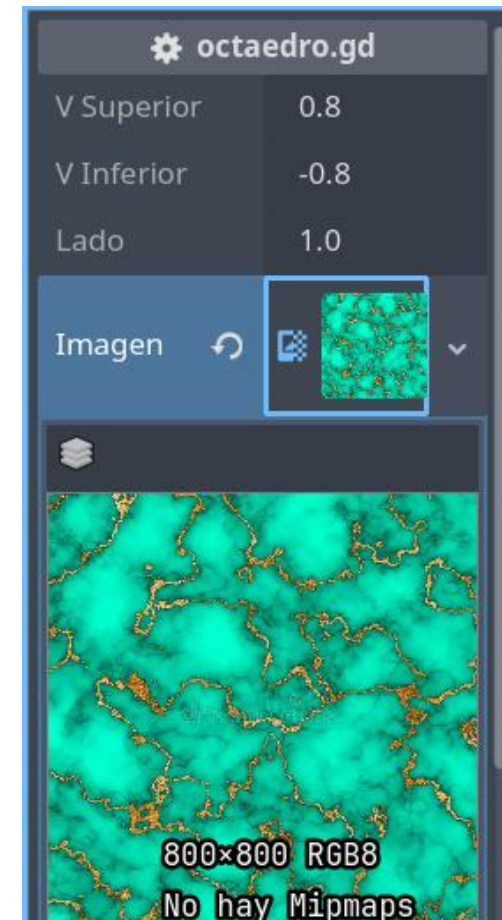
```
...
```

```
# lista colores
```

```
...
```

```
# lista coordenadas textura
```

```
var coor_tex= [ Vector2(0.0, 0.0),  
                Vector2(1.0, 0.0),  
                Vector2(1.0, 1.0),  
                Vector2(0.0, 1.0),  
                Vector2(0.5, 0.5),  
                Vector2(0.5, 0.5) ]
```



3. Objetos (mallas)

! Malla creada con **ArrayMesh**. Listas de vértices y caras. Malla aislada de la visualización (ejemplo octaedro)

```
# función para manejar un evento
```

```
func _input(event):
```

```
...
```

```
    if event is InputEventKey and event.pressed:  
        match event.keycode:
```

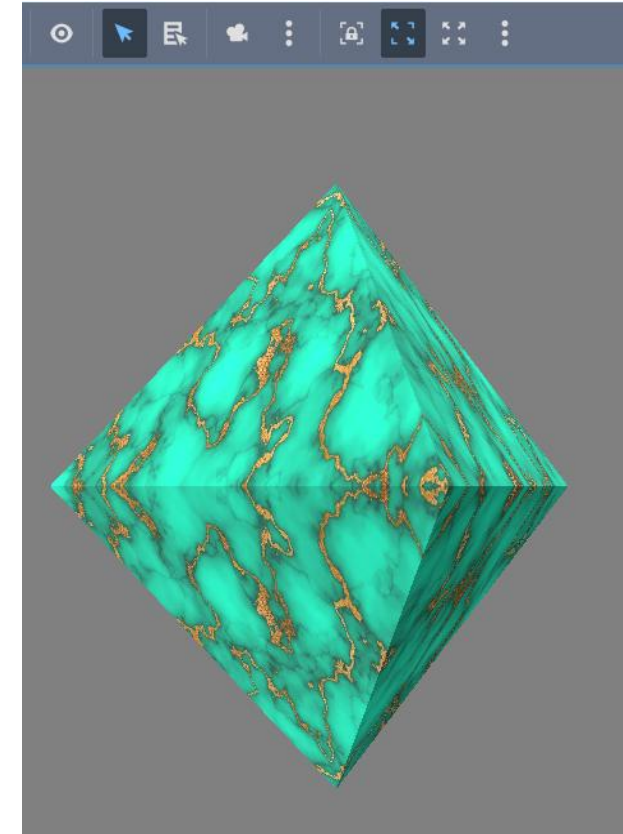
```
            ...
```

```
            KEY_3:
```

```
                octaedro.mesh=modo.malla_textura(vértices,  
                                                    caras, coor_tex)
```

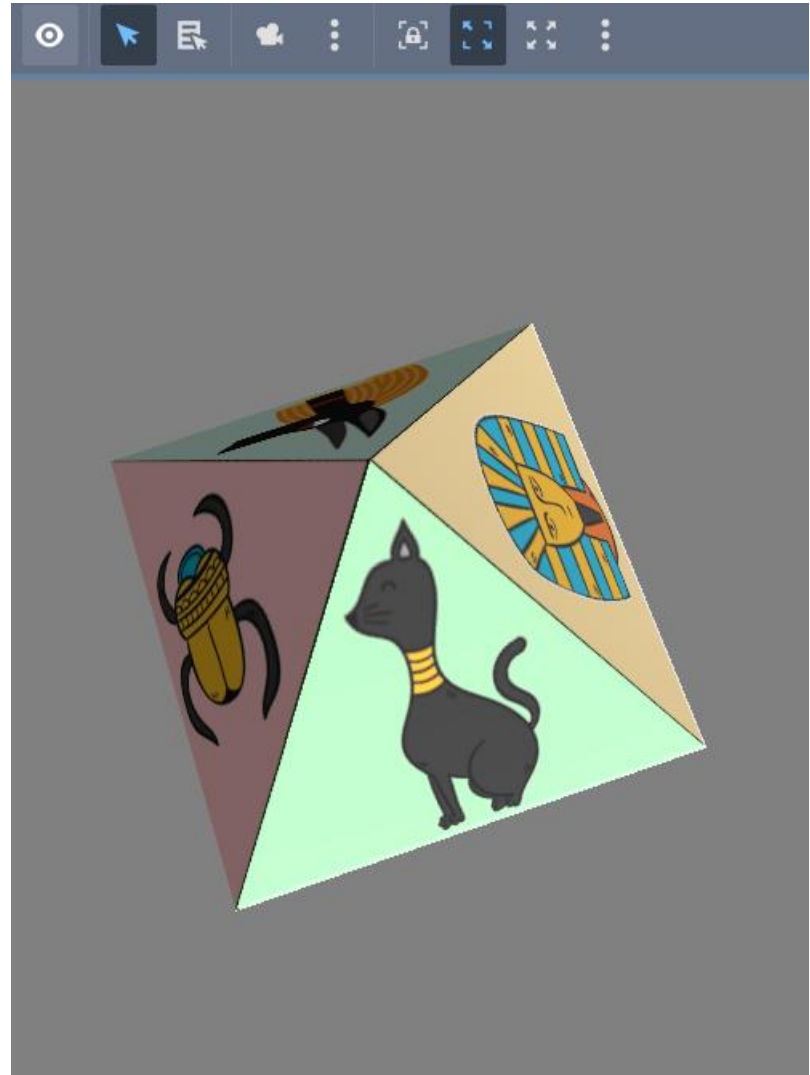
```
                material.albedo_texture = imagen
```

```
                octaedro.material_override=material
```



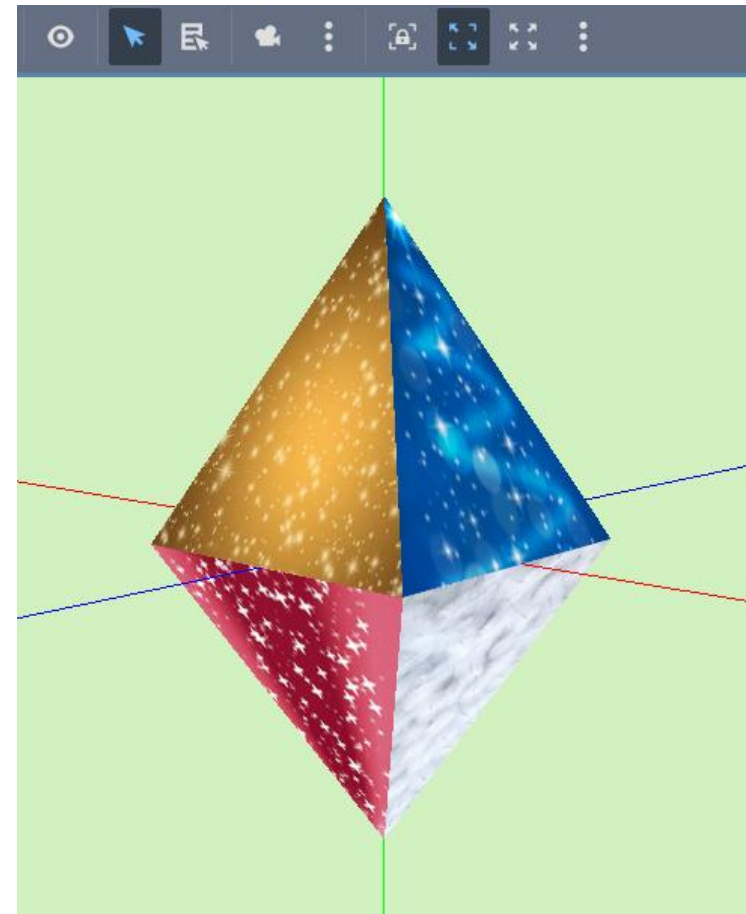
3. Objetos (mallas)

! Agrupando imágenes en una única textura



3. Objetos (mallas)

- ! Agrupando imágenes en una única textura.
 - Si hay que repetir vértices puede ser más manejable usar triángulos aislados en vez de mallas indexadas



3. Objetos (mallas)

! Agrupando imágenes en una única textura. Listas de triángulos aislados. Añadiendo función para texturas

`extends Node3D`

función para crear un sólido visualizado con iluminación plana y textura

`static func malla_textura (caras: Array, coor_tex: Array)->ArrayMesh:`

`var st = SurfaceTool.new()`

`st.begin(Mesh.PRIMITIVE_TRIANGLES)`

`for i in caras.size():`

`# se accede a la lista de triángulos y`

`# coordenadas de textura correspondientes`

`st.set_normal (Plane(caras[i][0], caras[i][1], caras[i][2]).normal)`

`for j in range(0, 3):`

`st.set_uv( coor_tex[i][j] )`

`st.add_vertex( caras[i][j] )`

`return st.commit()`



3. Objetos (mallas)

! Agrupando imágenes en una única textura. Listas de triángulos aislados (ejemplo octaedro)

```
extends Node3D # se crea en un node_3D
```

```
@export var imagen: Texture2D
```

```
...
```

```
# lista de caras
```

```
...
```

```
# lista colores
```

```
...
```

```
# lista coordenadas textura
```

```
var t1=Vector2(0.25, 0.0); var t2=Vector2(0.75, 0.0)
```

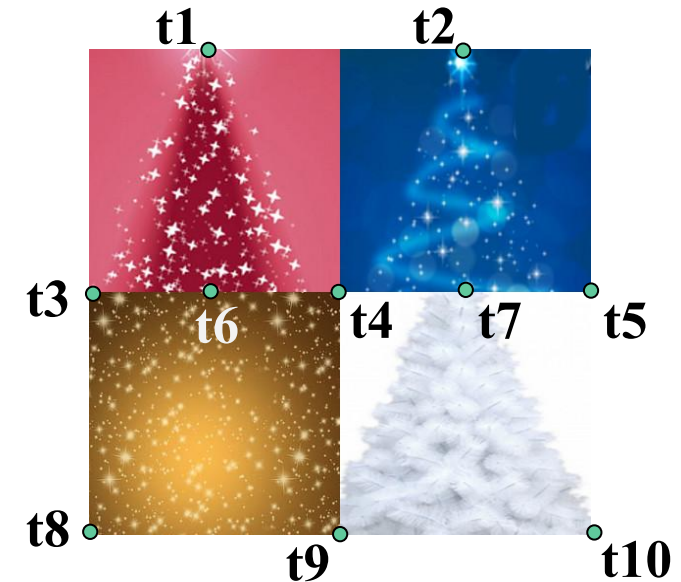
```
var t3=Vector2(0.0, 0.5); var t4=Vector2(0.5, 0.5); var t5=Vector2(1.0, 0.5)
```

```
var t6=Vector2(0.25, 0.5); var t7=Vector2(0.75, 0.5)
```

```
var t8=Vector2(0.0, 1.0); var t9=Vector2(0.5, 1.0); var t10=Vector2(1.0,  
1.0)
```

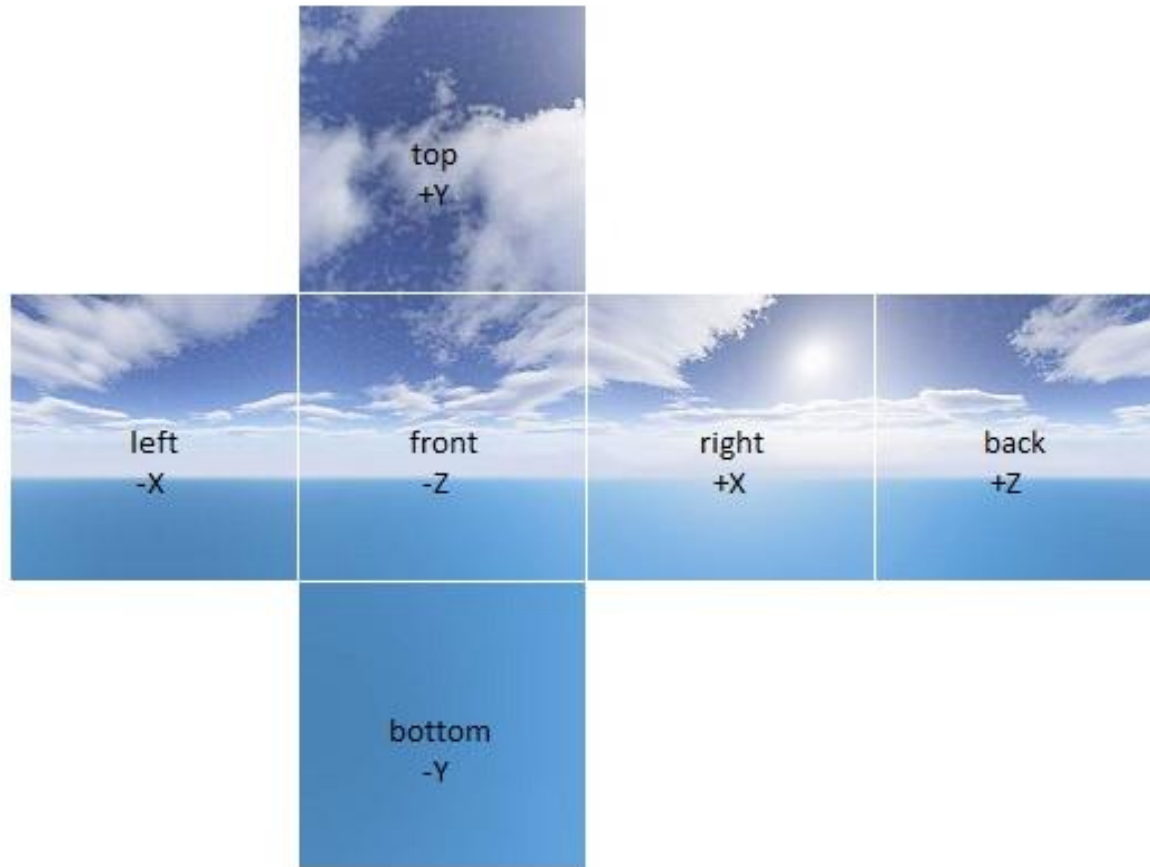
```
var coor_tex= [ [t3, t4, t1], [t4, t5, t2], [t8, t9, t6], [t9, t10, t7],  
[t9, t8, t6], [t9, t10,t7], [t3, t4, t1], [t3, t4, t1]]
```

```
...
```



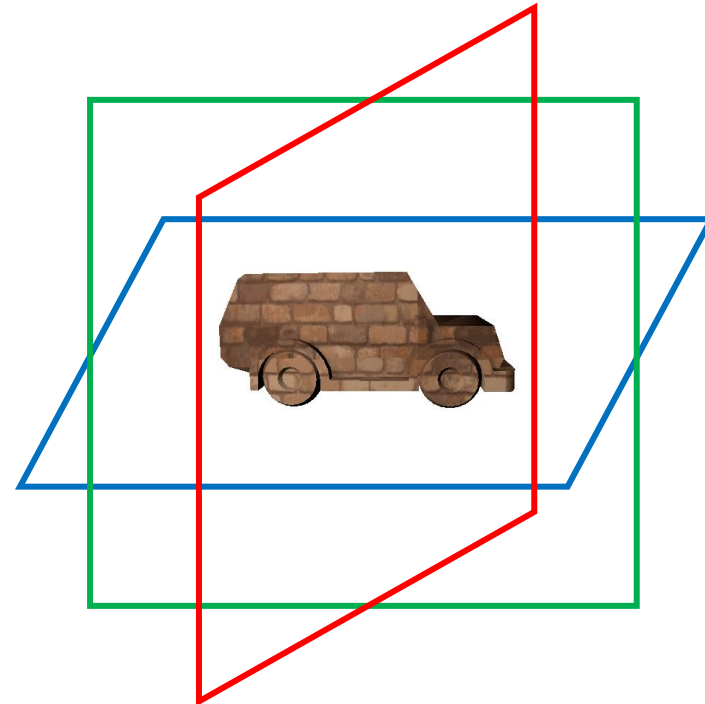
3. Objetos (mallas)

! Agrupando imágenes en una única textura. Skybox



7. Texturas

- ! Generación automática de coordenadas de textura con **triplanar**.
- ! Se generan coordenadas UV aplicando una textura a lo largo de tres planos principales (x, y, z) en coordenadas del espacio del mundo o del objeto
- ! Se usa con el canal UV1



7. Texturas

- ! Generación automática de coordenadas de textura con **triplanar**, para un archivo obj
 - Por defecto se repite según el tamaño del modelo



Textura aplicada



Controla la mezcla entre
las tres proyecciones

7. Texturas

- ! Generación automática de coordenadas de textura con **triplanar**, para un archivo obj



Se calcularían los valores máximos y mínimos de los vértices de la figura

$1/=(\max_x-\min_x);$

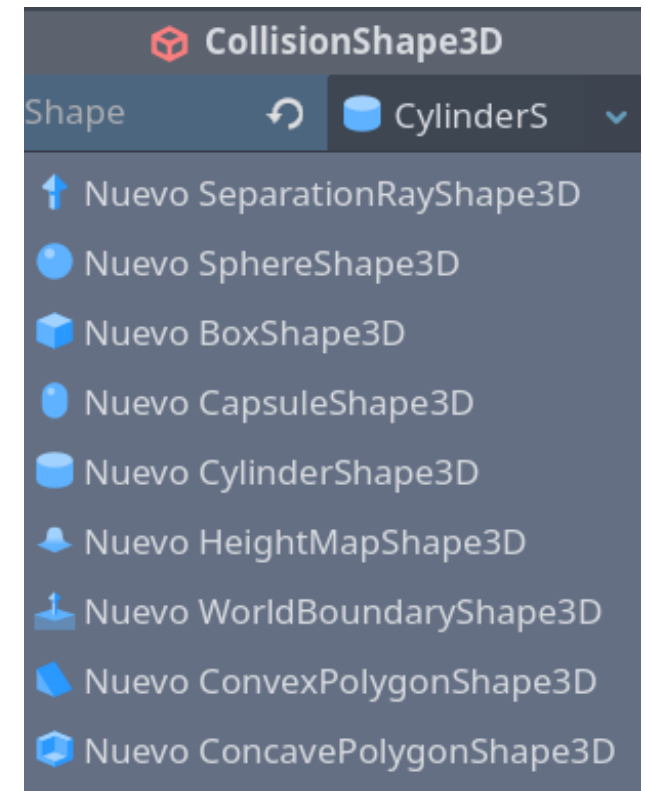
$1/=(\max_y-\min_y);$

....



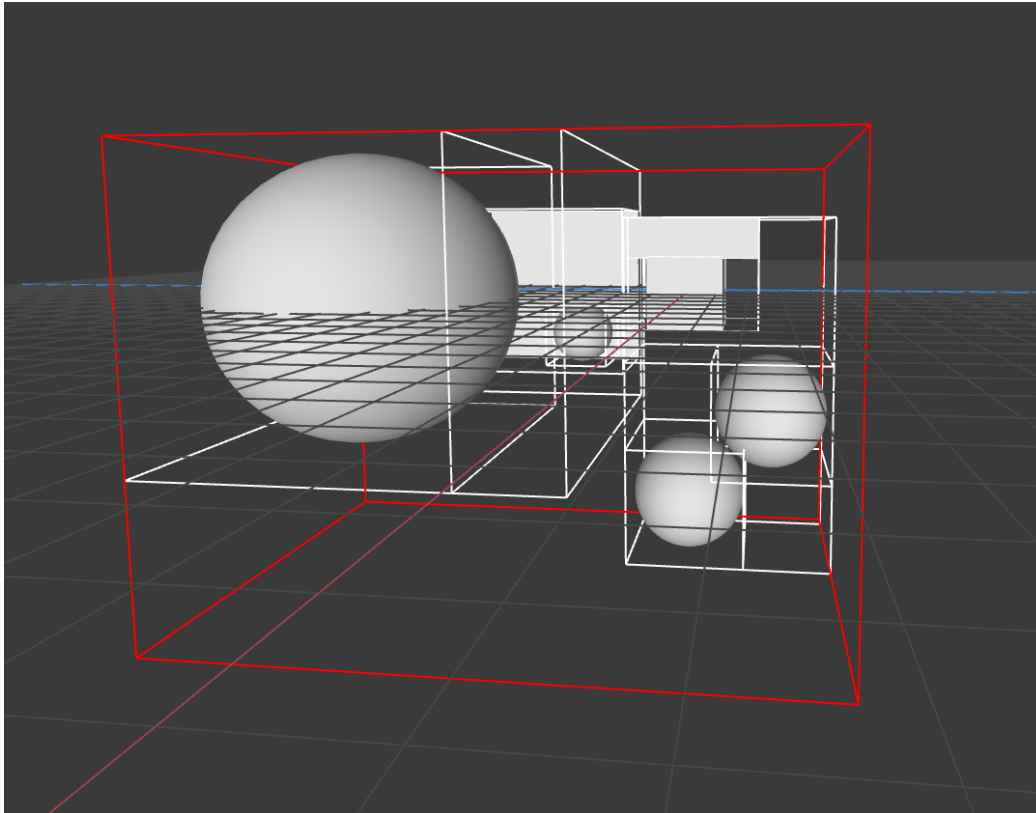
8. Selección

- ! Nodo **CollisionShape3D** define una forma para detectar colisiones envolviendo a un objeto de la escena
- ! Debe ser hijo de un nodo como **StaticBody3D** o **RigidBody3D**
- ! Hay diversas formas disponibles
- ! Con intersección se crea un contacto:
 - posición, normal, penetración.
- ! Para calcular colisiones **Godot** usa:
 - Bounding Volumes,
 - Bounding Volume Hierarchy (BVH)

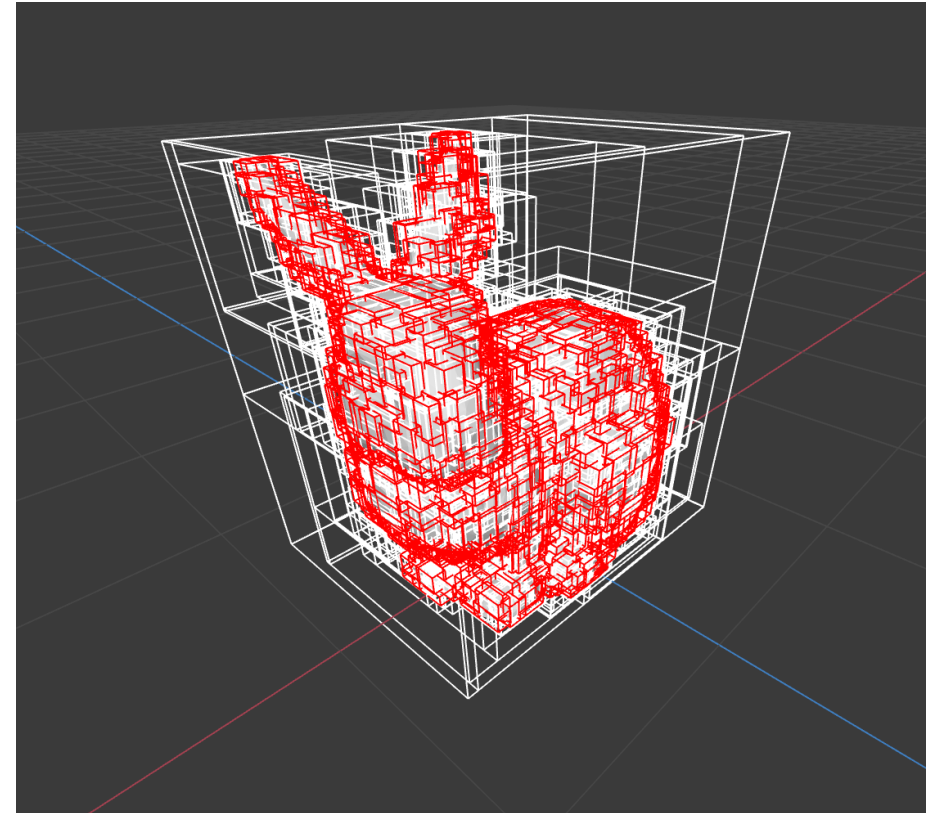


8. Selección

! Bounding Volume Hierarchy (BVH)



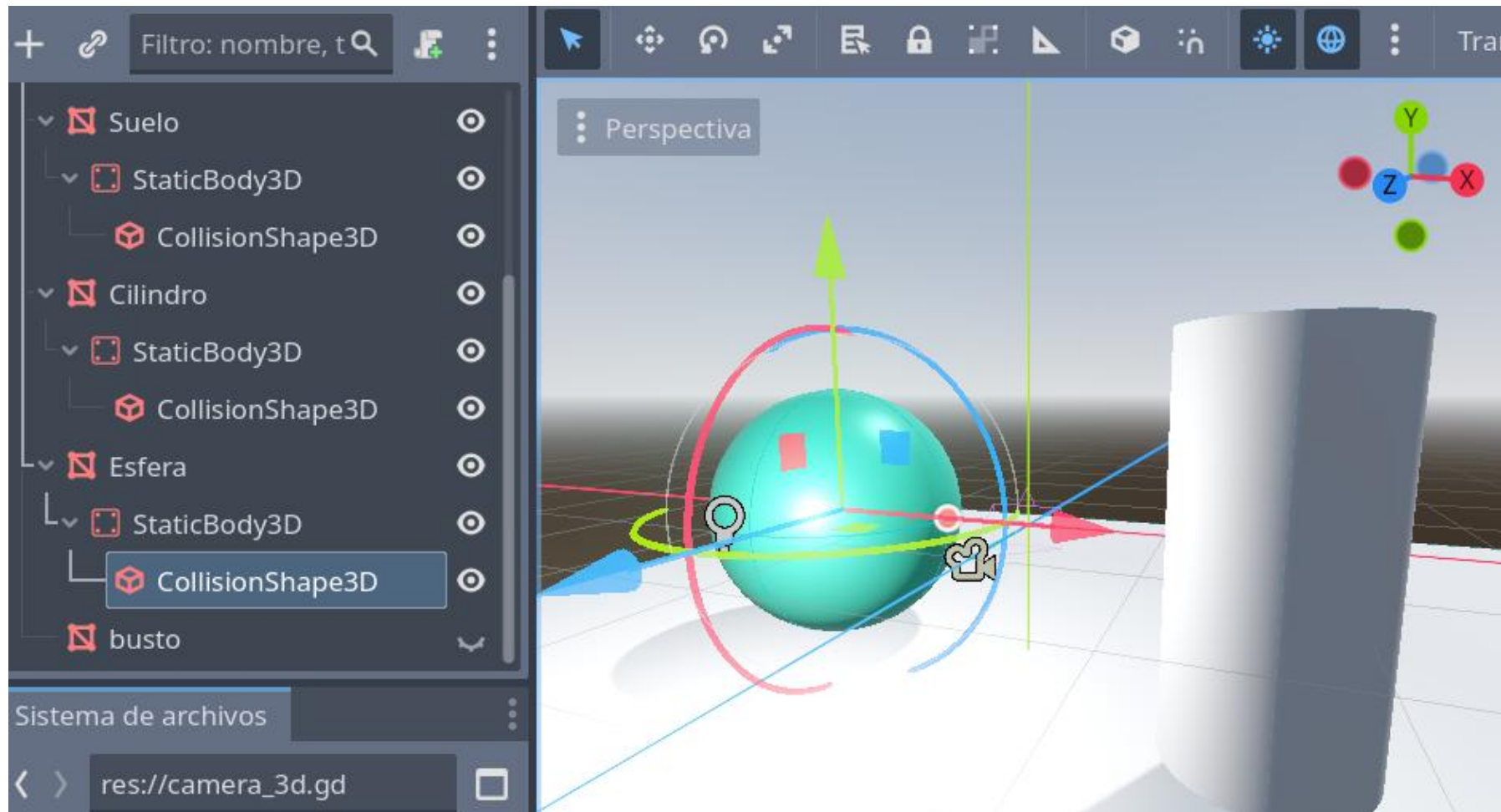
BVH para una escena



BVH para un objeto

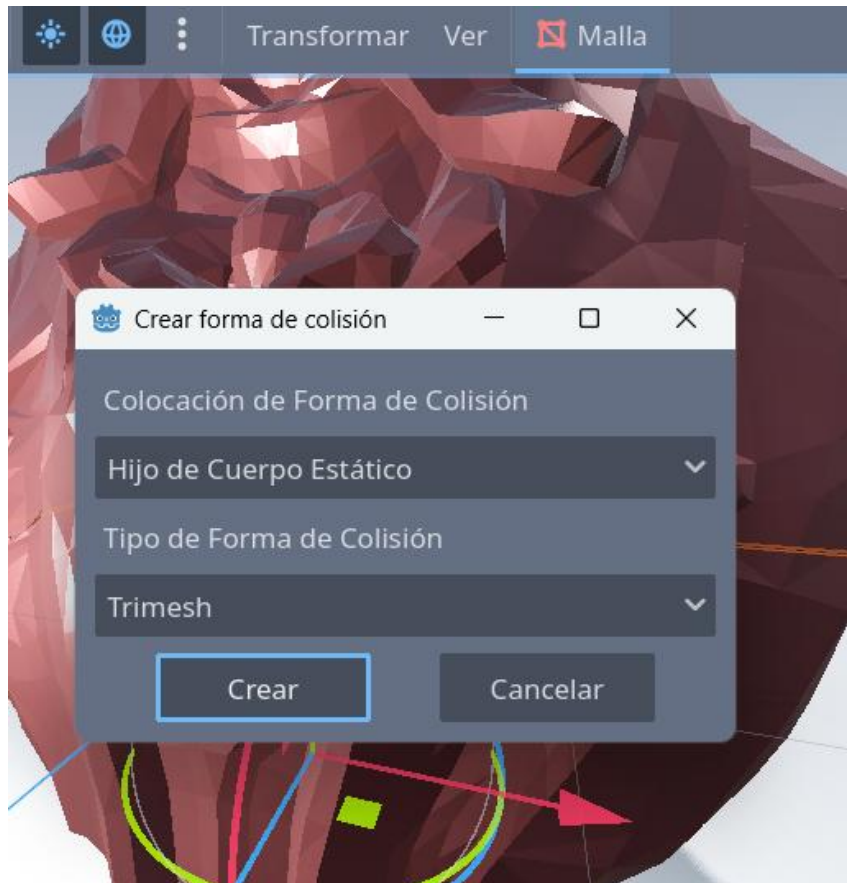
8. Selección

! Nodo **CollisionShape3D** con **MeshInstance3D** para tipo **PrimitiveMesh**. Jerarquía de nodos



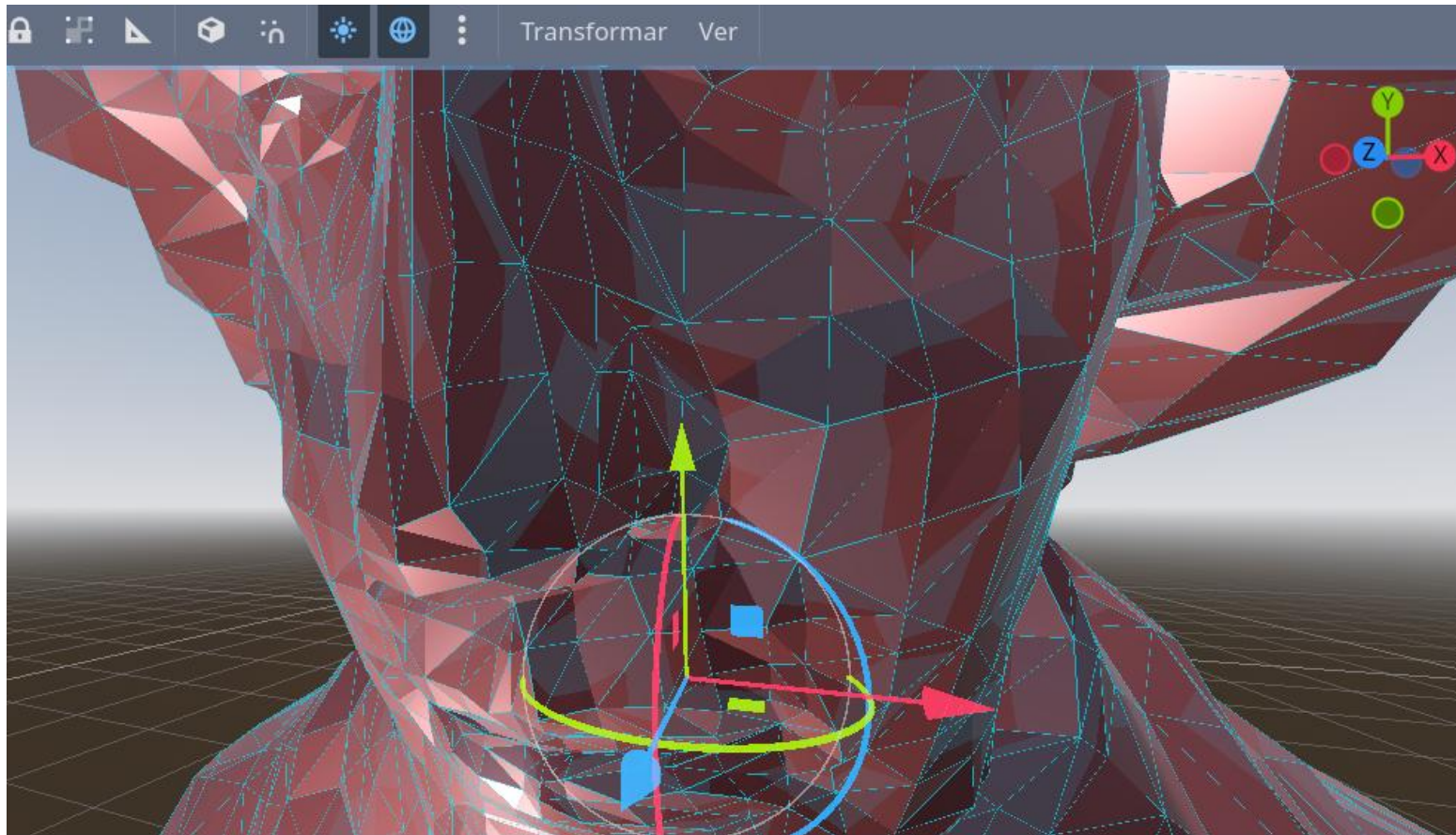
8. Selección

- ! Nodo **CollisionShape3D** para objetos complejos. Se crea con el menú Malla → Creación de Forma de Colisión
 - Dos opciones: **Trimesh** o **Múltiples convexos**



8. Selección

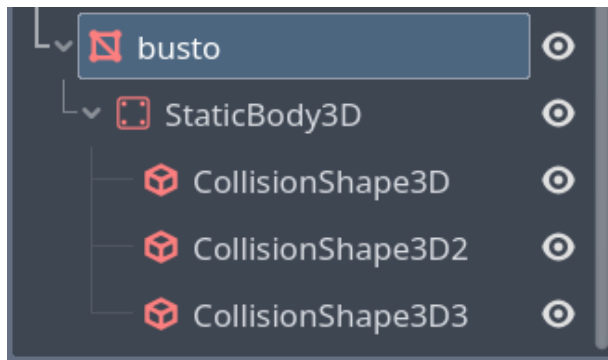
- ! Nodo **CollisionShape3D** objetos complejos. **Trimesh**
- La forma de colisión es exacta al ser el propio objeto
 - Internamente se trata como **ConcavePolygonShape3D**



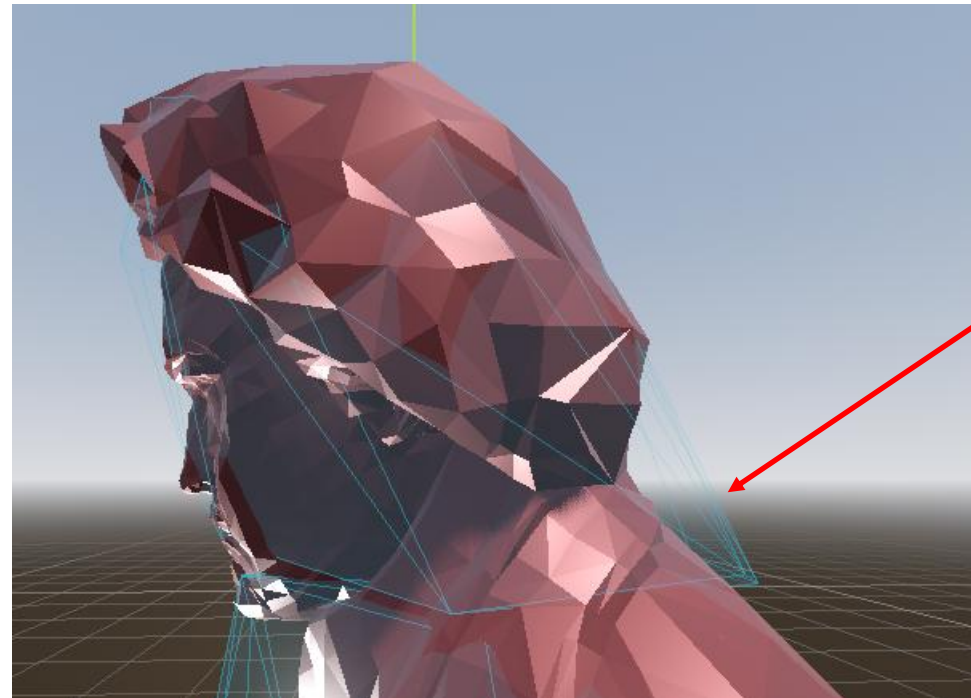
8. Selección

! Nodo **CollisionShape3D** objetos complejos. **Múltiples convexos**

- Crea una aproximación usando varias formas convexas
- Usa el algoritmo VHACD (Volumetric Hierarchical Approximate Convex Decomposition, K. Mamou 2016)



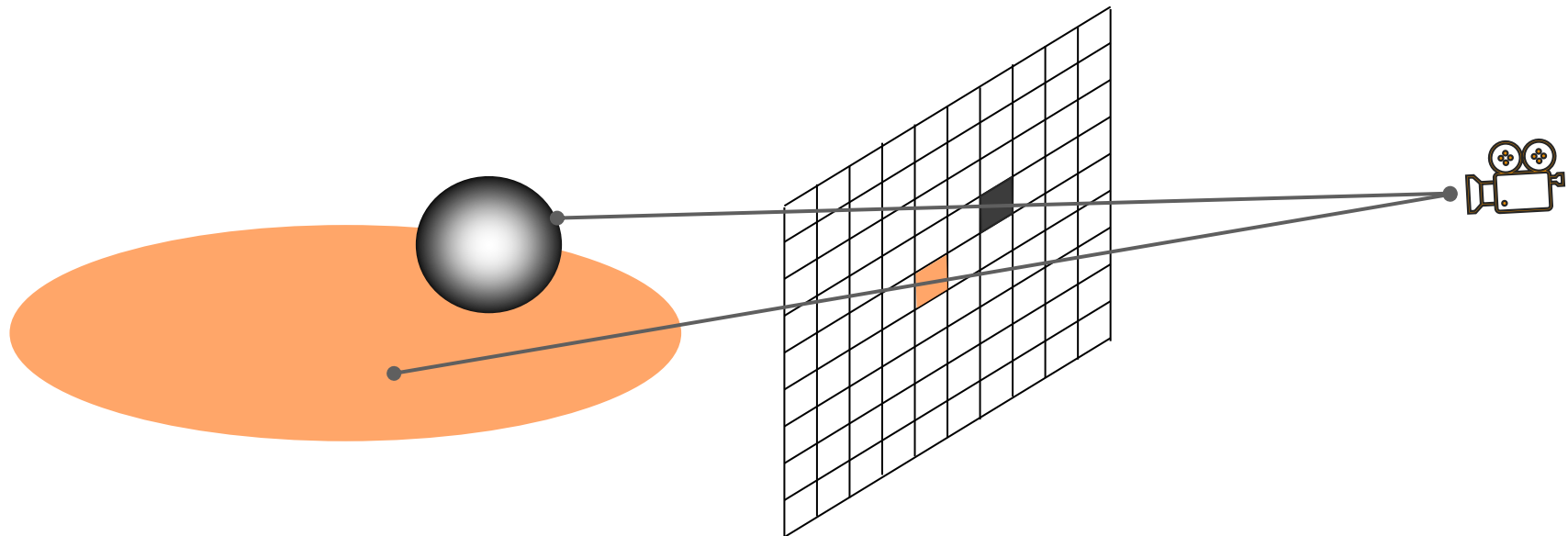
Se han creado tres formas convexas para este busto



8. Selección

! Nodo **RayCast3D**

- Proyecta un rayo sobre la escena desde su origen según una dirección dada y devuelve el objeto más cercano que interseca
- Para hacer selección se hace hijo de un nodo **Camera3D**, la posición origen será la propia de la **Camera3D**
- Ray Casting original (figura)



8. Selección

! Nodo **RayCast3D**

! Propiedades

- **enabled** activa o desactiva **RayCast3D**
- **position_target** (Vector3D), punto final del rayo en coordenadas locales del nodo

! Métodos

- **is_colliding()** true cuando hay colisión
- **force_raycast_update()** obliga a calcular la colisión en cada frame
- **get_collider()** devuelve el objeto intersectado
- **get_collision_point()** devuelve el punto de colisión en coordenadas mundiales
- **get_collision_normal()** normal de la superficie de colisión



8. Selección

- ! Nodo **RayCast3D** como hijo de un nodo **Camera3D**
- ! Código de selección en nodo **Camera3D**. Se lanza un rayo salga del origen de la cámara y que pase por un punto del plano de proyección

```
extends Camera3D
```

```
@onready var ray = $Rayo
```

```
...
```

```
func _process(_delta):
```

```
    # pixel seleccionado
```

```
    var mouse_pos = get_viewport().get_mouse_position()
```

```
    # posición de la cámara, punto inicial del rayo
```

```
    var from = project_ray_origin(mouse_pos)
```

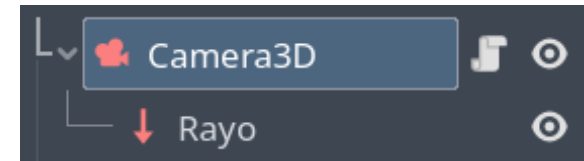
```
    # punto final de rayo
```

```
    var to = project_ray_normal(mouse_pos)*1000
```

```
    ray.global_position = from
```

```
    ray.target_position = ray.to_local(to)
```

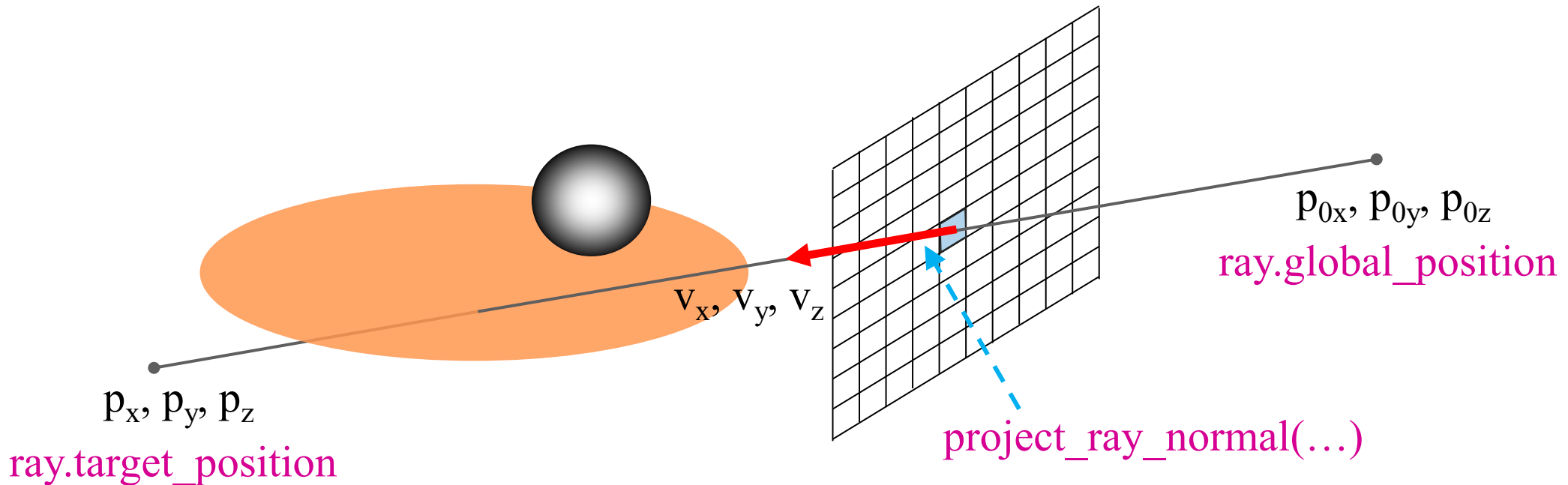
```
    ray.force_raycast_update()
```



8. Selección

! Nodo **RayCast3D** como hijo de un nodo **Camera3D**

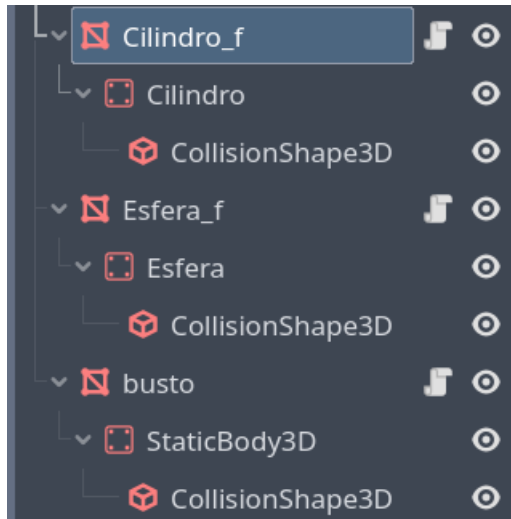
! El rayo es una línea recta paramétrica



$$\begin{cases} p_x = p_{0x} + t v_x \\ p_y = p_{0y} + t v_y \\ p_z = p_{0z} + t v_z \end{cases} \quad t \in \mathbb{R}$$

8. Selección

- ! Cambiando propiedades al seleccionar, recuperándolas al deselectionar (ejemplo color albedo)
- ! Se añade en todos los objetos a seleccionar el script siguiente:



```
extends MeshInstance3D
```

```
# se accederá a estas dos variables en el nodo Camera3D
```

```
var color_albedo
```

```
var cambio = 0
```

```
# si cambio=1 se altera el color inicial del albedo
```

```
func _ready() -> void:
```

```
# se guarda el color inicial del albedo
```

```
var material = get_active_material(0)
```

```
color_albedo = material.albedo_color
```

8. Selección

! Modificaciones en script de Camera3D

```
func _unhandled_input(event):  
    if ( event is InputEventMouseButton and  
        event.pressed and event.button_index == MOUSE_BUTTON_LEFT ):  
        if ray.is_colliding():  
            var objeto = ray.get_collider()  
            # accedemos al padre del objeto StaticBody3D detectado  
            var objeto_p = objeto.get_parent()  
            var material_p = objeto_p.get_active_material(0)  
            if material_p is StandardMaterial3D: # evita errores  
                if objeto_p.get("cambio") != null:  
                    if objeto_p.cambio==0:  
                        material_p.albedo_color = Color(1.0,0.0,0.0) # color selección  
                        objeto_p.cambio=1  
                    else :  
                        material_p.albedo_color = objeto_p.color_albedo  
                        objeto_p.cambio = 0 # recuperamos el color inicial
```



8. Selección

! Cambiando propiedades al seleccionar, recuperándolas al deselectionar (ejemplo color albedo)

